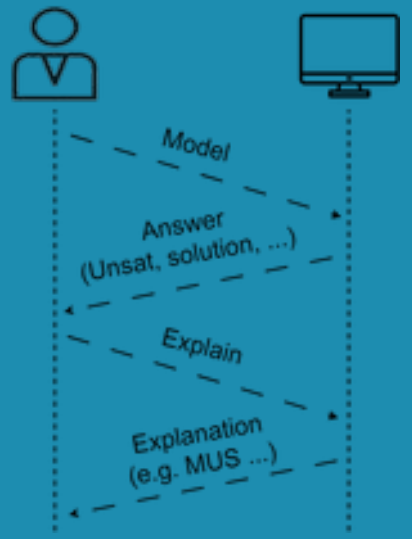
KU LEUVEN

Explainable Constraint Solving



Tias Guns, Ignace Bleukx, Dimos Tsouros

KU Leuven, Belgium UOWM, Greece

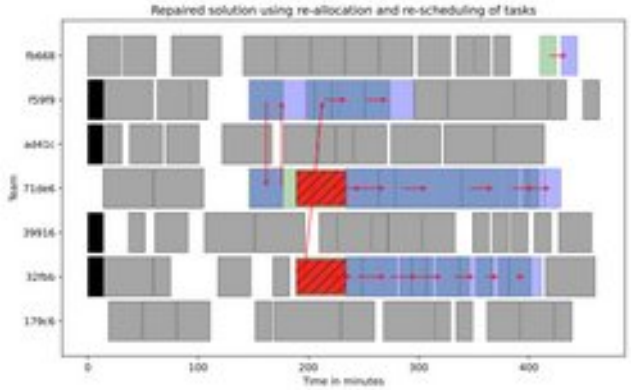
Link to slides and runnable examples:
https://github.com/CPMpy/XCP-explain

European Research Council

Constraint Solving

"Solving discrete, *constrained* optimisation problems"

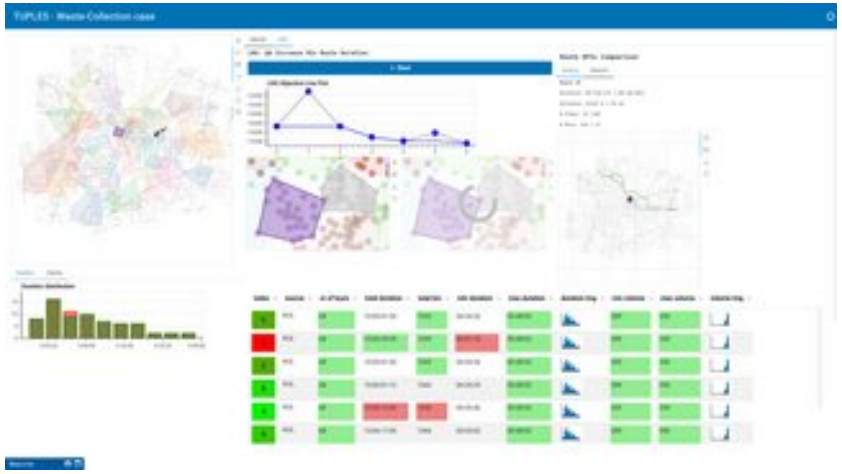
Workforce allocation



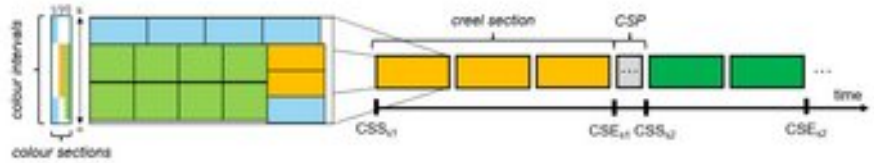
Assembly scheduling and topo. optimisation



Waste collection routing



Industrial weave production planning

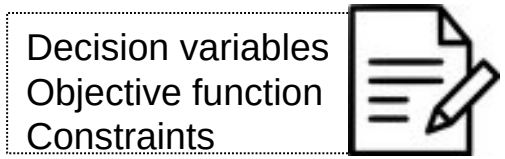


https://cpmpy.cs.kuleuven.be

Declarative paradigm: model + solve

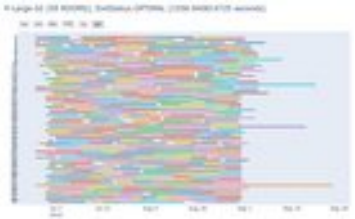
Model + Solve

Decision variables
Objective function
Constraints



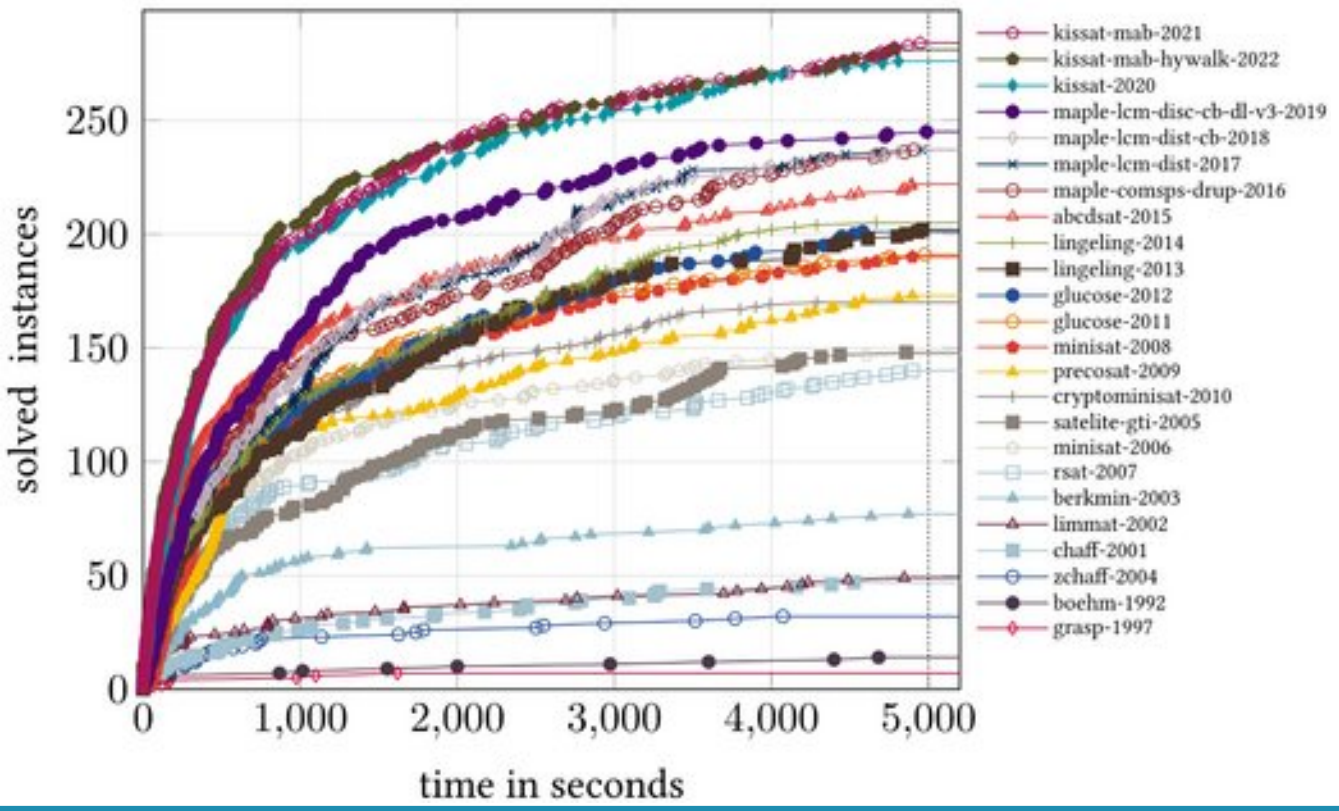






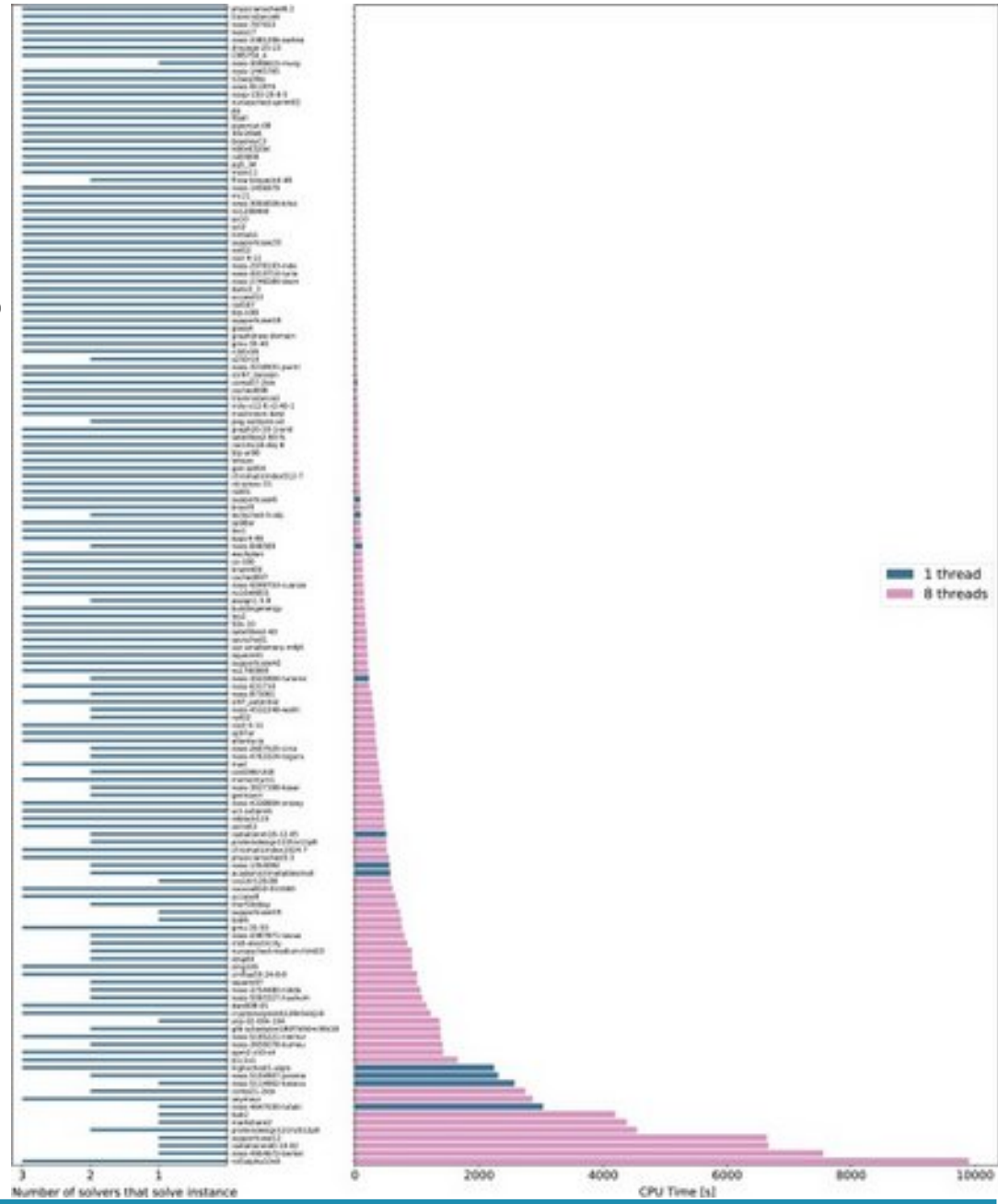
Tremendous progress in solver efficiency!

SAT



[SAT Museum, Bierre et al, 2023]

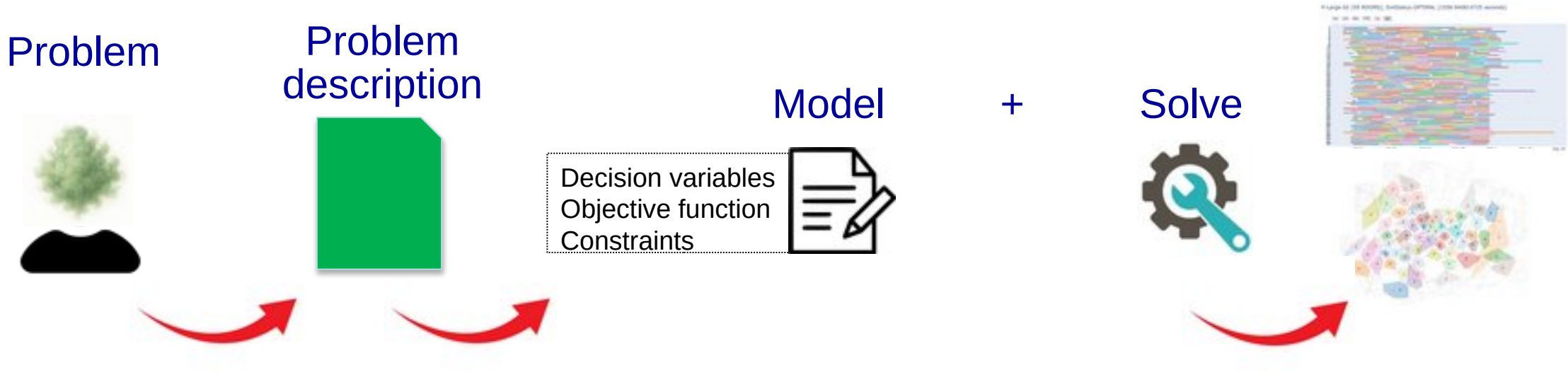
MIP



[Progress in Math. Prog., Koch et al, 2022]

KU LEUVEN

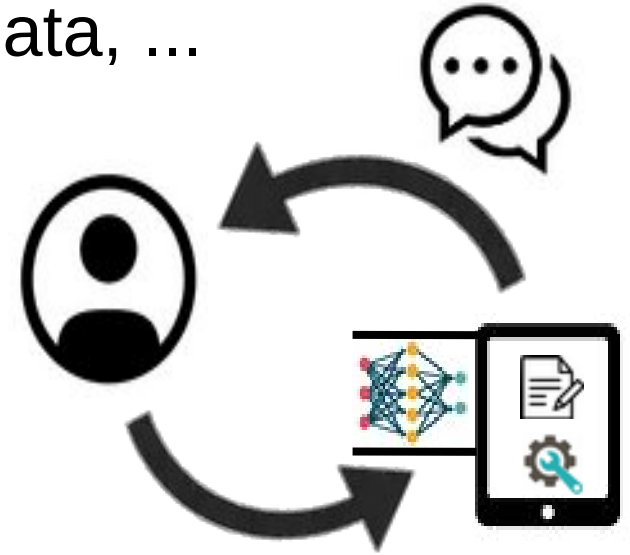
Model + Solve paradigm, in practice



- What if no solution is found?
- What if the user does not like the solution?
- What if the user expected a different solution?

Beyond Model + Solve

- Learning from images, production data, ...
- Learning user preferences
- Explaining constraint solving
- Stateful interaction





CHAT-Opt: Conversational Human-Aware Technology for Optimisation

Trustworthy & Explainable constraint solving

Human-aware AI:

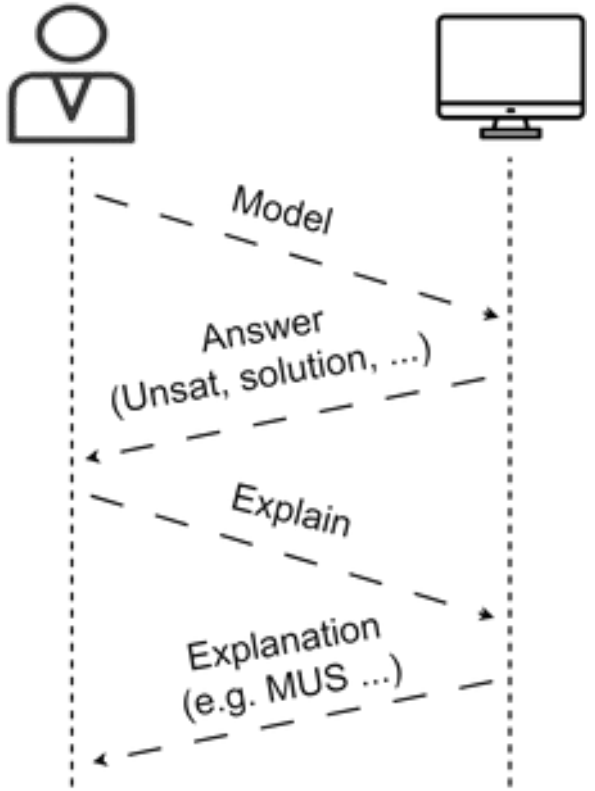
- Respect human agency
- Support users in decision making
- Provide explanations and learning opportunities

Acknowledges that a Model is only an approximation of the problem, that it might result in undesirable solutions.



Explainable Constraint Programming (XCP)

In general, "**Why X?**" (with X a solution or UNSAT)



Explaining constraint 'propagation'?

Concept from Lazy Clause Generation / CP-SAT solvers (Stuckey, 2010)

"every time a propagator determines a domain change of a variable, it records a **clause** that *explains* the domain change."

[Emir's lecture yesterday:]
A single constraint propagator generates a clause to explain a domain change to a solver.

Today we focus on user-oriented explanations, typically involving multiple constraints.

Explainable Constraint Programming (XCP)

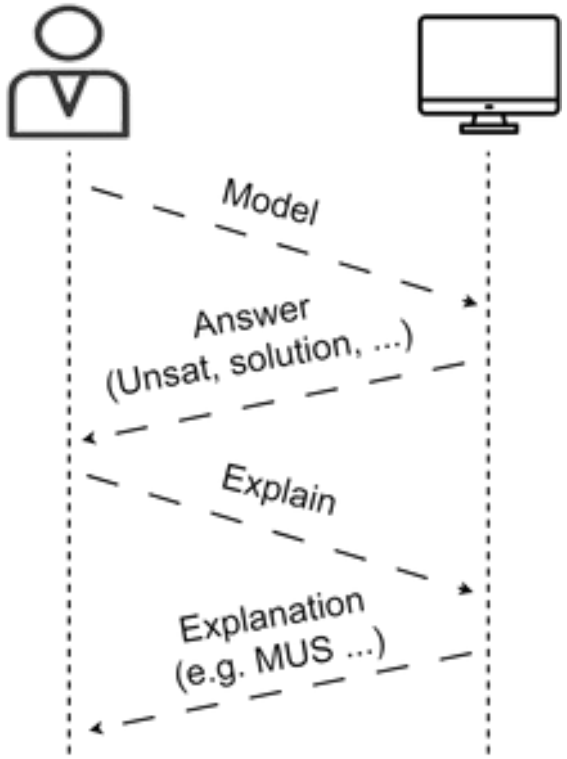
In general, "**Why X?**" (with X a solution or UNSAT)

Deductive explanation:

- *What constraints cause X?*

Counterfactual explanation:

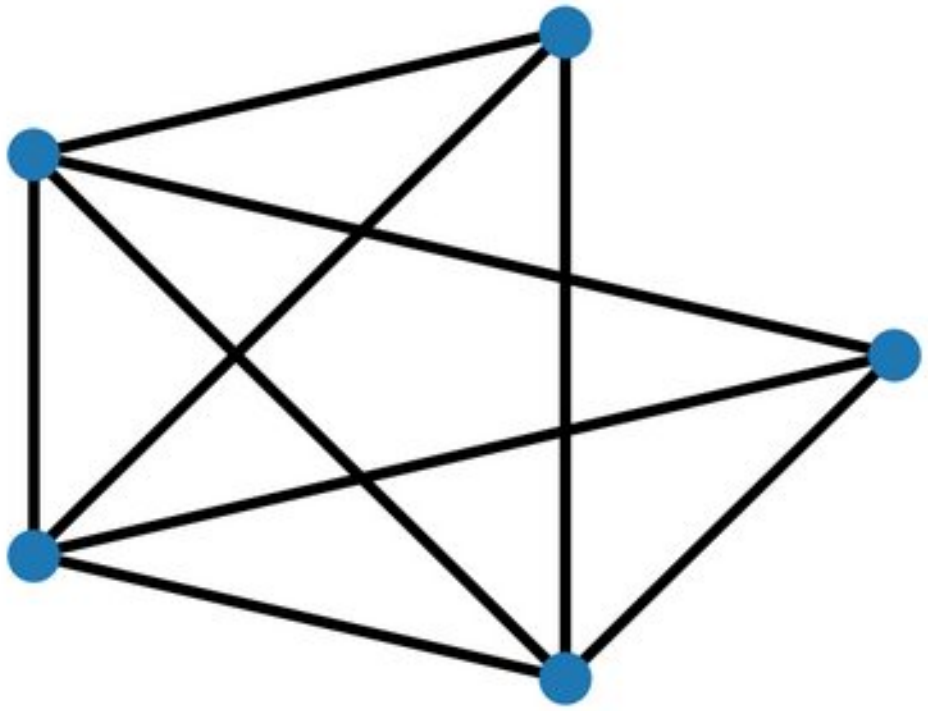
- *What to change if I want Y instead of X?*



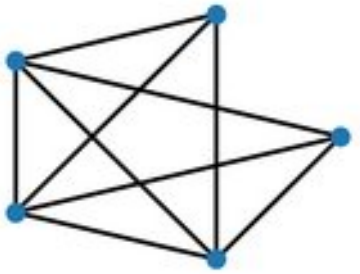
Example XCP interaction

Toy example, graph coloring:

```
In [2]:  G = nx.fast_gnp_random_graph(5, 0.8, seed=0)
         draw(G)
```



Modeling Minimum Graph Coloring as a CP problem



Minimize $\max(X)$

s.t. $X_i \neq X_j$ $\forall (i,j) \in Edges$

$X_i \in \{1, \ldots, max_colors\}$ $\forall\, i \in Nodes$

```python
def graph_coloring(G, max_colors=None):
    n = G.number_of_nodes()
    if max_colors is None:
        max_colors = n  # upper bound: all distinct

    x = cp.intvar(1, max_colors, shape=n, name="x")
    m = cp.Model(
        [x[i] != x[j] for i,j in G.edges()],
        minimize=cp.max(x)
    )
    return m, x
```

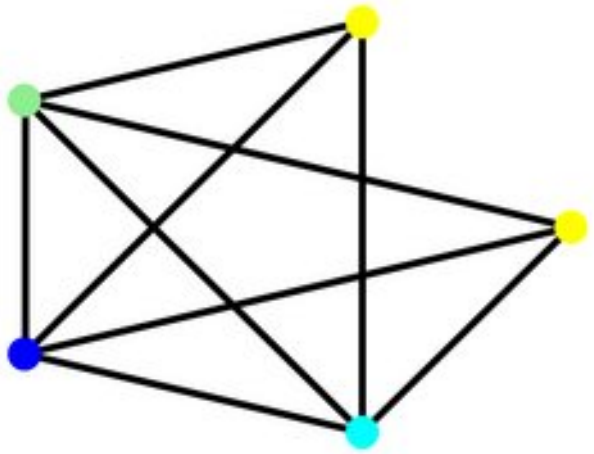
CPMpy

Modeling Minimal Graph Coloring as a CP problem

Lets color the graph...

```
In [4]:  m, nodes = graph_coloring(G, max_colors=None)
         if m.solve():
             print(m.status())
             print(f"Found optimal coloring with {m.objective_value()} colors")
             draw(G, node_color=[cmap[v.value()] for v in nodes])
         else:
             print("No solution found.")
```

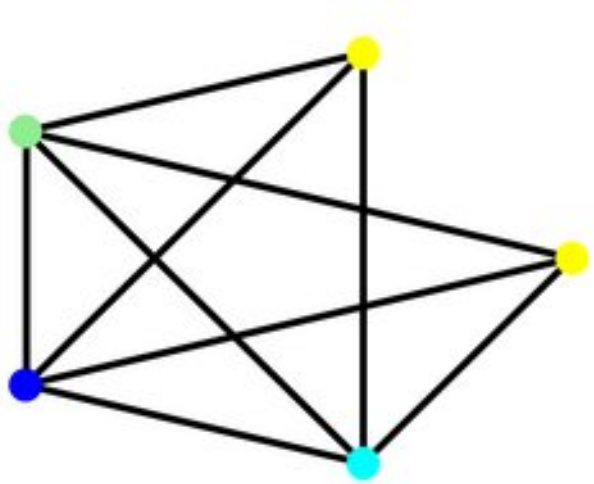
ExitStatus.OPTIMAL (0.003024 seconds)
Found optimal coloring with 4 colors



Example XCP interaction

Why do we need at least 4 colors?

Deductive explanation: pinpoint to constraints *causing* this fact

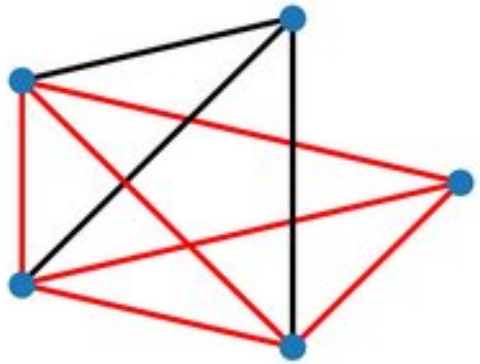


```
In [6]:  from cpmpy.tools.explain import mus

         m, nodes = graph_coloring(G, max_colors=3)  # less than 4?

         if m.solve() is False:
             conflict = mus(m.constraints)  # Minimal Unsatisfiable Subset
             print("UNSAT is caused by the following constraint(s):")
             graph_highlight(G, conflict)
```

UNSAT is caused by the following constraint(s):

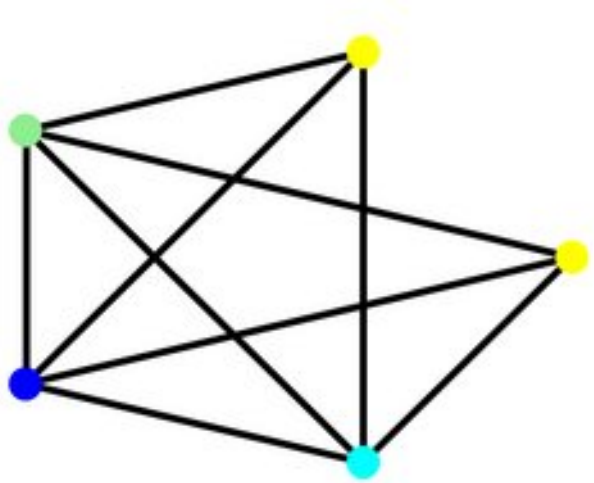


MUS = Minimal Unsatisfiable Subset

Example XCP interaction

Why do we need at least 4 colors?

Counterfactual explanation: pinpoint to constraint *changes* that would allow, e.g. 3 colors

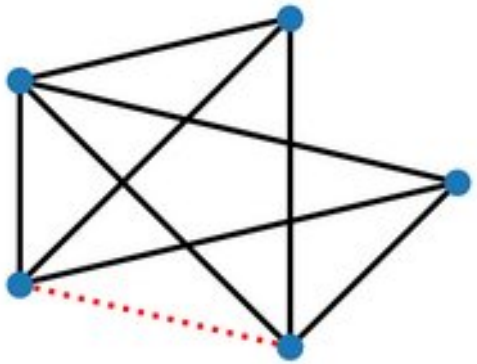


```
In [7]:  from cpmpy.tools.explain import mcs

         m, nodes = graph_coloring(G, max_colors=3)  # less than 4?

         if m.solve() is False:
             corr = mcs(m.constraints)  # Minimal Correction Subset
             print("UNSAT can be resolved by removing the following constraint(s):")
             graph_highlight(G, corr, dotted=True)
```

UNSAT can be resolved by removing the following constraint(s):



MCS = Minimal Correction Subset

Note that an MCS fixes *all* conflicts
(here: breaks both 4-cliques)

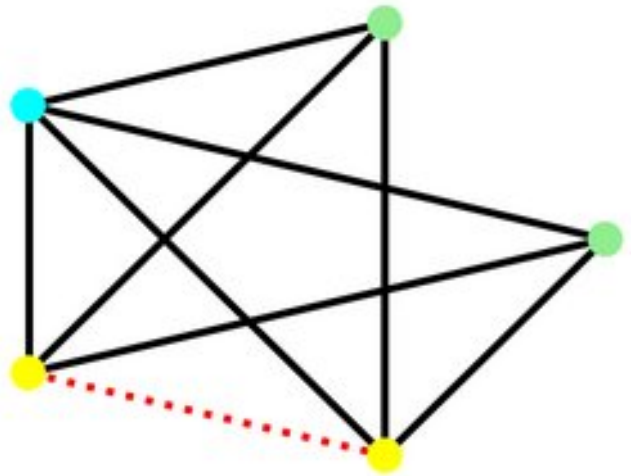
Example XCP interaction

Why do we need at least 4 colors?

Counterfactual explanation: pinpoint to constraint *changes* that would allow, e.g. 3 colors

Can now compute the counterfactual solution:

```
In [8]:  # compute and visualise counter-factual solution
         m2 = cp.Model([c for c in m.constraints if c not in corr])
         m2.solve()
         graph_highlight(G, corr, node_color=[cmap[n.value()] for n in nodes], dotted=True)
```



Explainable Constraint Programming (XCP)

In general, "**Why X?**" (with X a solution or UNSAT)

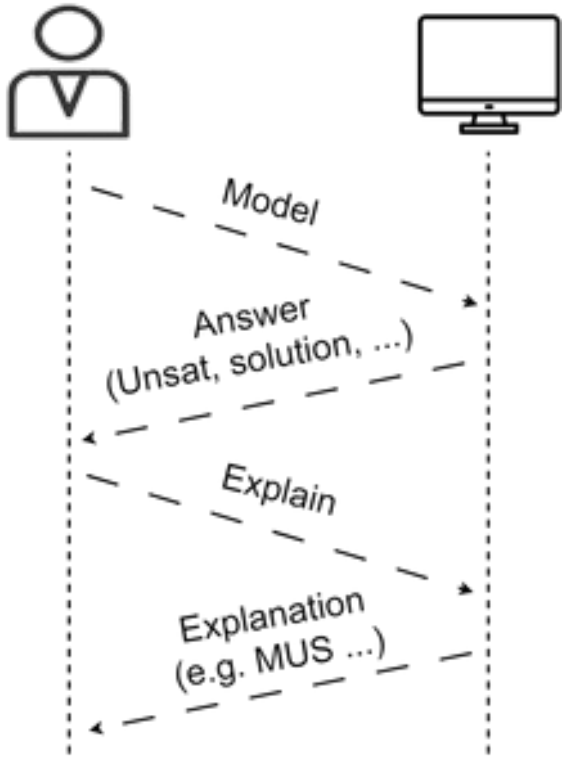
Deductive explanation:

- *What constraints cause X?*

Counterfactual explanation:

- *What to change if I want Y instead of X?*
  (includes feasibility restoration)

Especially useful/needed in application context!



Applications of explanation techniques

"Human-centred feasibility restoration in practice", *Ilankaikone Senthooran*, *Gleb Belov*, *Kevin Leo*, *Michael Wybrow*, *Matthias Klapperstueck*, *Tobias Czauderna*, *Mark Wallace*, *and Maria Garcia De La Banda*. CP 2021.

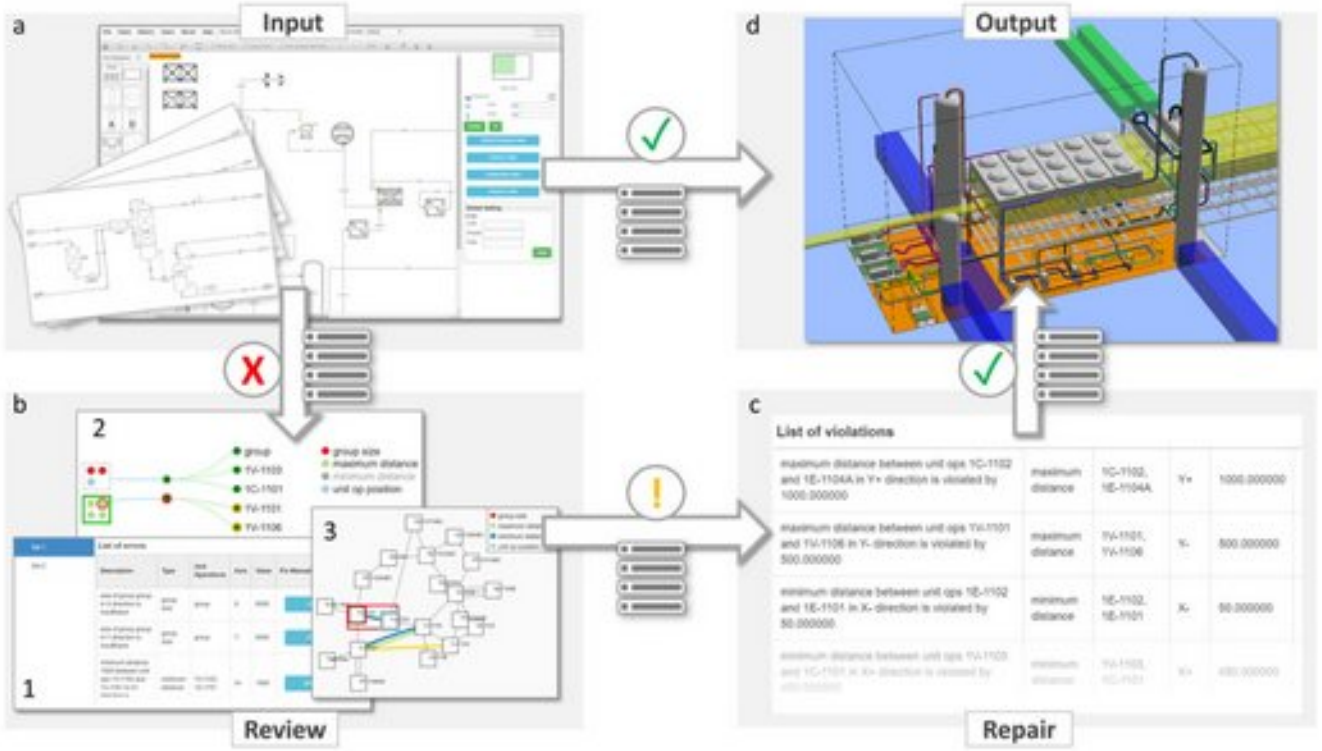
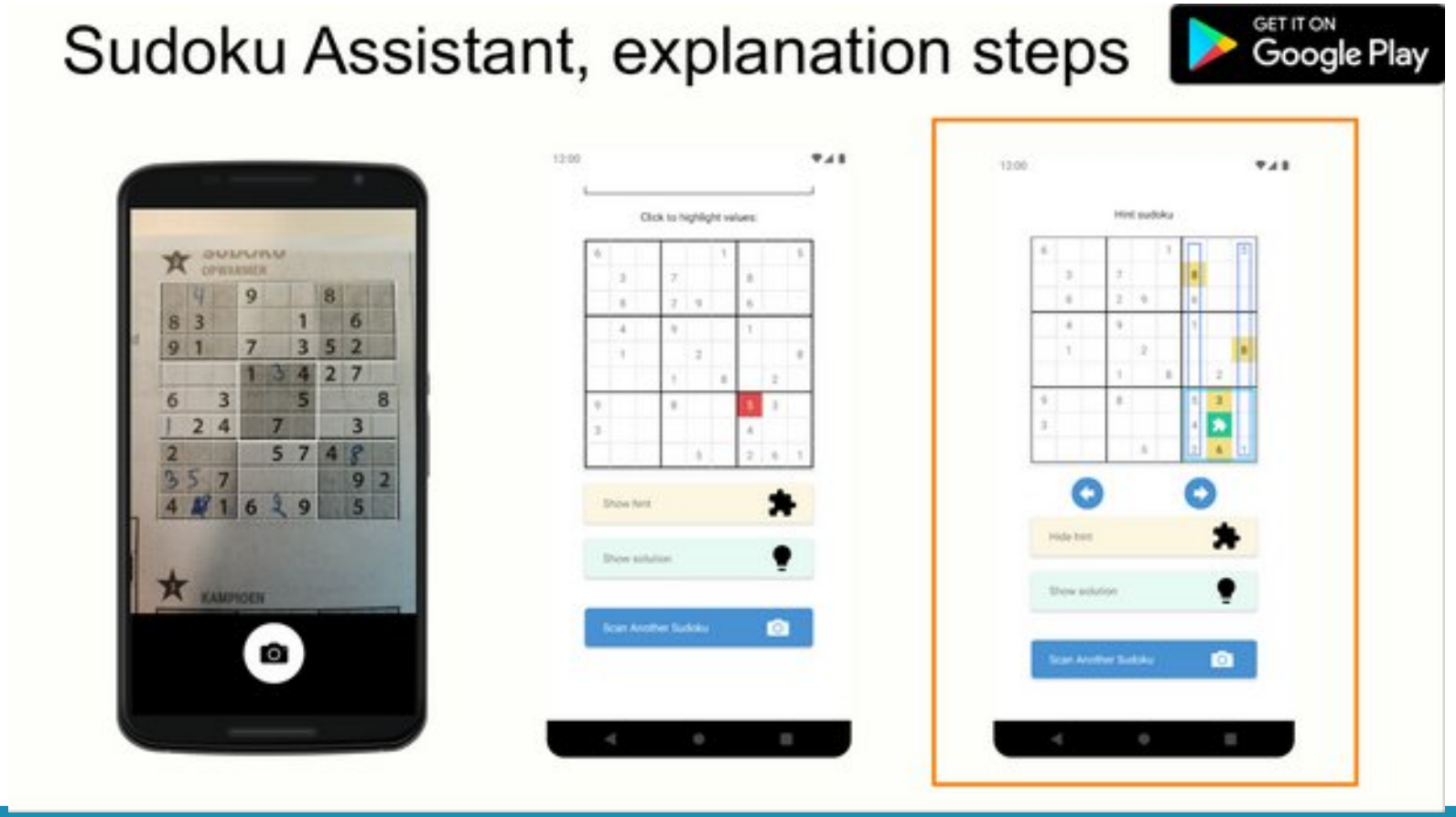


Fig. 1 Simplified Plant Layout workflow for a feasible plant instance (a⇒d) and for feasibility restoration (a⇒b⇒c⇒d)

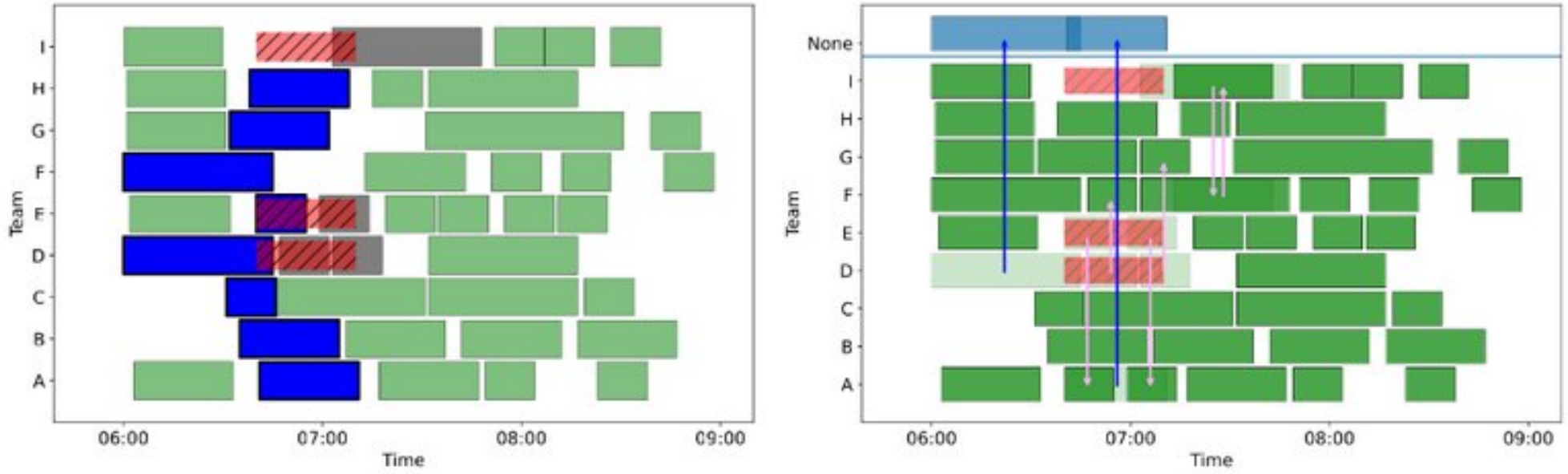
Applications of explanation techniques

"Sudoku Assistant–an AI-Powered App to Help Solve Pen-And-Paper Sudokus". *Tias Guns, Emilio Gamba, Maxime Mulamba, Ignace Bleukx, Senne Berden, and Milan Pesa*. AAAI-2023 Best Demo Award.



Applications of explanation techniques

"Modeling and Explaining an Industrial Workforce Allocation and Scheduling Problem". *Ignace Bleukx, Ryma Boumazouza, Tias Guns, Nadine Laage, and Guillaume Poveda.* CP 2025 Best Application Award



(a) Visualization of a MUS.

(b) Visualization of a MCS.

Figure 2 Visualization of two explanation methods for the same working shift and disruption.

Applications of explanation techniques

"CrewAId: Interactive Optimisation for Human-In-The-Loop Crew Rostering and Rerostering". *Matthias Klapper-stueck, Frits de Nijs, Ilankaikone Senthooran, Matteo Miceli, and Michael Wybrow.* CP 2026 Best Application Award

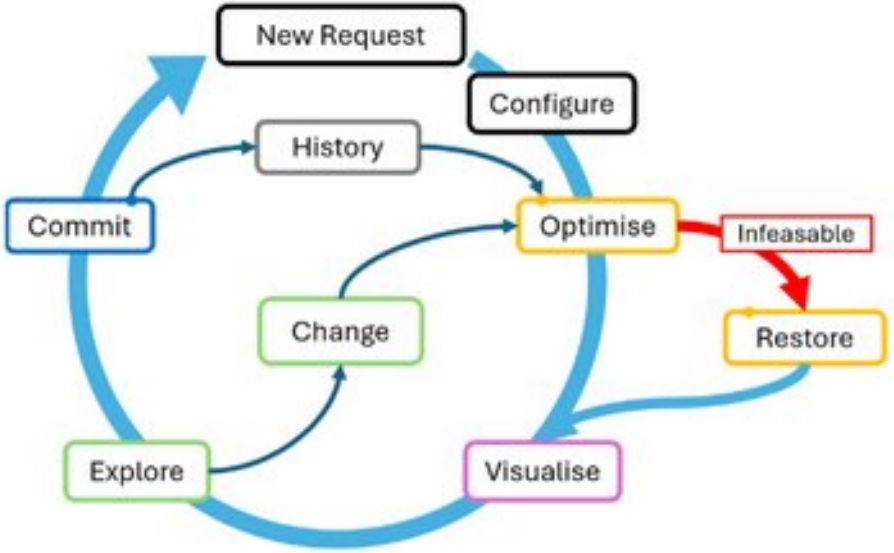


Figure 10 Showing the general workflow using our crew roster and rerostering system. Each new request incorporates knowledge and decisions from previous operations, for creating new solutions to explore. In case of infeasibility the circle is extended to restore that particular instance and to continue the process.

Relation to Explainable AI (XAI) at large

D. Gunning, 2015: DARPA XAI challenge

"Every explanation is set within a context that depends..." -> domain dependent

M. Fox et al, 2017: Explainable Planning

Need for trust, interaction and transparancy.

T. Miller, 2018: Explainable AI: Beware of Inmates Running the Asylum

Insights from the social sciences: *Someone* explains *something* to *someone*

R. Guidotti, 2018: A survey of methods for explaining black box ML models

The vast majority of work/attention...

H. Zhaom, 2024: Explainability for large language models: A survey.

The latest hype

Formal XAI

J. Marques-Silva, A. Ignatiev, AAAI-2022: Delivering Trustworthy AI through Formal XAI

"... best known XAI approaches [in ML] fail to provide sound explanations or they exhibit significant redundance."

Formal explanations are not only sound but guarantee irredundancy.

All formal XAI methods relate back to the same concepts and related algorithms:

- Minimal Unsatisfiable Subset (MUS)
- Minimal Correction Subset (MCS)

What conceptually changes is the **set of what** (and the solvers underneath)

Explainable Constraint Programming (XCP)

In general, "**Why X?**" (with X a solution or UNSAT)

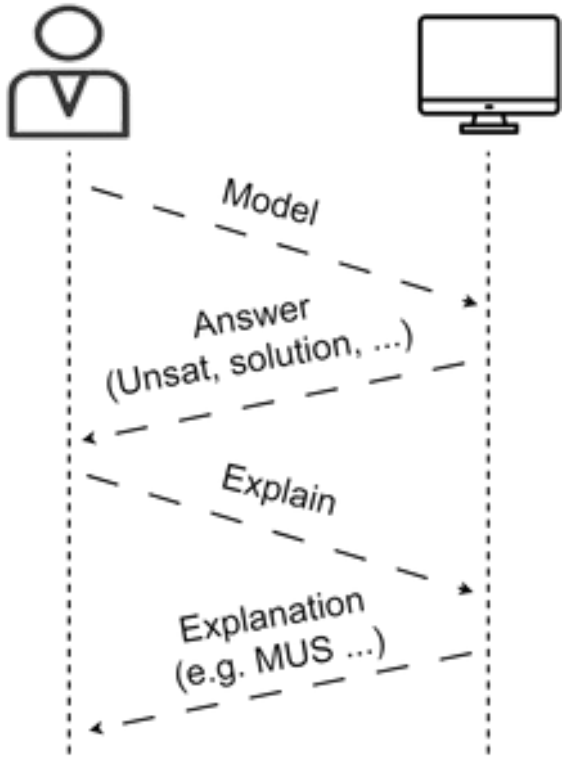
Deductive explanation:

- *What constraints cause X?*

Counterfactual explanation:

- *What to change if I want Y instead of X?*
  (includes feasibility restoration)

Especially useful/needed in application context!



Outline of the lectures

Introduction to XCP

- Deductive explanations for UNSAT

Break

Deductive explanations for SAT/OPT

Counterfactual explanations for UNSAT/SAT/OPT

Use case, challenges and conclusion

Explainable Constraint Programming (XCP)

In general, "**Why X?**" (with X a solution or UNSAT)

Deductive explanation:

- *What constraints cause X?*

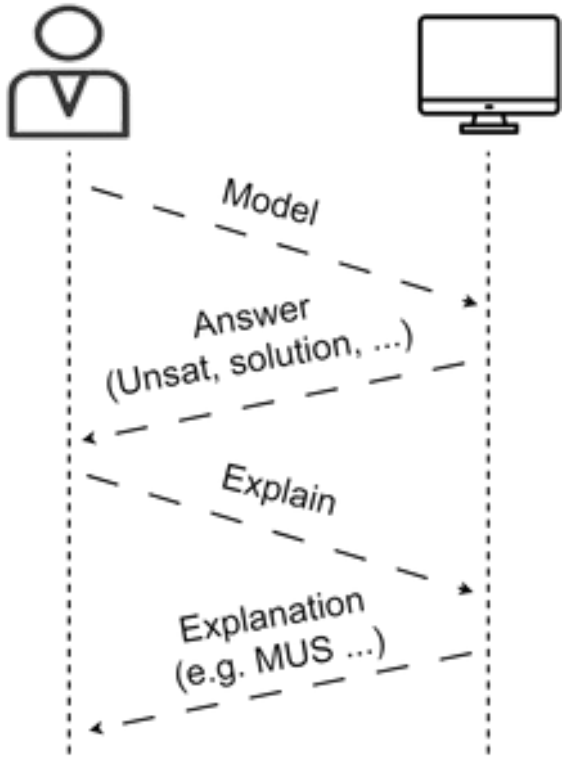
E.g.,

- *why* is there no solution
- *why* does an assignment follow from the constraints
- *why* is it optimal/is there no better solution
- ...

All these cases can typically be reduced to explaining UNSAT.

Example, explain why *fact* follows from constraints *C*:

$$\mathrm{C} \models \mathrm{fact} \quad :: \quad \mathrm{C} \wedge \neg\, \mathrm{fact} \models \bot$$



Running example in this talk: Nurse Scheduling

- The assignment of *shifts* and *holidays* to nurses.
- Each nurse has their own restrictions and preferences;
- so does the hospital.

```
In [9]:  instance = "Benchmarks/Instance1.txt"  # "http://www.schedulingbenchmarks.org/nrp/data/Instance1.txt"
         data = get_data(instance)
```

```
In [10]: print("Nr of days to schedule:", data.horizon)
         print("Nr of shift types:", len(data.shifts))

         print("Staff holidays:")
         pd.merge(data.days_off, data.staff[["# ID","name"]], left_on="EmployeeID", right_on="# ID", how="left")
```

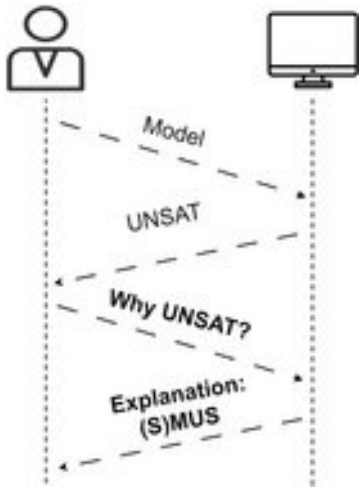
Nr of days to schedule: 14
Nr of shift types: 1
Staff holidays:

	EmployeeID	DayIndex	# ID	name
0	A	0	A	Megan
1	B	5	B	Katherine
2	C	8	C	Robert
3	D	2	D	Jonathan
4	E	9	E	William
5	F	5	F	Richard
6	G	1	G	Kristen
7	H	7	H	Kevin

An example solution (day requirements and day preferences as soft constraints)

	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan	-	D	D	D	D	-	-	D	D	-	-	D	D	D	9	4320
Katherine	D	D	D	D	D	-	-	-	D	D	-	-	D	D	9	4320
Robert	D	D	D	-	-	D	D	D	-	-	D	D	-	-	8	3840
Jonathan	D	D	-	-	-	D	D	D	D	D	-	-	-	-	7	3360
William	-	D	D	D	D	-	-	D	D	-	-	D	D	D	9	4320
Richard	D	D	D	D	D	-	-	D	D	-	-	D	D	-	9	4320
Kristen	-	-	D	D	D	-	-	D	D	D	-	-	D	D	8	3840
Kevin	D	D	-	-	-	-	-	-	D	D	D	D	D	-	7	3360
Cover D	5/5	7/7	6/6	5/4	5/5	2/5	2/5	6/6	7/7	4/4	2/2	5/5	6/6	4/4	0	0

Nurse rostering, explaining unsatisfiability



Model nurse rostering problem as decision problem (no objective)

All constraints and day preferences are **hard** constraints.

```
In [12]: # model as decision model
         factory = NurseSchedulingFactory(data)
         model, nurse_view = factory.get_decision_model()  # CMPpy DECISION Model
         model.solve()
```

False

... no solution found

```
print(f"Model has {len(model.constraints)} constraints:")
for cons in model.constraints: print("-", cons, " "*(60-len(str(cons))), cons.__repr__())
```

✓ 0.0s

Slide Type: slide Python

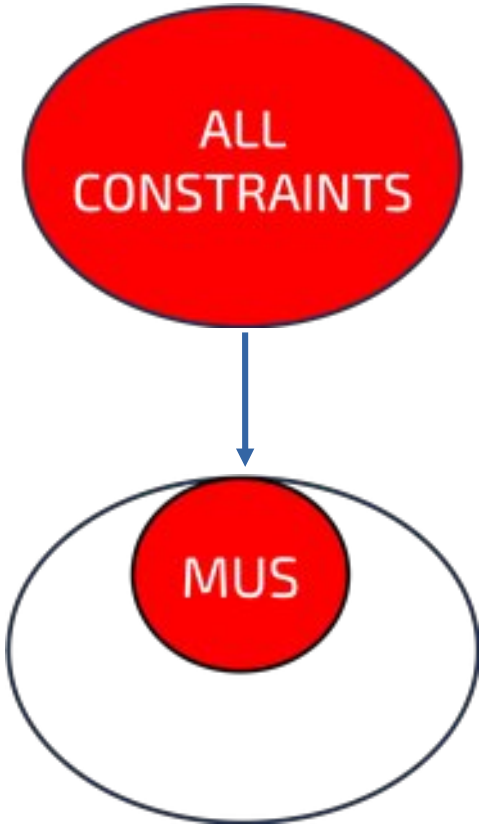
```
Model has 360 constraints:
- Megan can work at most 14 D-shifts                          count([roster[0,0],roster[0,1],roster[0,2
- Katherine can work at most 14 D-shifts                      count([roster[1,0],roster[1,1],roster[1,2
- Robert can work at most 14 D-shifts                         count([roster[2,0],roster[2,1],roster[2,2
- Jonathan can work at most 14 D-shifts                       count([roster[3,0],roster[3,1],roster[3,2
- William can work at most 14 D-shifts                        count([roster[4,0],roster[4,1],roster[4,2
- Richard can work at most 14 D-shifts                        count([roster[5,0],roster[5,1],roster[5,2
- Kristen can work at most 14 D-shifts                        count([roster[6,0],roster[6,1],roster[6,2
- Kevin can work at most 14 D-shifts                          count([roster[7,0],roster[7,1],roster[7,2
- Megan cannot work more than 4                               sum([  0 480][roster[0,0]], [  0 480][ros
- Katherine cannot work mor
- Robert cannot work more
- Jonathan cannot work m
- William cannot work mo
- Richard cannot work mo
- Kristen cannot work mor
- Kevin cannot work more tha
- Megan should work at least 336
- Katherine should work at least 3360min                      sum([  0 480][roster[1,0]], [  0 480][ros
- Robert should work at least 3360min                         sum([  0 480][roster[2,0]], [  0 480][ros
- Jonathan should work at least 3360min                       sum([  0 480][roster[3,0]], [  0 480][ros
- William should work at least 3360min                        sum([  0 480][roster[4,0]], [  0 480][ros
- Richard should work at least 3360min                        sum([  0 480][roster[5,0]], [  0 480][ros
- Kristen should work at least 3360min                        sum([  0 480][roster[6,0]], [  0 480][ros
- Kevin should work at least 3360min                          sum([  0 480][roster[7,0]], [  0 480][ros
...
- Shift D on Thu 2 must be covered by 2 nurses out of 8       count([roster[0,10],roster[1,10],roster[2
- Shift D on Fri 2 must be covered by 5 nurses out of 8       count([roster[0,11],roster[1,11],roster[2
- Shift D on Sat 2 must be covered by 6 nurses out of 8       count([roster[0,12],roster[1,12],roster[2
- Shift D on Sun 2 must be covered by 4 nurses out of 8       count([roster[0,13],roster[1,13],roster[2
```

ALL CONSTRAINTS

The set of all constraints is unsatisfiable.

But do all constraints contribute to this?

Deductive Explanations for UNSAT problems



Minimal Unsatisfiable Subset (MUS)

Pinpoint to constraints causing a conflict

... shrink model to a minimal set of constraints

... minimize cognitive burden for user

How to compute a MUS?

Deletion-based MUS algorithm

[Joao Marques-Silva. Minimal Unsatisfiability: Models, Algorithms and Applications. ISMVL 2010. pp. 9-14]

From Alexey's class on Monday:

Input : Set $\mathcal{F}$
Output: Minimal subset $\mathcal{M}$
begin
  $\mathcal{M} \leftarrow \mathcal{F}$
  foreach $c \in \mathcal{M}$:
    if $\neg SAT(\mathcal{M} \setminus \{c\})$:
      $\mathcal{M} \leftarrow \mathcal{M} \setminus \{c\}$
  return $\mathcal{M}$
end

How to compute a MUS?

Deletion-based MUS algorithm



How to compute a MUS?

Deletion-based MUS algorithm



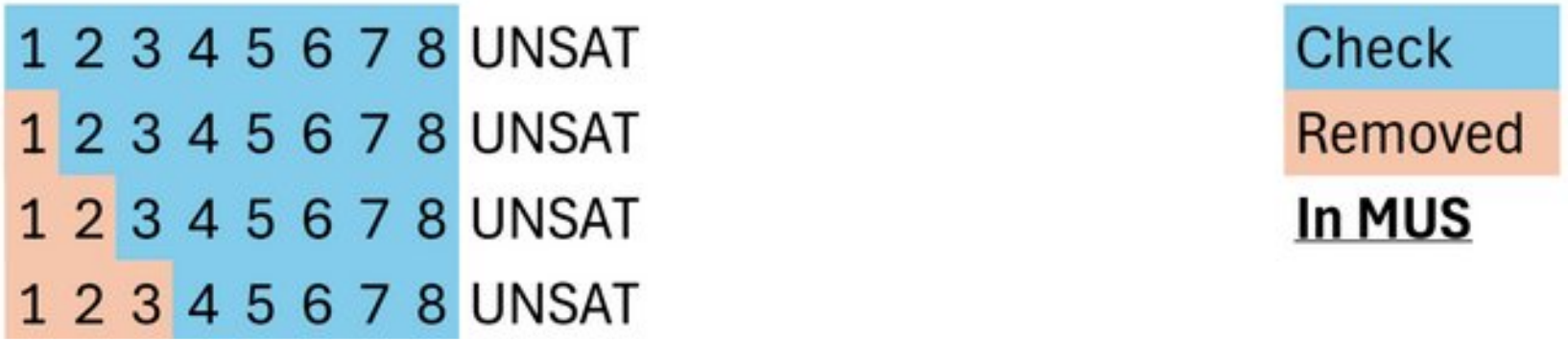
How to compute a MUS?

Deletion-based MUS algorithm



How to compute a MUS?

Deletion-based MUS algorithm



How to compute a MUS?

Deletion-based MUS algorithm



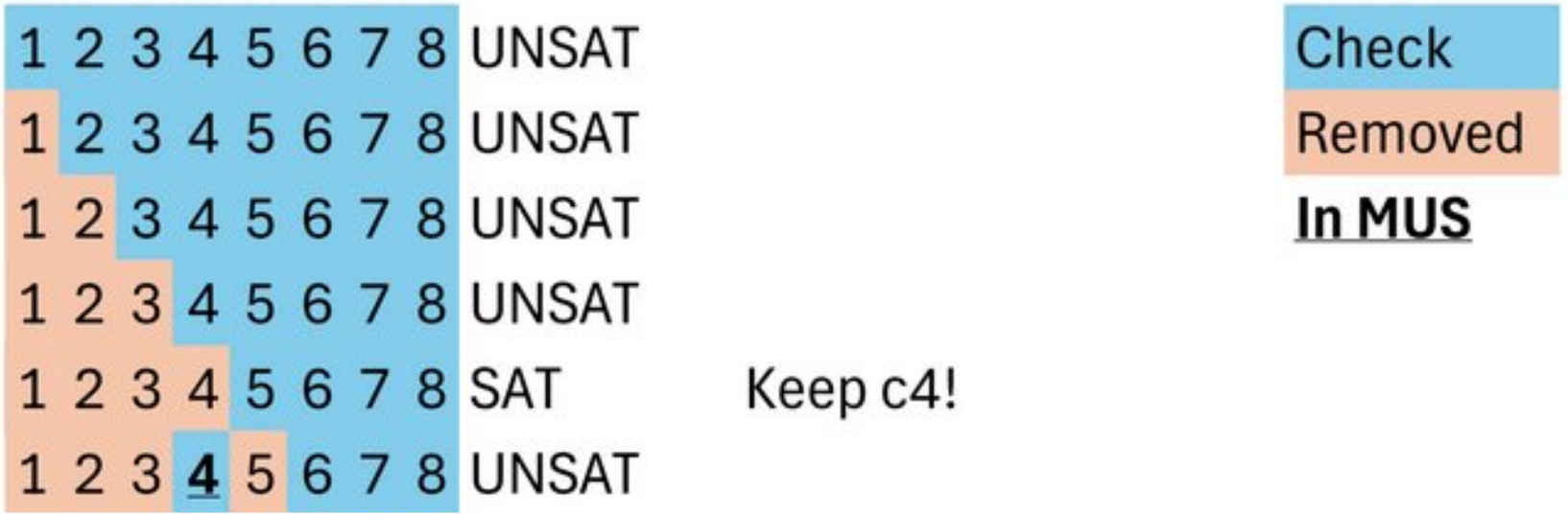
How to compute a MUS?

Deletion-based MUS algorithm



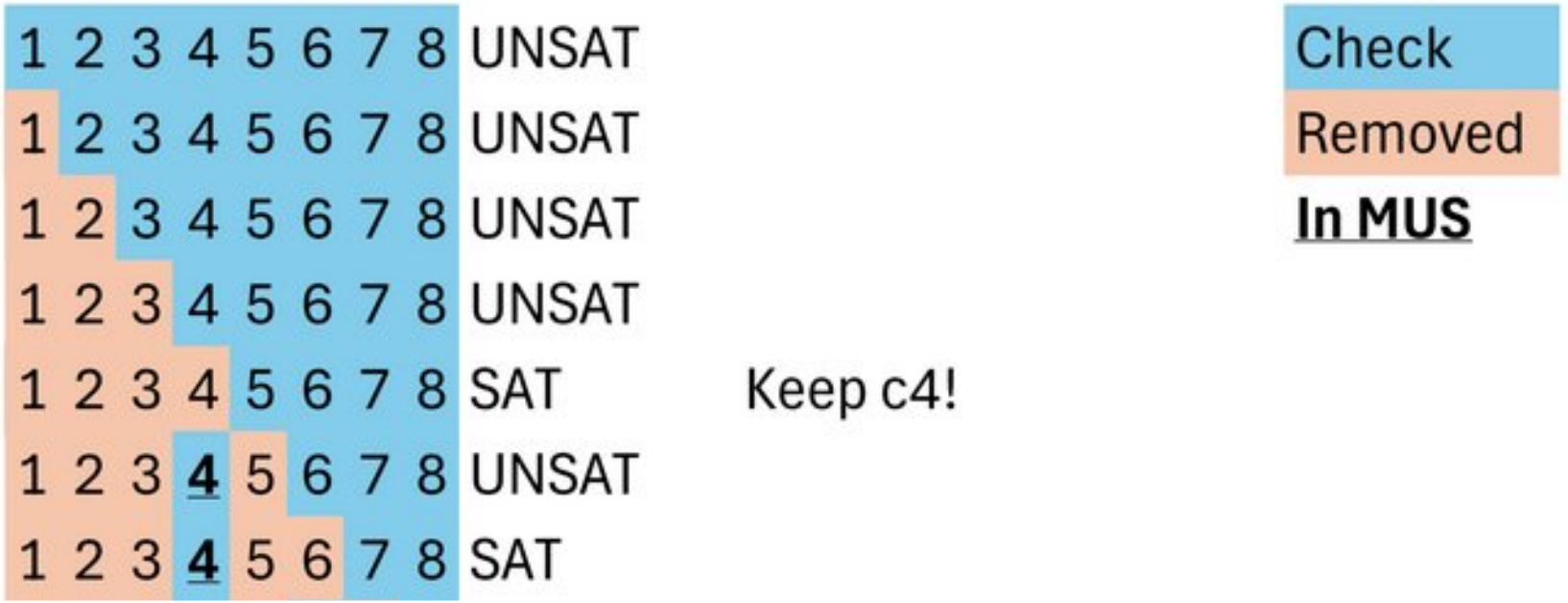
How to compute a MUS?

Deletion-based MUS algorithm



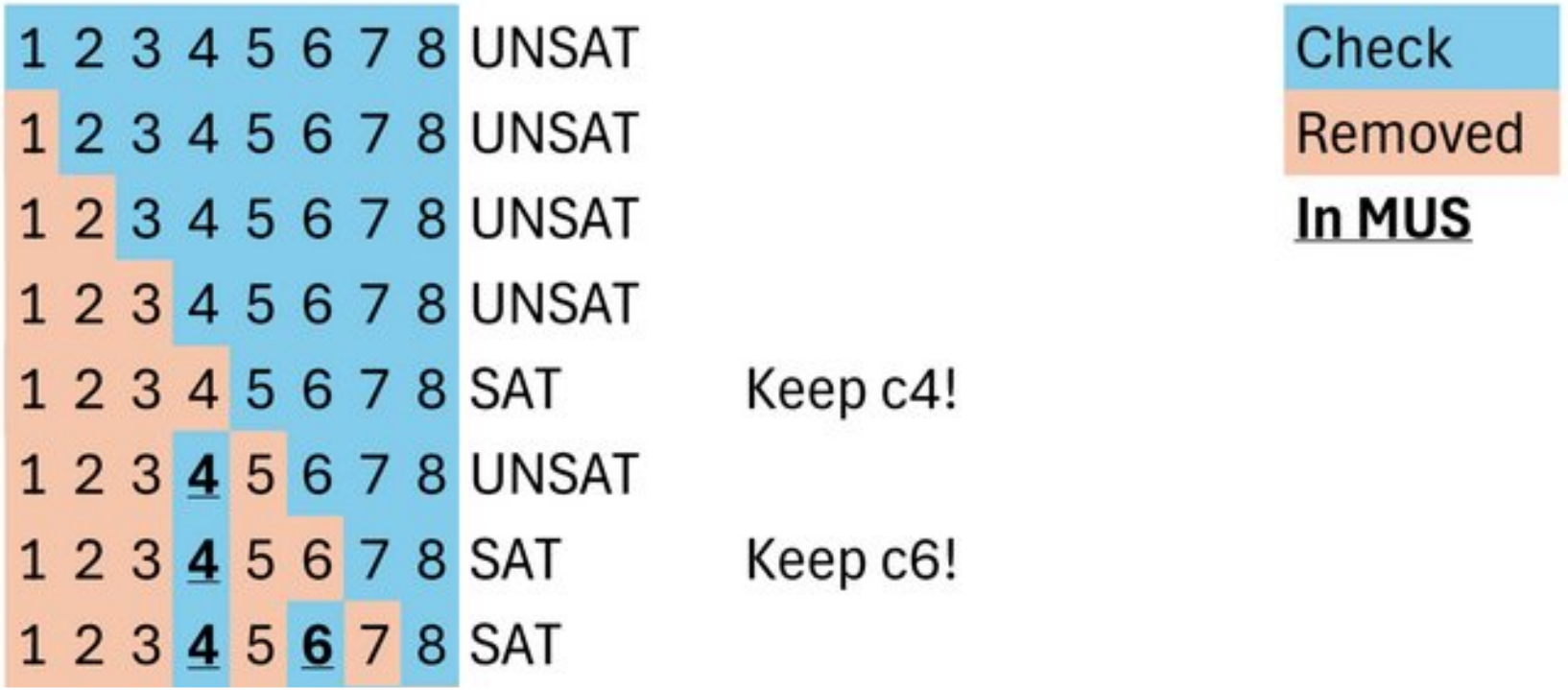
How to compute a MUS?

Deletion-based MUS algorithm



How to compute a MUS?

Deletion-based MUS algorithm



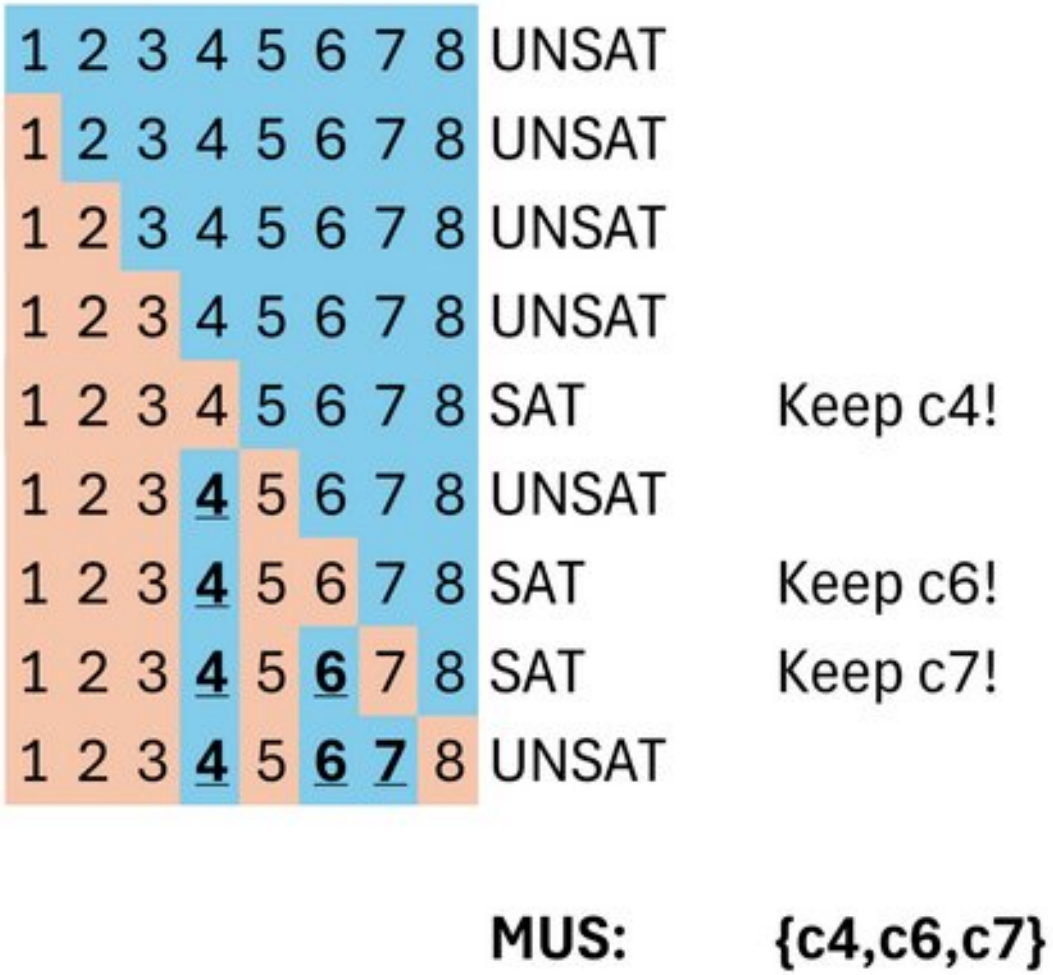
How to compute a MUS?

Deletion-based MUS algorithm



How to compute a MUS?

Deletion-based MUS algorithm



How to compute a MUS, efficiently?

```
    t0 = time.time()
    core = mus_naive(model.constraints)
    print(f"Naive MUS took {time.time()-t0} seconds")
```

[25] ✓ 11.8s — Slide Type: - Python

```
Naive MUS took 11.825149059295654 seconds
```

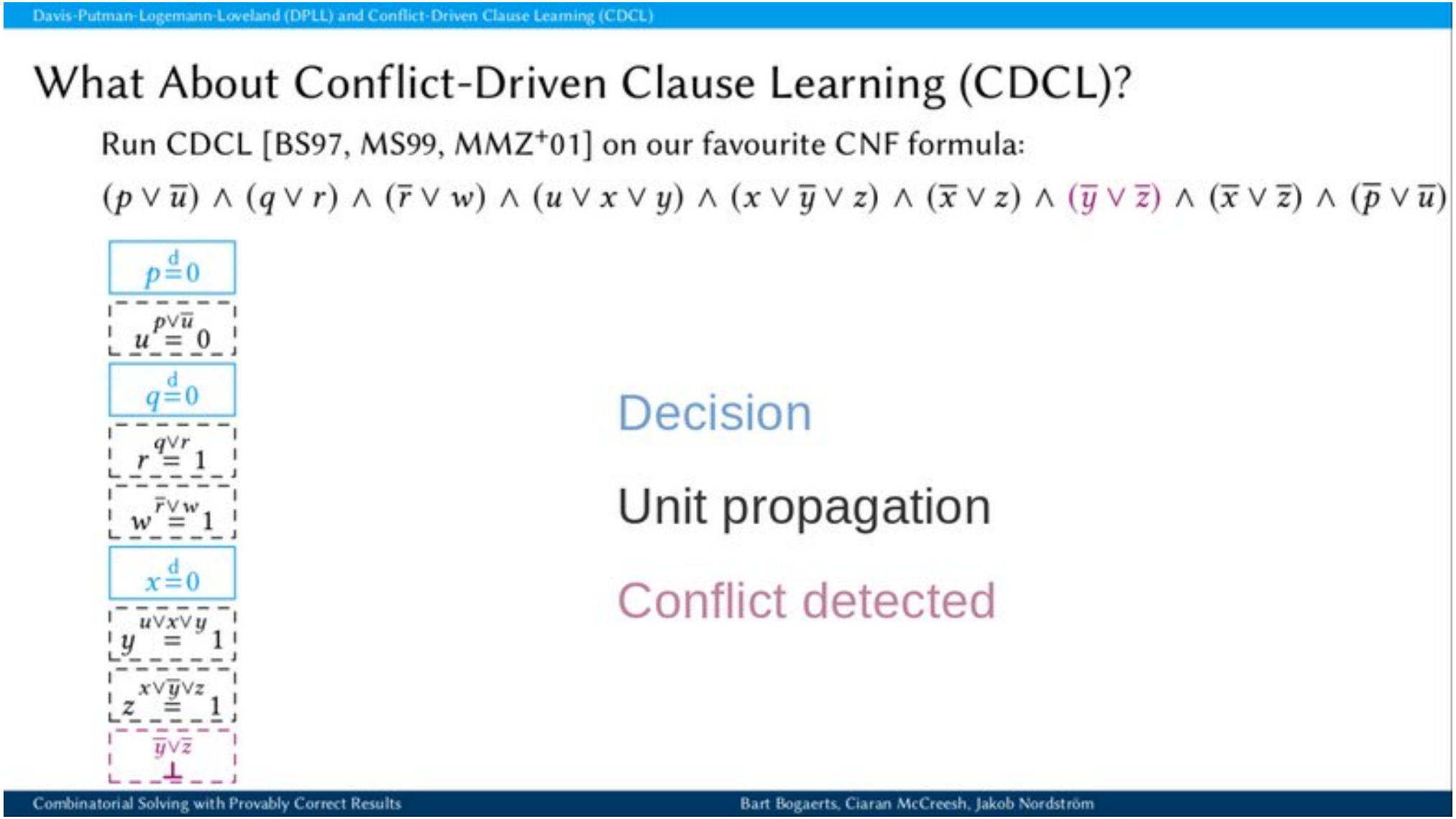
```
    from cpmpy.tools.explain import mus

    t0 = time.time()
    res = mus(model.constraints, solver="pumpkin")
    print(f"Assumption-based MUS took {time.time()-t0} seconds")
```

[27] ✓ 0.3s — Python

```
Assumption-based MUS took 0.36166858673095703 seconds
```

Deepdive: incremental CDCL solving with assumption variables 1/4

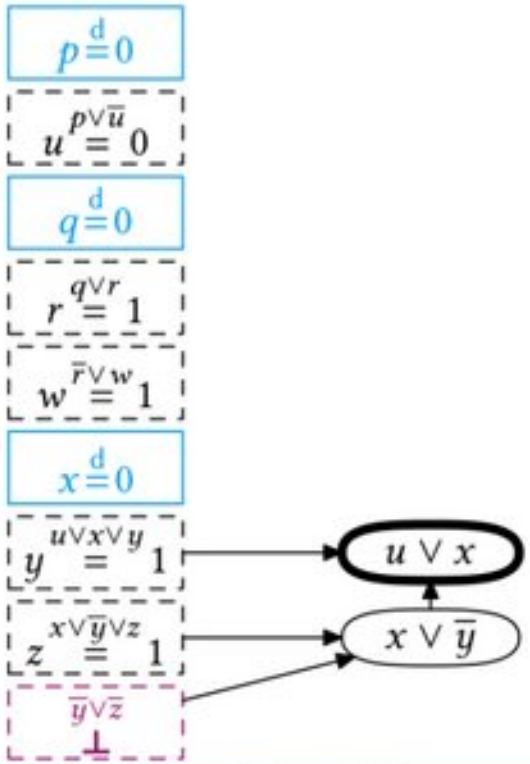


Deepdive: incremental CDCL solving with assumption variables 2/4

Conflict Analysis

Time to analyse this conflict and learn from it!

$$(p \vee \overline{u}) \wedge (q \vee r) \wedge (\overline{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee z) \wedge (\overline{y} \vee \overline{z}) \wedge (\overline{x} \vee \overline{z}) \wedge (\overline{p} \vee \overline{u})$$



Clause learning

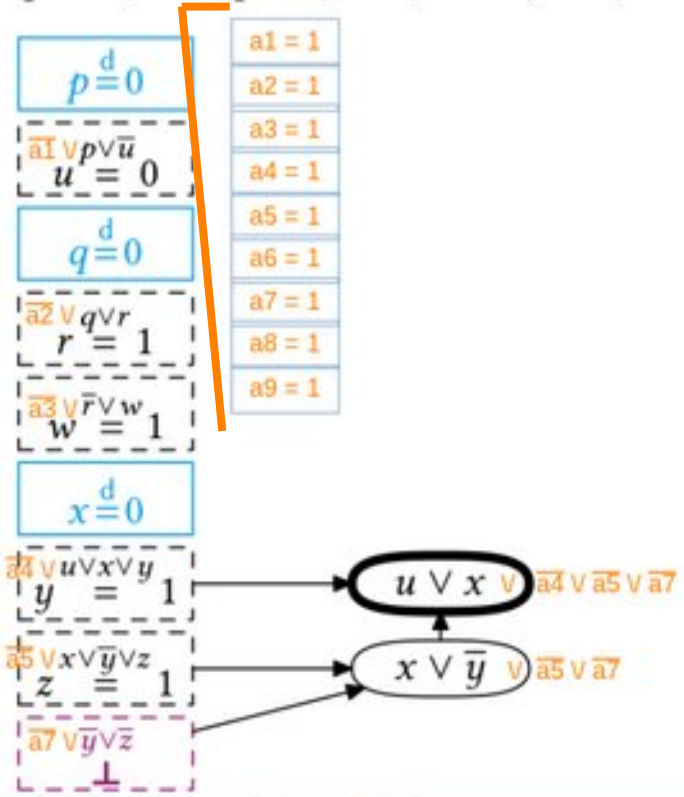
Case analysis over z for last two clauses:

- $x \vee \overline{y} \vee z$ wants $z = 1$
- $\overline{y} \vee \overline{z}$ wants $z = 0$
- Resolve clauses by merging them & removing z — must satisfy $x \vee \overline{y}$

Deepdive: incremental CDCL solving with assumption variables 3/4

Assumption variables :: a1 → (p \/ -u) :: -a1 \/ p \/ -u

$$(\overline{a1} \vee p \vee \overline{u}) \wedge (\overline{a2} \vee q \vee r) \wedge (\overline{a3} \vee \overline{r} \vee w) \wedge (\overline{a4} \vee u \vee x \vee y) \wedge (\overline{a5} \vee x \vee \overline{y} \vee z) \wedge (\overline{a6} \vee \overline{x} \vee z) \wedge (\overline{a7} \vee \overline{y} \vee \overline{z}) \wedge (\overline{a8} \vee \overline{x} \vee \overline{z}) \wedge (\overline{a9} \vee \overline{p} \vee \overline{u})$$



Clause learning with assumptions

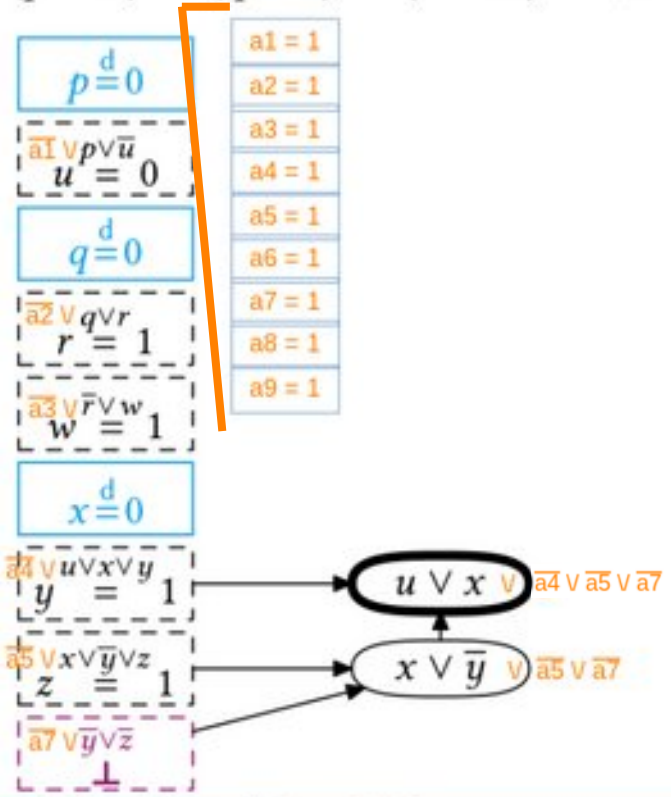
Case analysis over z for last two clauses:

- $\overline{a5} \vee x \vee \overline{y} \vee z$ wants $z = 1$
- $\overline{a7} \vee \overline{y} \vee \overline{z}$ wants $z = 0$
- Resolve clauses by merging them & removing z — must satisfy $x \vee \overline{y} \vee \overline{a5} \vee \overline{a7}$

Deepdive: incremental CDCL solving with assumption variables 4/4

Assumption variables

$$(\overline{a1} \vee p \vee \overline{u}) \wedge (\overline{a2} \vee q \vee r) \wedge (\overline{a3} \vee \overline{r} \vee w) \wedge (\overline{a4} \vee u \vee x \vee y) \wedge (\overline{a5} \vee x \vee \overline{y} \vee z) \wedge (\overline{a6} \vee \overline{x} \vee z) \wedge (\overline{a7} \vee \overline{y} \vee \overline{z}) \wedge (\overline{a8} \vee \overline{x} \vee \overline{z}) \wedge (\overline{a9} \vee \overline{p} \vee \overline{u})$$



Clause learning with assumptions

- Can extract UNSAT core:
  assumption variables present in 'final' conflict
- Can solve repeatedly with diff. assumption variable
  learned clauses remain valid (contain the assum

How to compute a MUS, efficiently?

Assumption-based incremental solving **only for Boolean SAT** problems?

No!

- SMT solvers: *SAT Module Theories with CDCL* (e.g. Z3)
- CP-solvers: *Lazy Clause Generation* (e.g. OR-Tools, Pumpkin)
- Pseudo-Boolean solvers: *Conflict-Driven Cutting Plane Learning* (e.g. RoundingSAT, Exact)
- MaxSAT solvers: *Core-guided solvers*

Reason for using CPMpy here: incremental solving

Solver	Technology	ISAT?	Assumption variables?	IOPT?	Notes
Z3	SMT	Yes	Yes	No	
Or-Tools	CP-SAT	No	Yes	No	Assumptions NOT incremental! Every solve starts from scratch
Pumpkin	CP-SAT	Yes	Yes	No	
HiGHS	ILP	Yes	No	Yes	
Exact	Pseudo-Boolean	Yes	Yes	Yes	Only linux, manual installation on Mac possible
PySAT	SAT	Yes	Yes	/	
+ 11 more					GCS, Choco, CP Optimizer, MiniZinc, Hexaly, SCIP, Gurobi, CPLEX, RC2, Pindakaas, PySDD

How to compute a MUS, efficiently?

```python
def mus_assum(constraints, solver="ortools"):
    # add indicator variable per expression
    constraints = toplevel_list(constraints, merge_and=False)
    ind = cp.boolvar(shape=len(constraints), name="ind")  # Boolean indicators
    m = cp.Model(ind.implies(constraints))  # [ind[i] -> constraints[i] for all i]
    s = cp.SolverLookup.get(solver, m)
    assert s.solve(assumptions=ind) is False, "Model should be UNSAT"

    core = s.get_core()  # start from solver's UNSAT core of assumption variables
```

1) add Boolean indicator variables

2) Assume they are set to ‘true’

3) Extract an UNSAT subset from solver

How to compute a MUS, efficiently?

```python
def mus_assum(constraints, solver="ortools"):
    # add indicator variable per expression
    constraints = toplevel_list(constraints, merge_and=False)
    ind = cp.boolvar(shape=len(constraints), name="ind")  # Boolean indicators
    m = cp.Model(ind.implies(constraints))  # [ind[i] -> constraints[i] for all i]
    s = cp.SolverLookup.get(solver, m)
    assert s.solve(assumptions=ind) is False, "Model should be UNSAT"

    core = s.get_core()  # start from solver's UNSAT core of assumption variables
    i = 0
    while i < len(core):
        subcore = core[:i] + core[i+1:]  # try all but constraint 'i'
        if s.solve(assumptions=subcore) is True:
            i += 1  # removing 'i' makes it SAT, need to keep for UNSAT
        else:
            core = s.get_core()
            assert core[:i] == subcore[:i], "Core so far should stay the same"
    return [c for c,var in zip[tuple[Expression, Any]](constraints,ind) if var in core]
```

1) add Boolean indicator variables

2) Assume they are set to ‘true’

3) Extract an UNSAT subset from solver

4) Incrementally solve different subsets

5) clause set refinement: unsat core may contain even *fewer* constraints

How to compute a MUS, efficiently?

Assumption-based incremental solving for MUS computation:

- Biggest downside:
  - `*ind* → *constraint*` needs extra bookkeeping (and weaker pre-solve)
  - For global constraints even worse: can not use global* but must decompose!

- By far the biggest impact in practice: first UNSAT core

Can we get an UNSAT core *without* introducing indicator variables/implication constraints?

Spoiler alert: proof logging!

* In some cases you can use the global on auxiliary variables, and reify channeling constraints:
”Efficient Reformulations of Half-reified Global Constraints using Auxiliary Variables” Bleukx et al, JAIR26

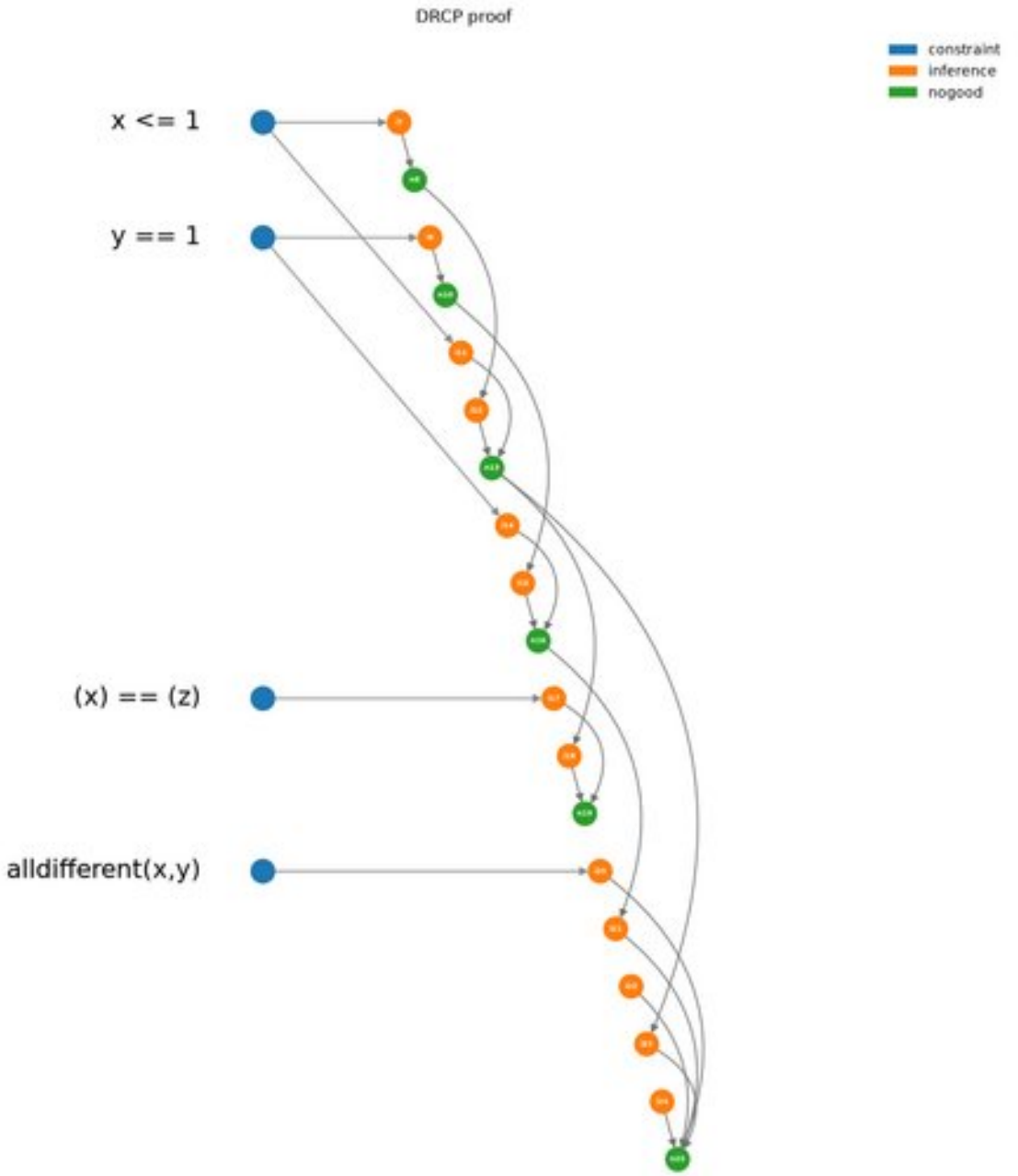
How to compute a MUS: from prooflog

Pumpking solver has CP-level proof format
+ allows users to ‘tag’ constraints to reverse-map

Example input:

1) AllDifferent(*x*, *y*)
2) (*x*) == (*z*)
3) *x* <= *1*
4) *a* + *b* <= *15*
5) y == *1*
6) *x* >= *y*

A visualisation of a DRCP proof in terms of
user-level constraints (in blue):



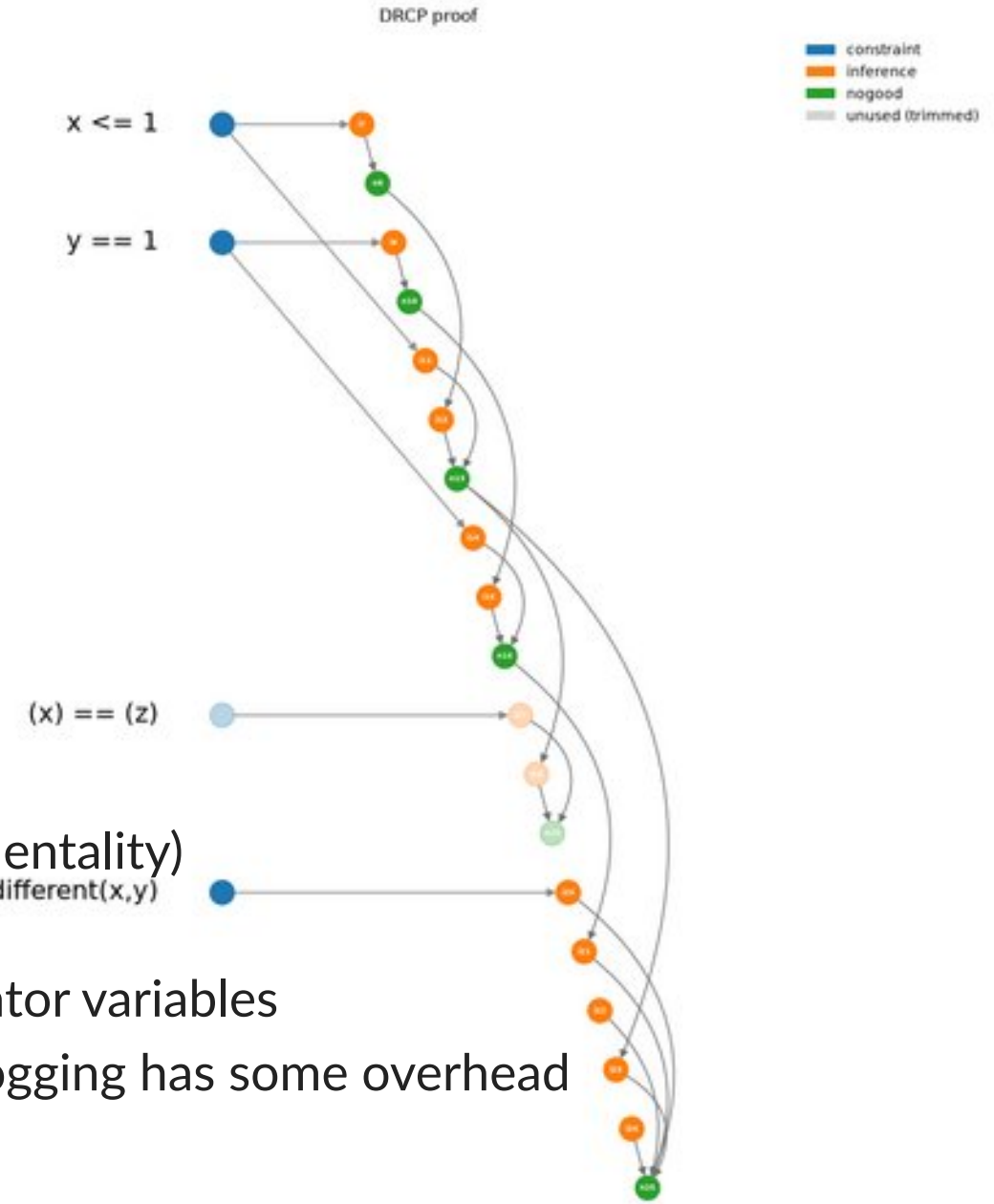
How to compute a MUS: from prooflog

Prooflog contains ALL solver steps,
including those that did not contribute to failure.

Trimming: walk backwards and only keep only elements connected to the conclusion

Is it a MUS, is it **minimal**?

- Not guaranteed!
- Need to use a MUS algorithm (but no assumption-incrementality)



Advantage: first smaller core, without needing to use indicator variables

Disadvantage: few/young prooflogging solvers, and prooflogging has some overhead

How to compute a MUS, efficiently?

```python
from cpmpy.tools.explain import mus

t0 = time.time()
res = mus(model.constraints, solver="pumpkin")
print(f"Assumption-based MUS took {time.time()-t0} seconds")
```

✓ 0.3s Python

```
Assumption-based MUS took 0.36166858673095703 seconds
```

```python
from proofcore import PumpkinProofCore

t0 = time.time()
s = PumpkinProofCore(model, proof="proof.drcp")
assert s.solve() is False
core = s.get_core()
print(f"Proof-based core took {time.time()-t0} seconds")

res = mus(core, solver="pumpkin")
print(f"Proof-based core + MUS took {time.time()-t0} seconds")
```

✓ 0.3s Python

```
Proof-based core took 0.3013732433319092 seconds
Proof-based core + MUS took 0.3125152587890625 seconds
```

Bigger difference on more sizable problems! (later)

Running example

MUS explanation of why the nurse rostering instance is UNSAT

```
In [18]: from cpmpy.tools.mus import mus

         solver = "ortools" # change to "exact" for incremental
         subset = mus(model.constraints, solver=solver)

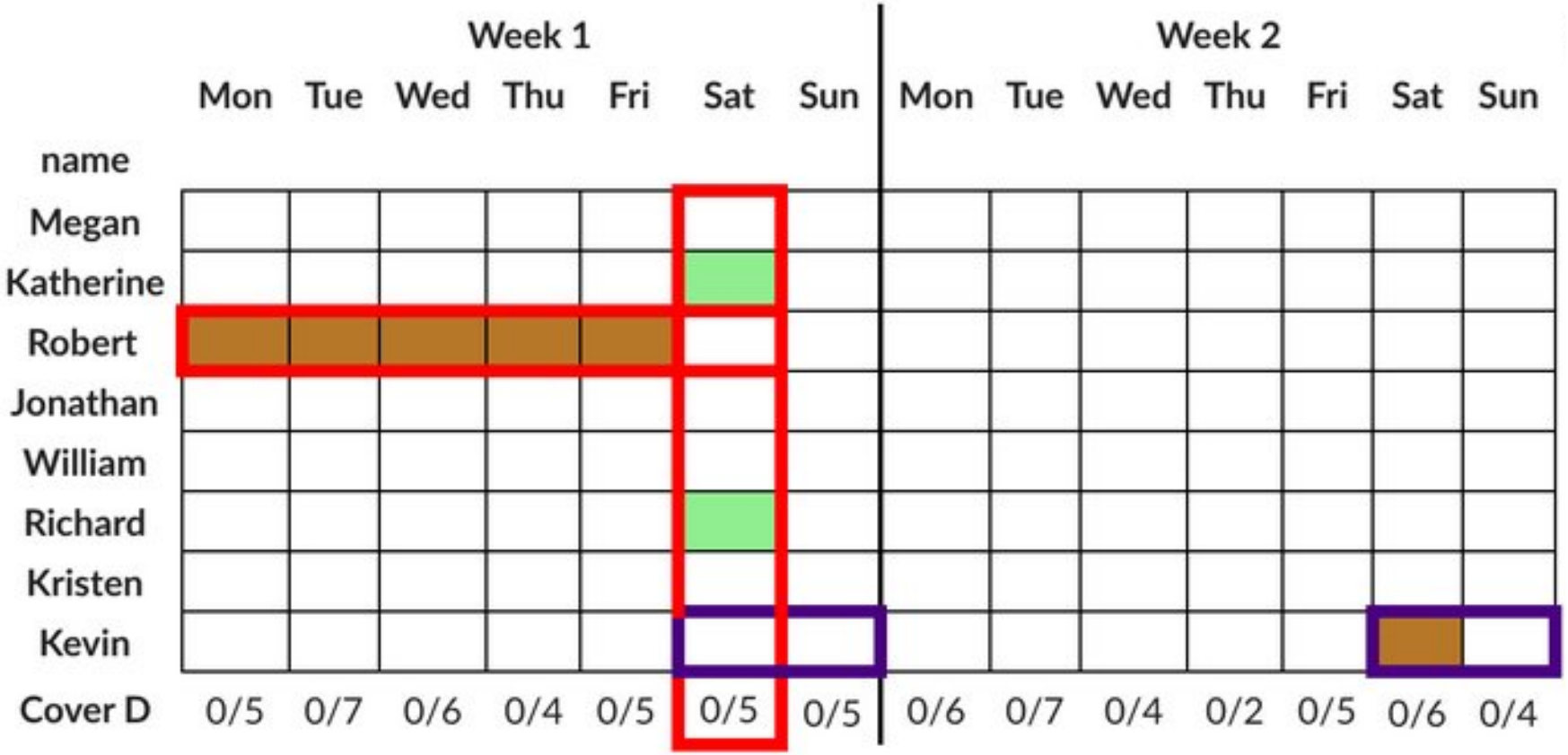
         print("Length of MUS:", len(subset))
         for cons in subset: print("-", cons)
```

Length of MUS: 11
- Robert requests to work shift D on Wed 1
- Richard has a day off on Sat 1
- Shift D on Sat 1 must be covered by 5 nurses out of 8
- Robert requests to work shift D on Thu 1
- Robert can work at most 5 days before having a day off
- Kevin should work at most 1 weekends
- Robert requests to work shift D on Mon 1
- Robert requests to work shift D on Tue 1
- Kevin requests to work shift D on Sat 2
- Katherine has a day off on Sat 1
- Robert requests to work shift D on Fri 1

Running example

MUS explanation of why the nurse rostering instance is UNSAT

```
In [19]: visualize_constraints(subset, nurse_view, factory)
```



Can we find more MUS'es systematically?

Liffiton, M.H., & Malik, A. (2013). Enumerating infeasibility: Finding multiple MUSes quickly. *In Proceedings of the 10th International Conference on Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2013) (pp. 160–175)*

```
In [20]: from cpmpy.tools.explain import marco
         t0 = time.time()
         cnt = 0
         for (kind, sset) in marco(model.constraints, solver=solver, map_solver=solver):
             if kind == "MUS":
                 print("M", end="")
                 cnt += 1
             else: print(".", end="") # MSS

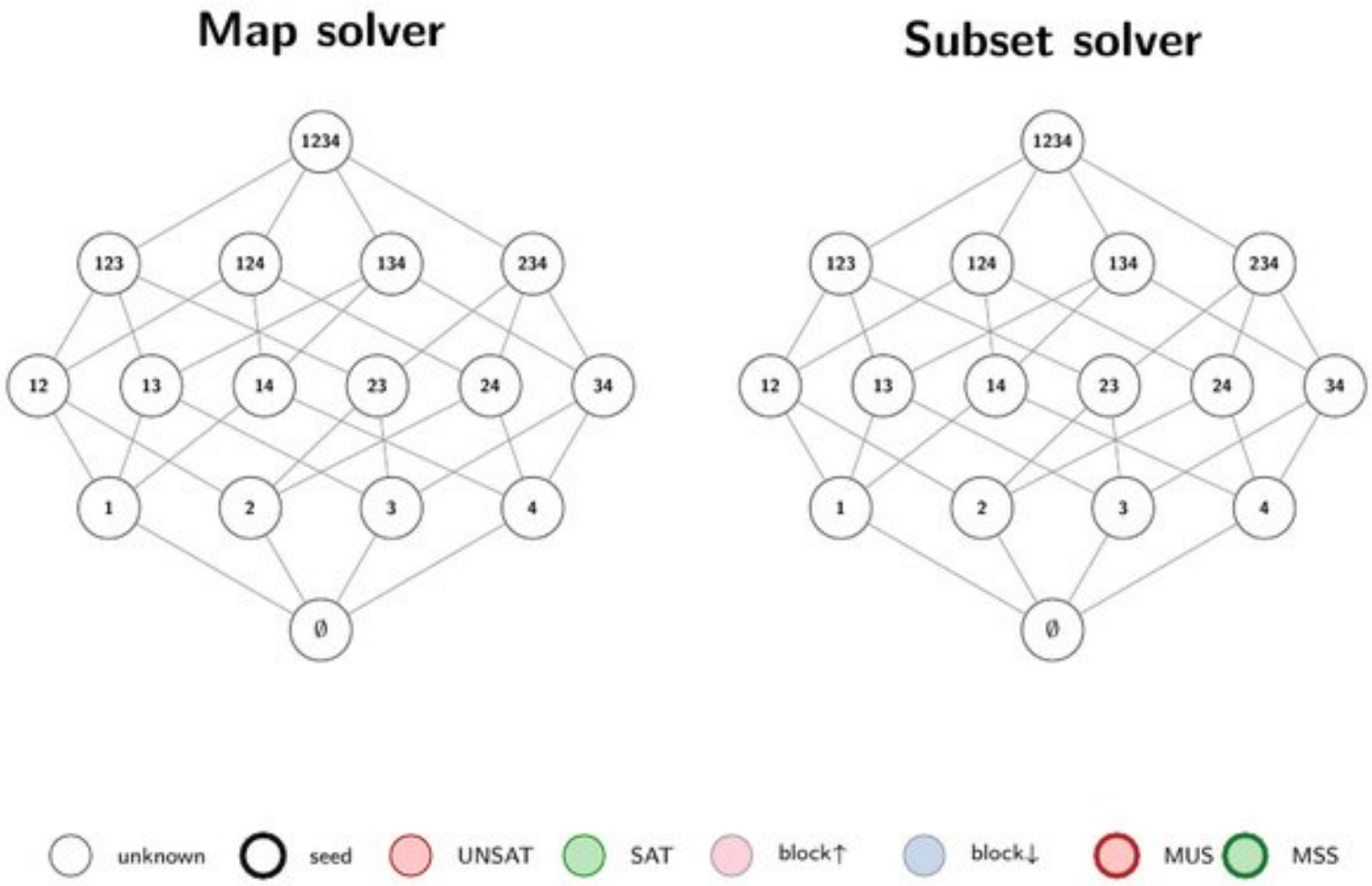
             if time.time() - t0 > 15:  break  # for tutorial: break after 15s
         print(f"\nFound {cnt} MUSes in", time.time() - t0)
```

```
MMMMMMMMMMMMMMMMMMMMMMMMMMM.MMMMMMMMMM.MM.MMMMMMMMM..MMMMMMMMMMMMMMMMM..MMMMM.MMMMMM.M.MM..
MMMMM.MMMMMMMM..MMM.M..M.M.MMM.MMM.MMM.MMMMMMMM..MM
Found 114 MUSes in 15.011045694351196
```

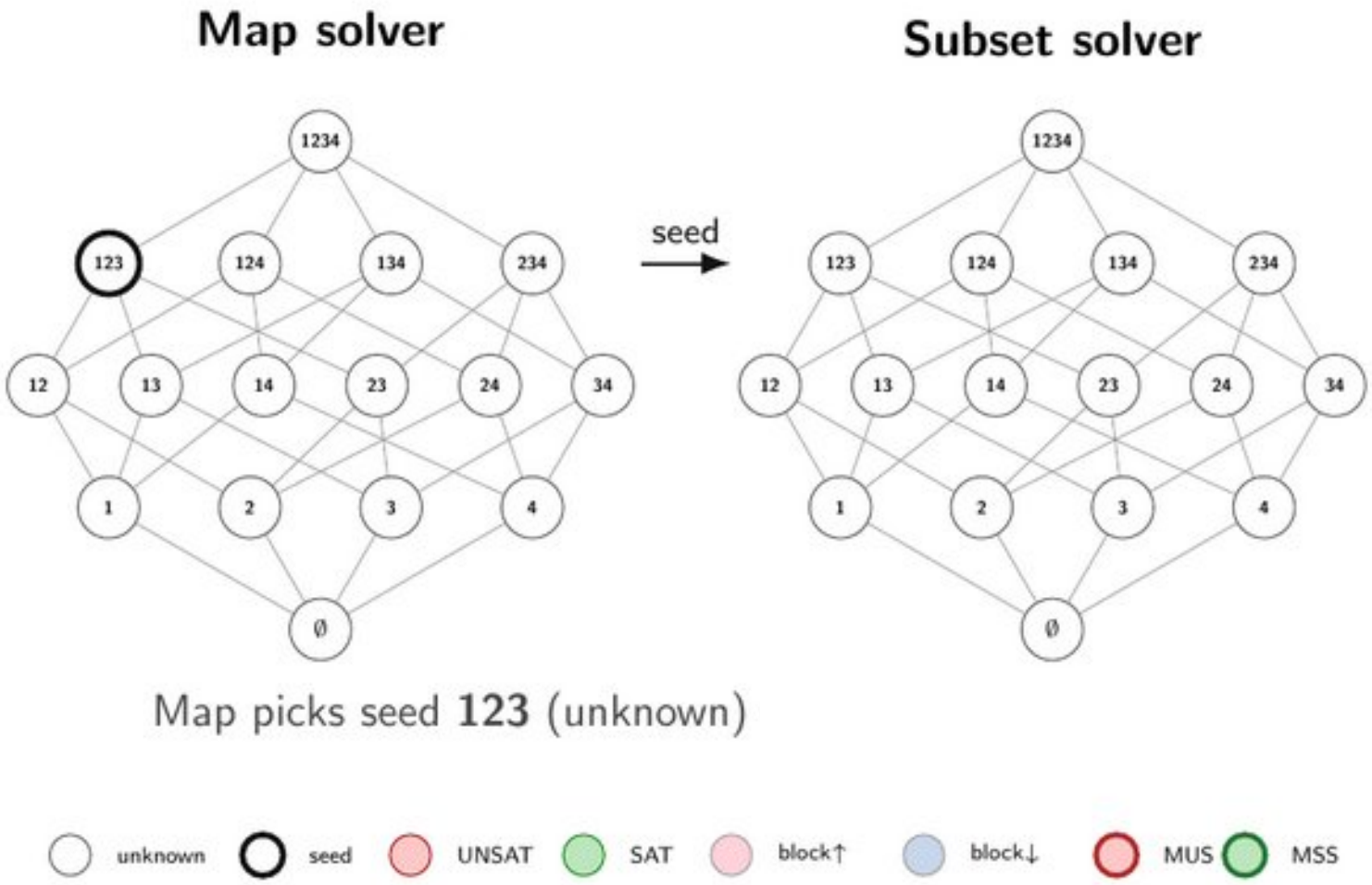
Marco uses 2 solvers:

- MAP solver = generate unknown candidate constraint set
- Subsolver = checks whether the candidate is SAT or UNSAT

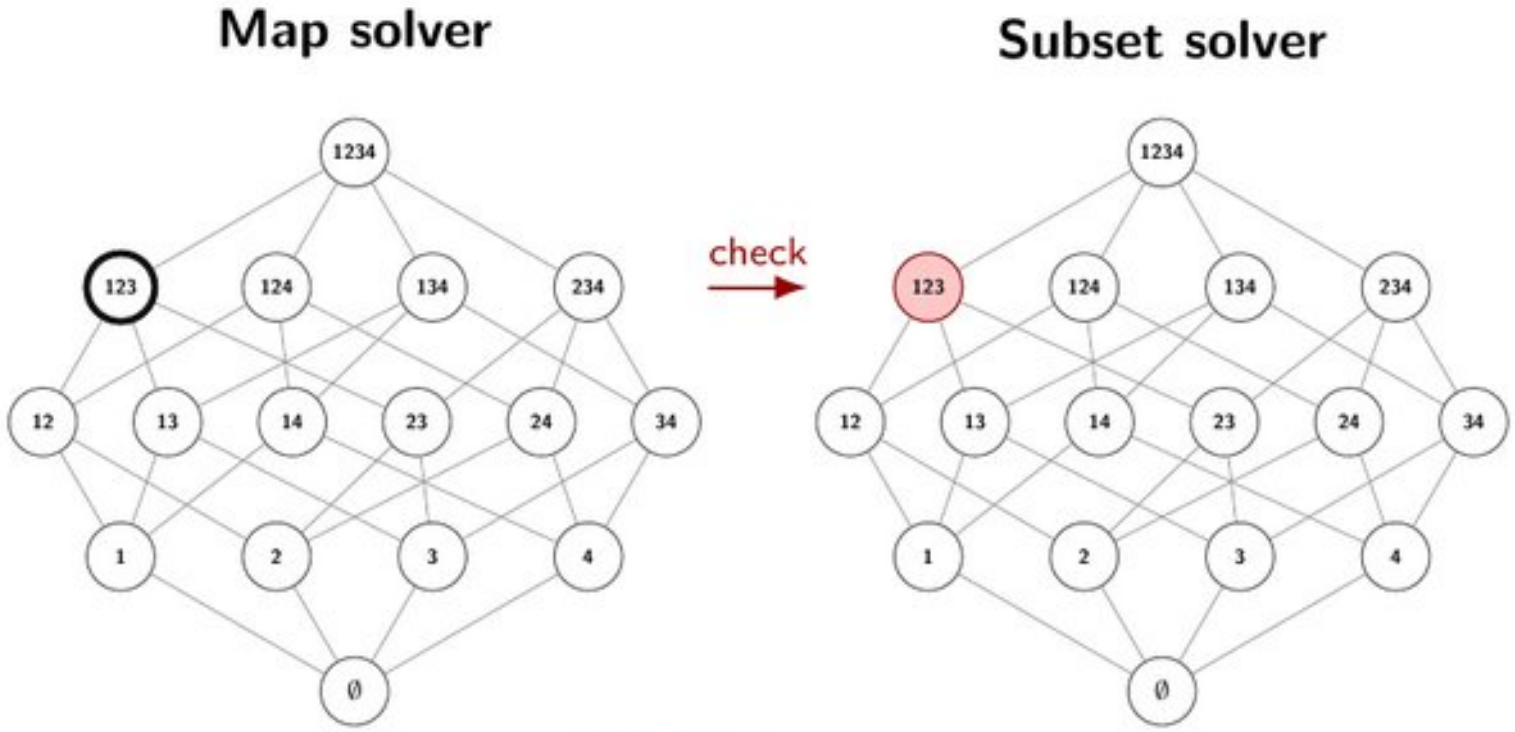
MARCO enumeration of MUS/MSS *(has implicit representation of lattice)*



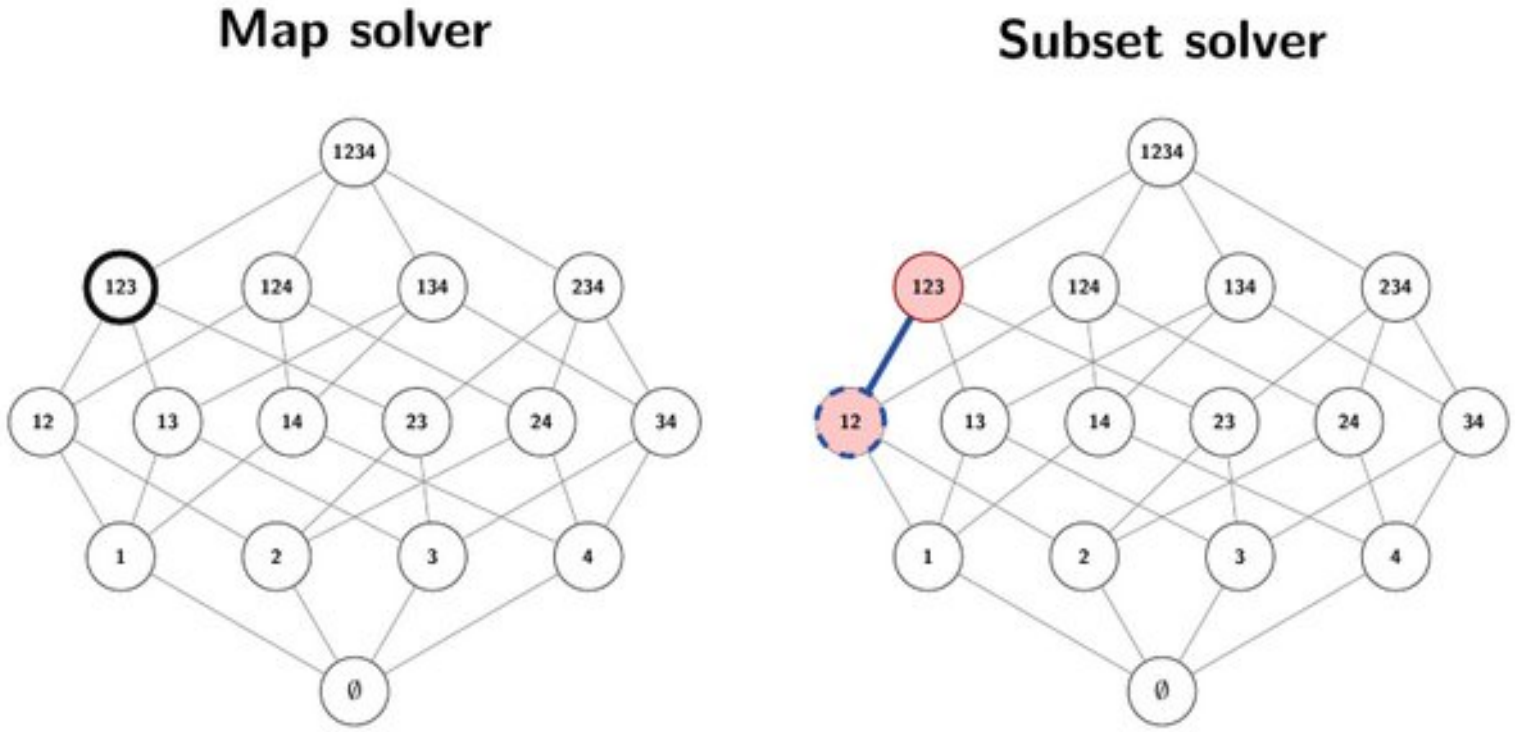
MARCO enumeration of MUS/MSS



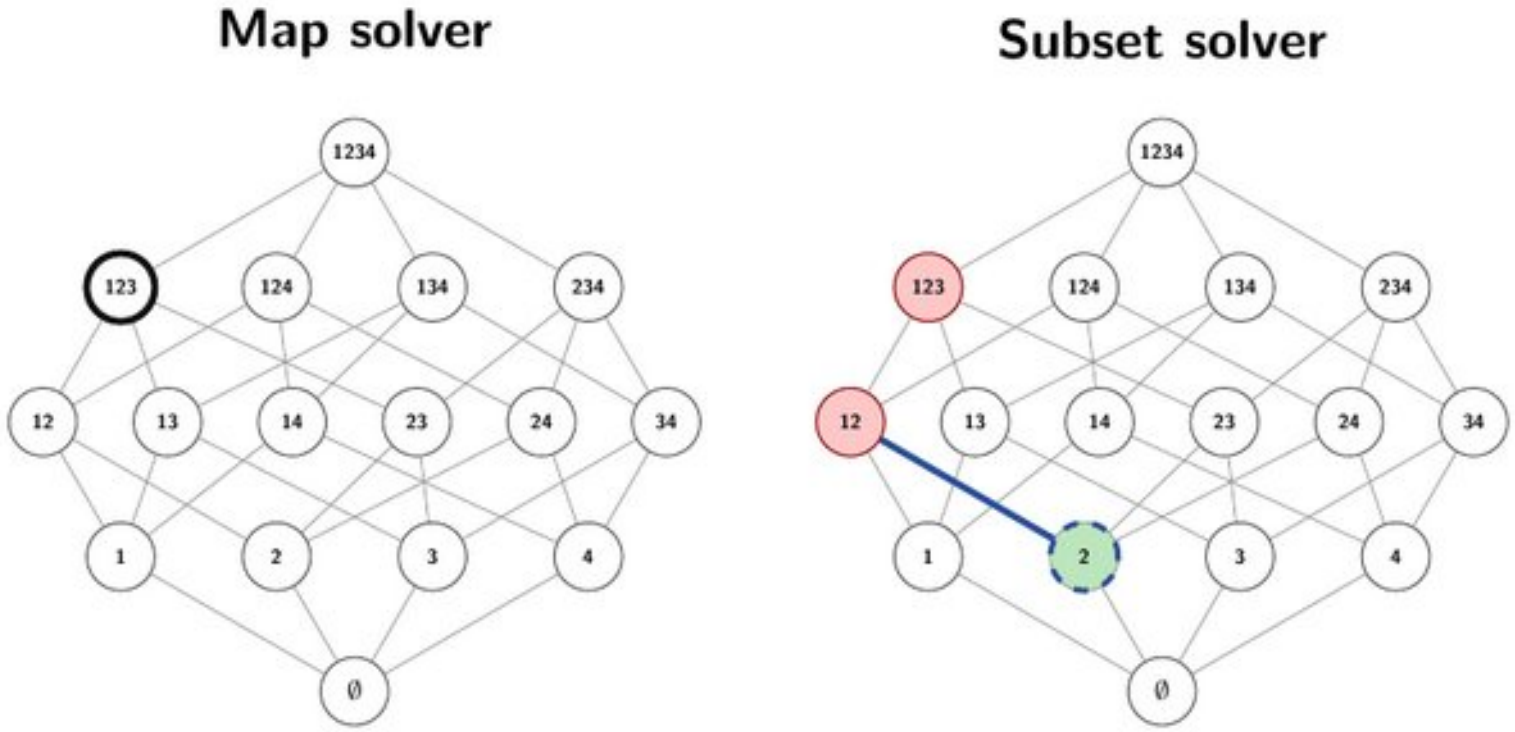
MARCO enumeration of MUS/MSS



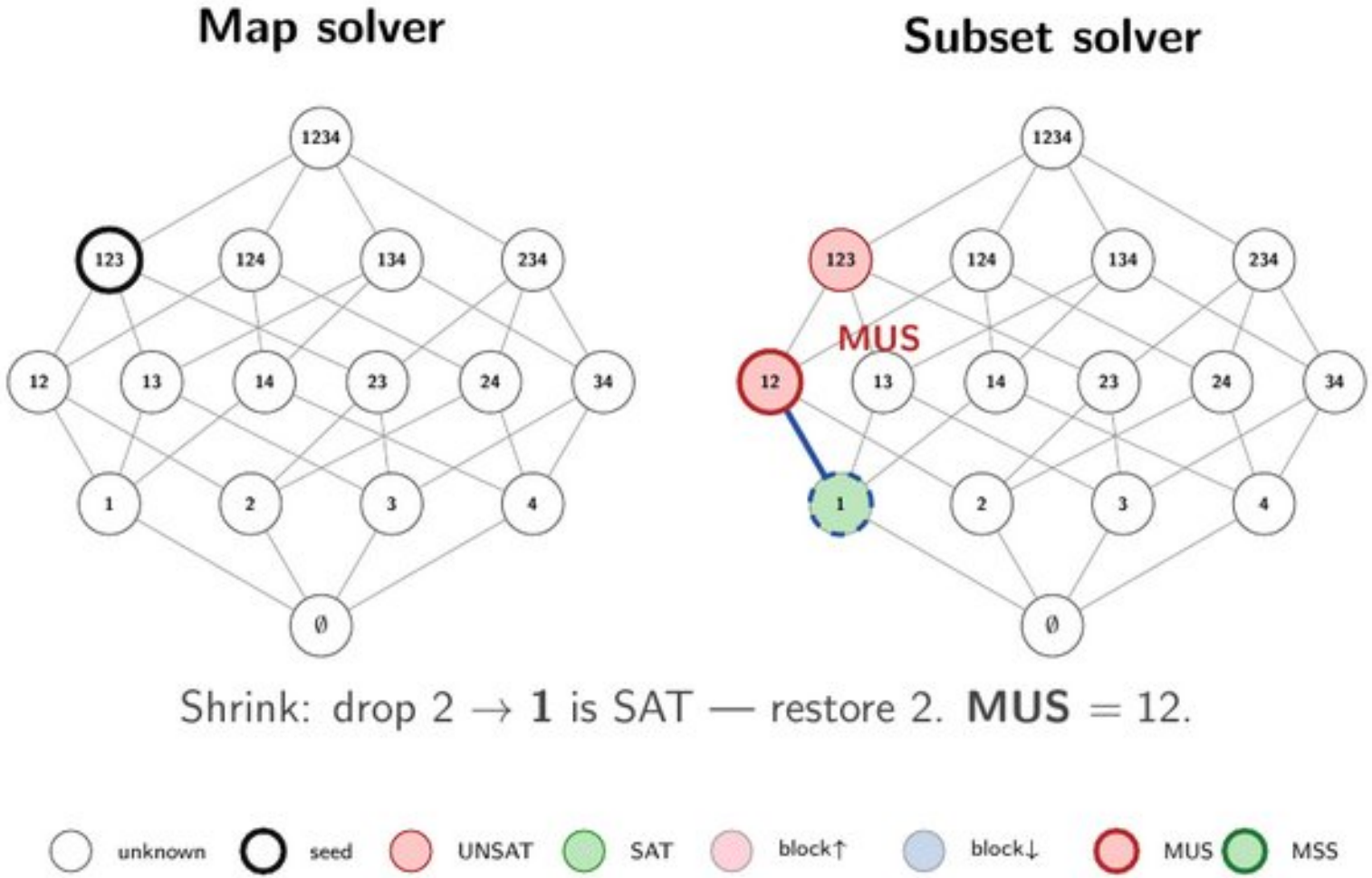
MARCO enumeration of MUS/MSS



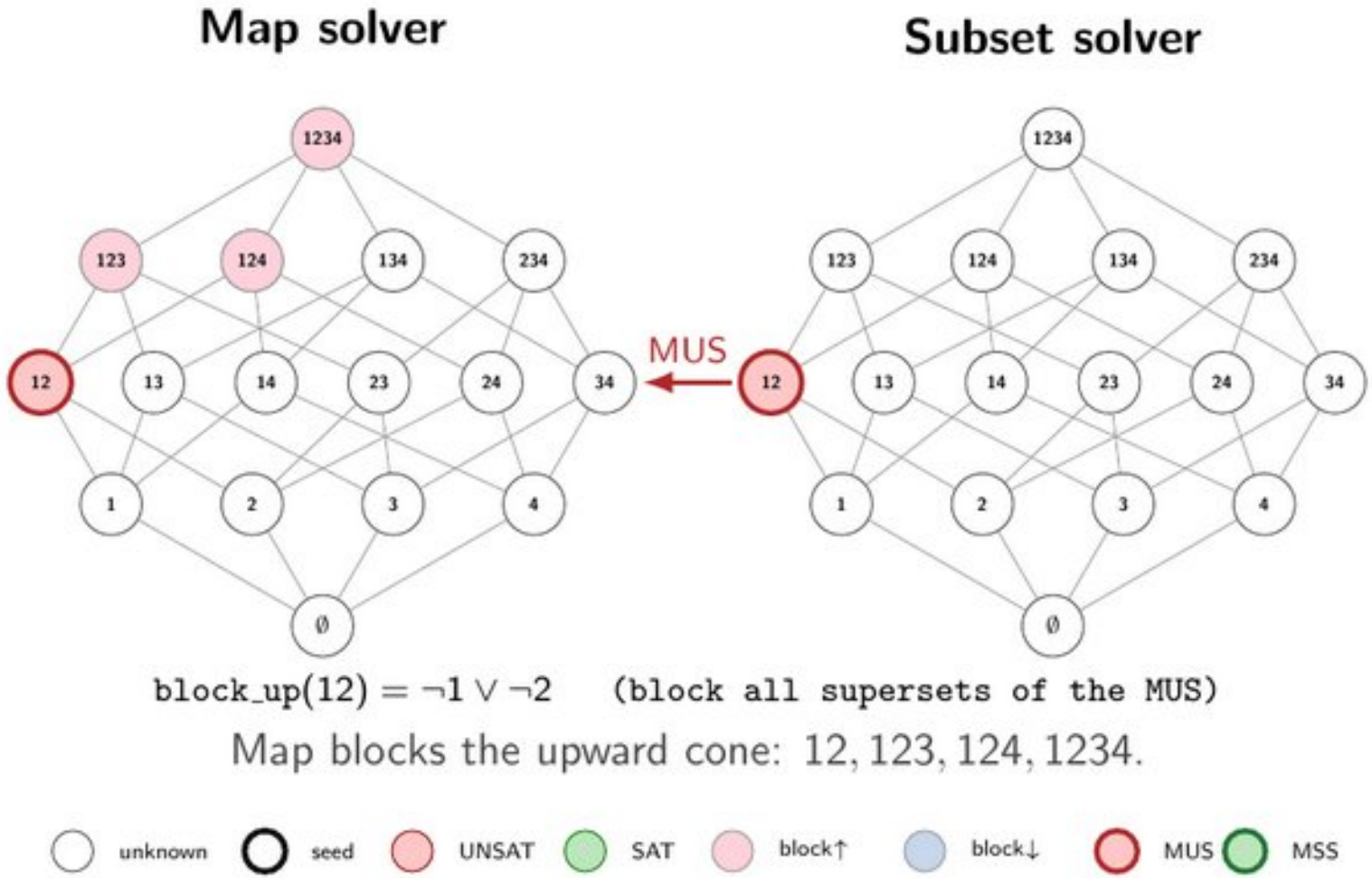
MARCO enumeration of MUS/MSS



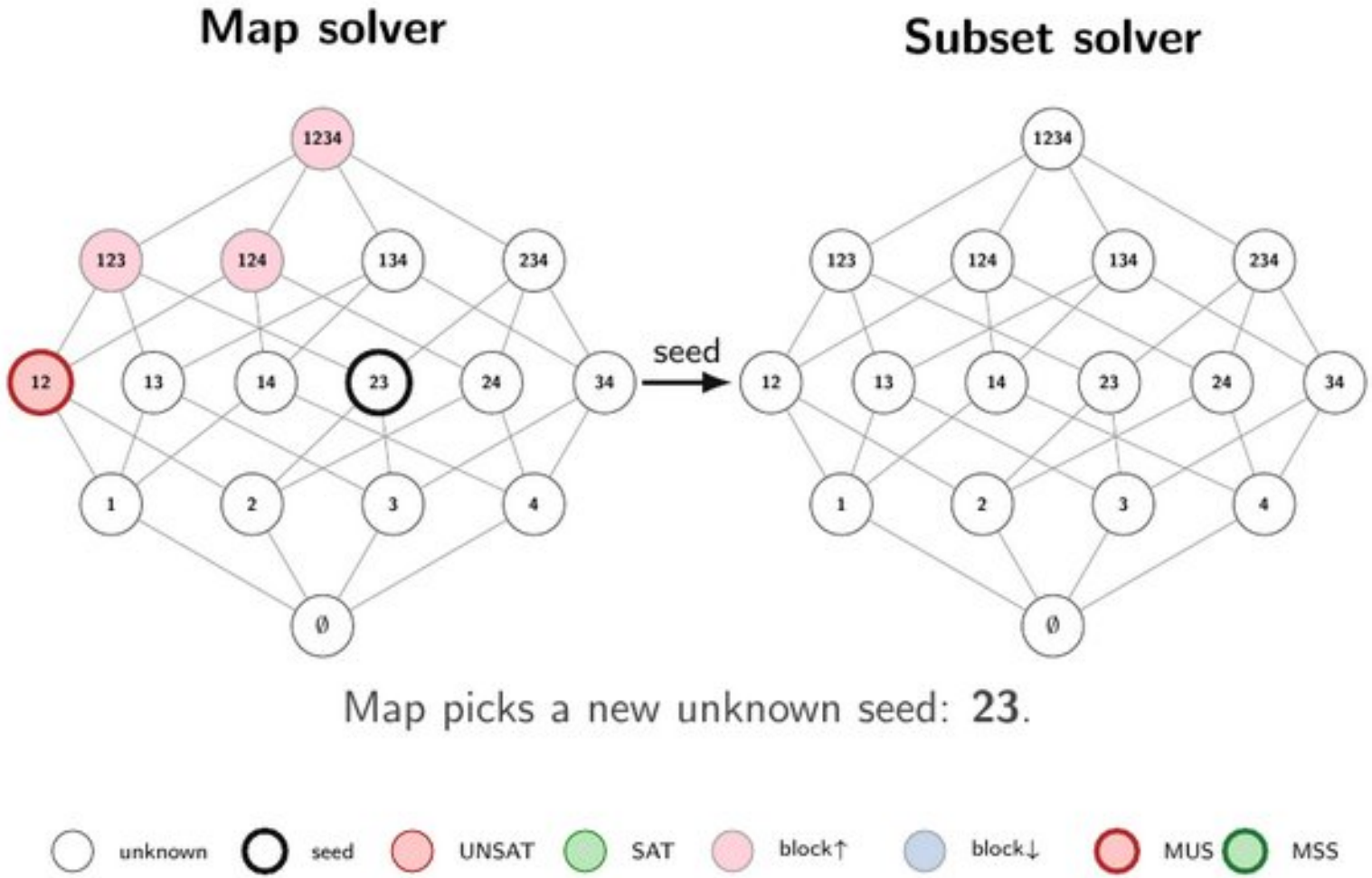
MARCO enumeration of MUS/MSS



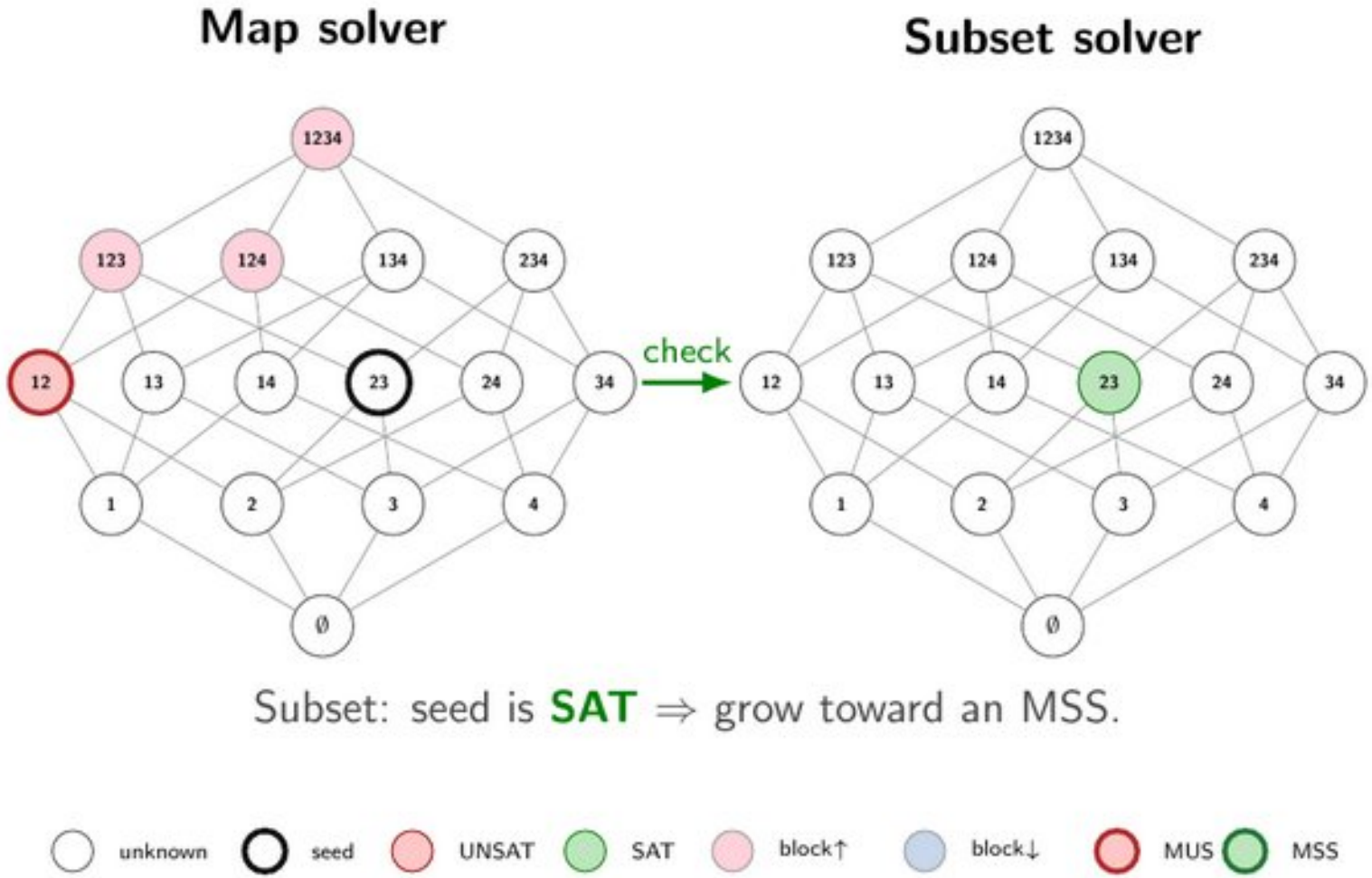
MARCO enumeration of MUS/MSS



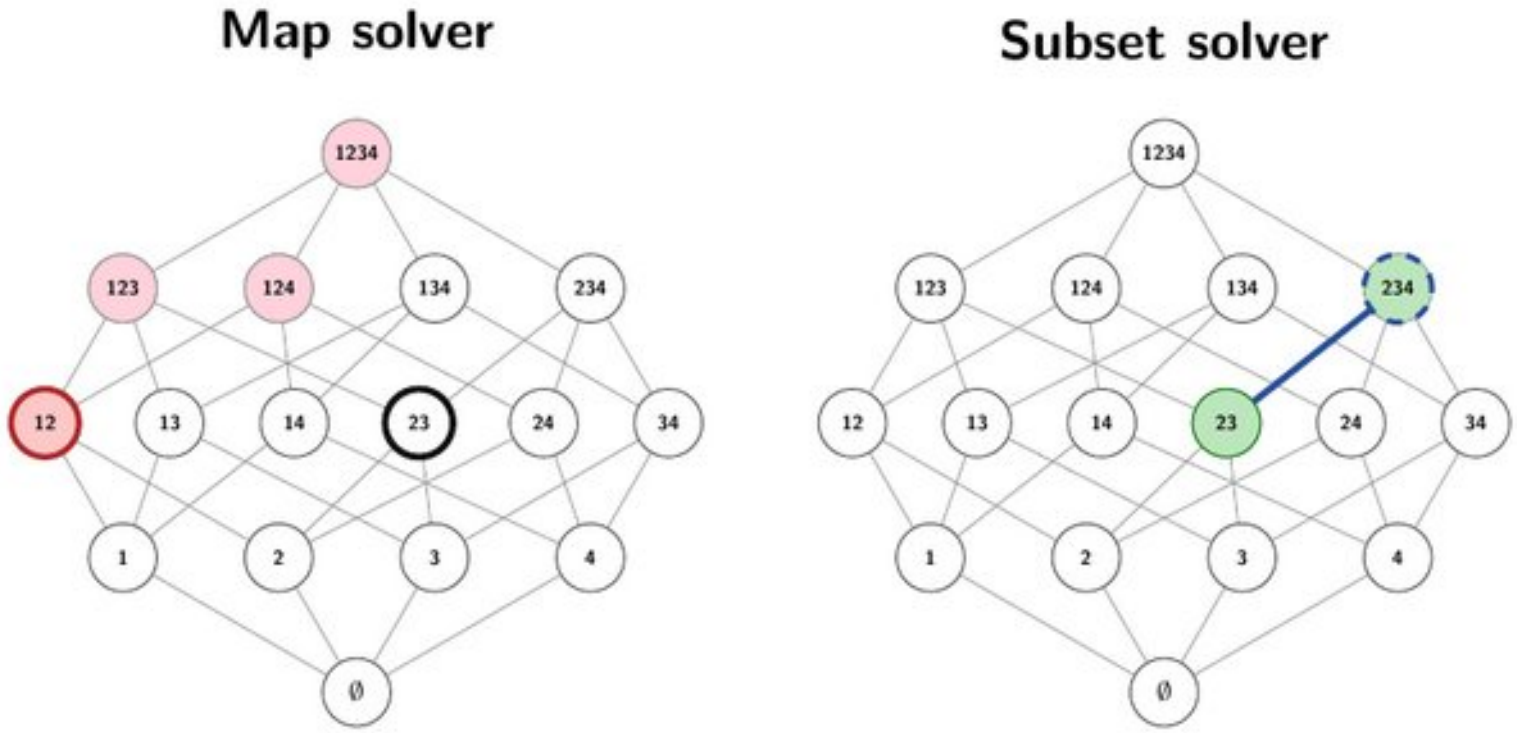
MARCO enumeration of MUS/MSS



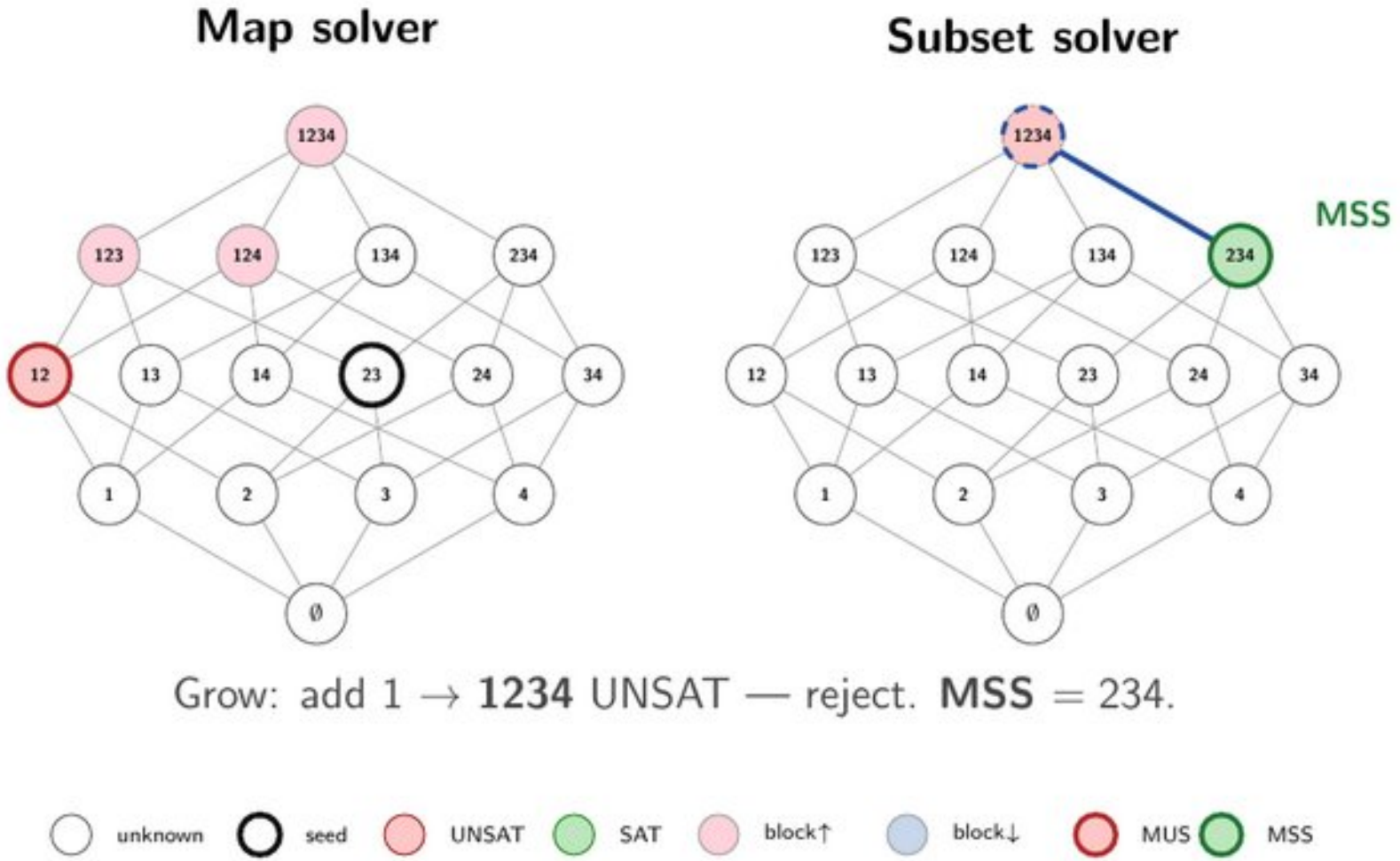
MARCO enumeration of MUS/MSS



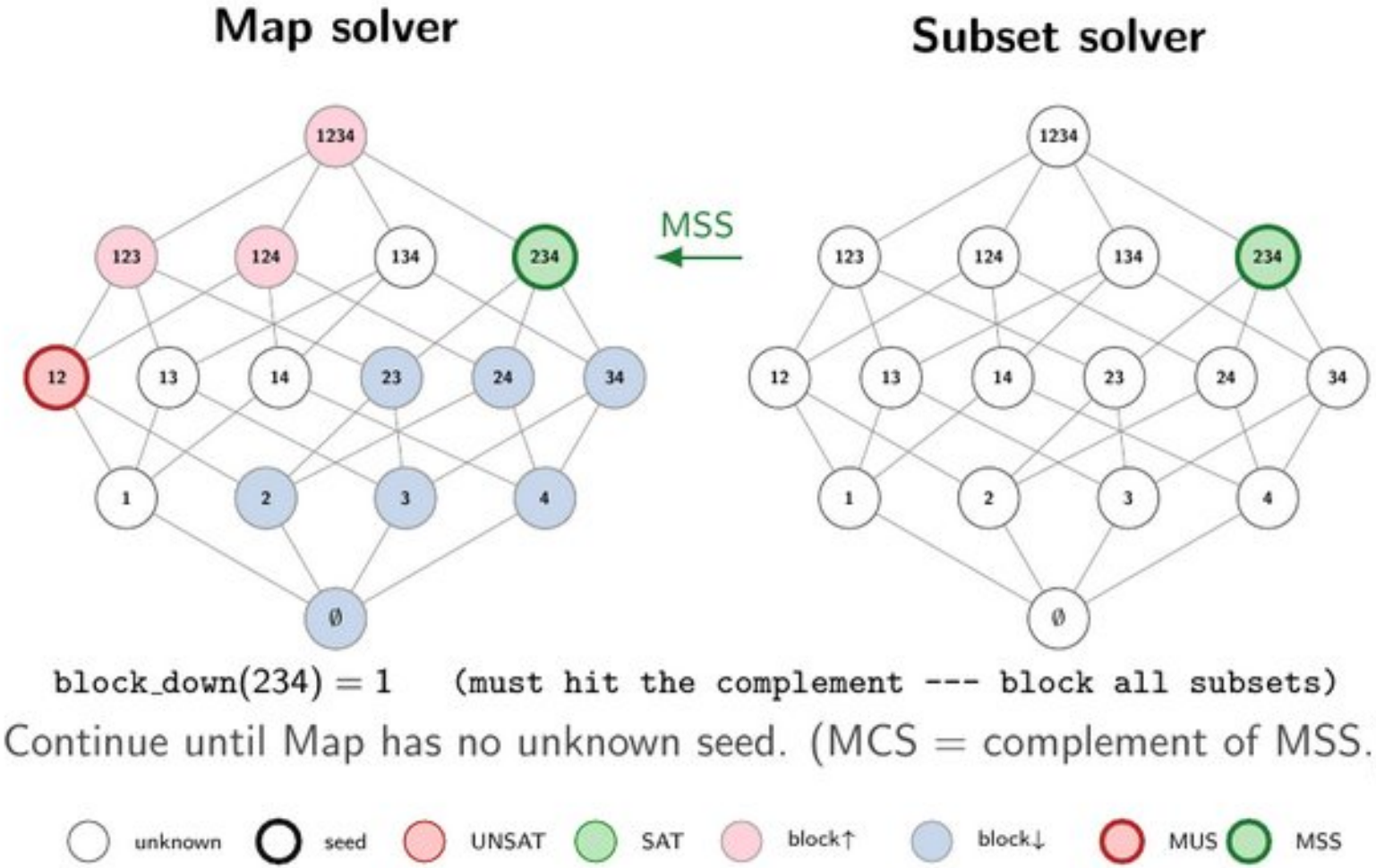
MARCO enumeration of MUS/MSS



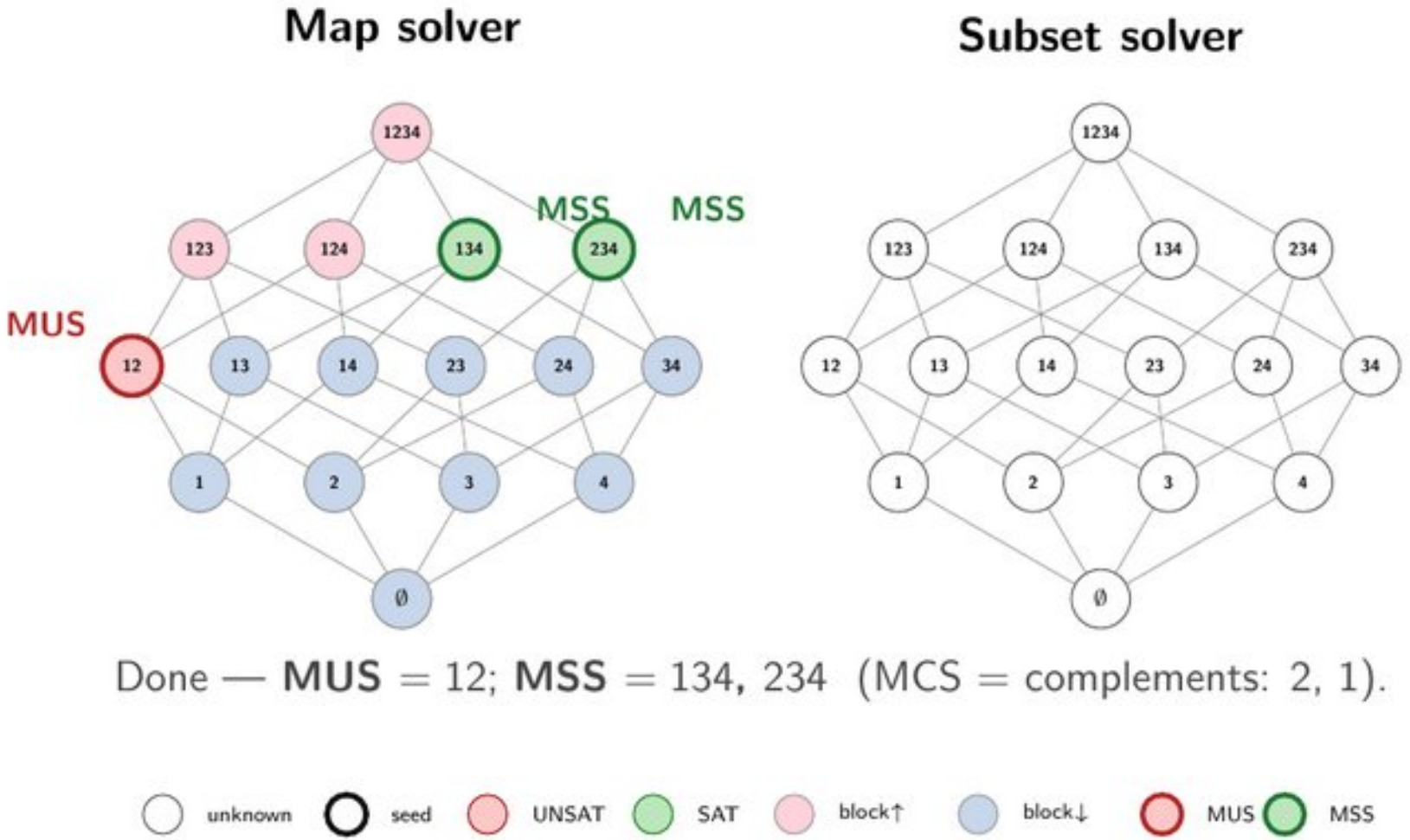
MARCO enumeration of MUS/MSS

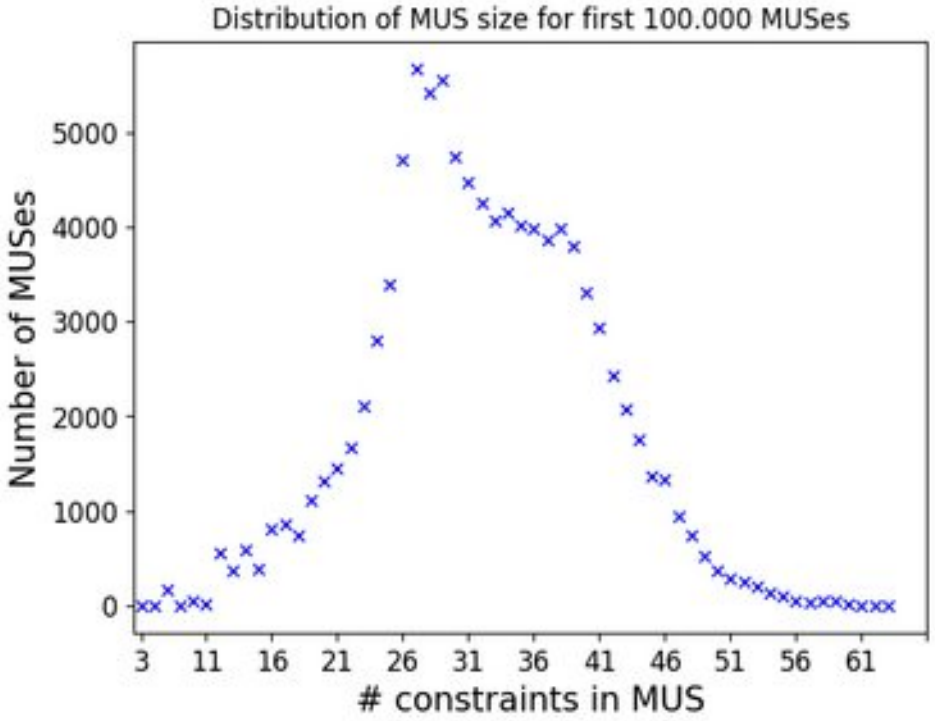


MARCO enumeration of MUS/MSS



MARCO enumeration of MUS/MSS





Many MUS'es may exist...

This problem has just 360 constraints, yet 100.000+ MUSes exist...

Which one to show?

Can we influence which MUS is found?

Influencing which MUS is found?

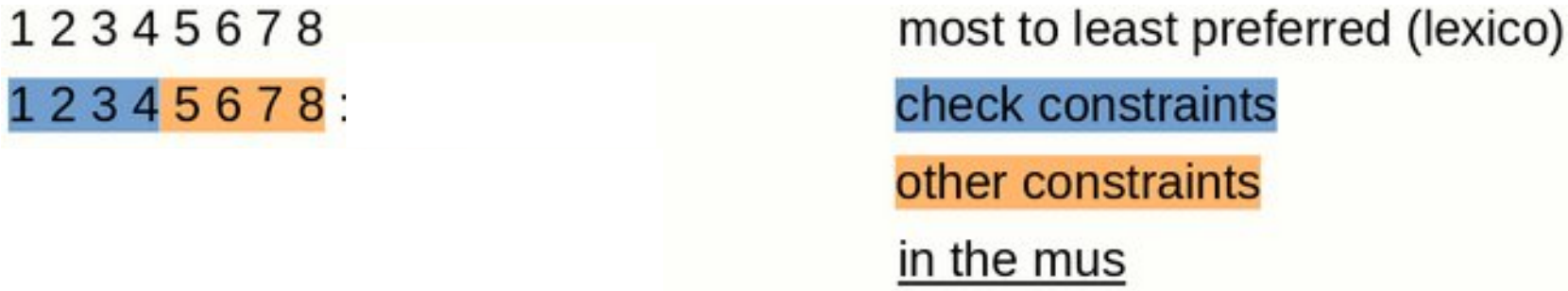
QuickXPlain algorithm *(Junker, 2004)*.

Widely used, in model-based diagnosis, recommender systems, verification, and more.

Dichotomic search given a lexicographic *preference* order over the constraints:
c_1 < c_2 < c_3 < ... < c_n (= have as few 'right-most' as possible)

Influencing which MUS is found?

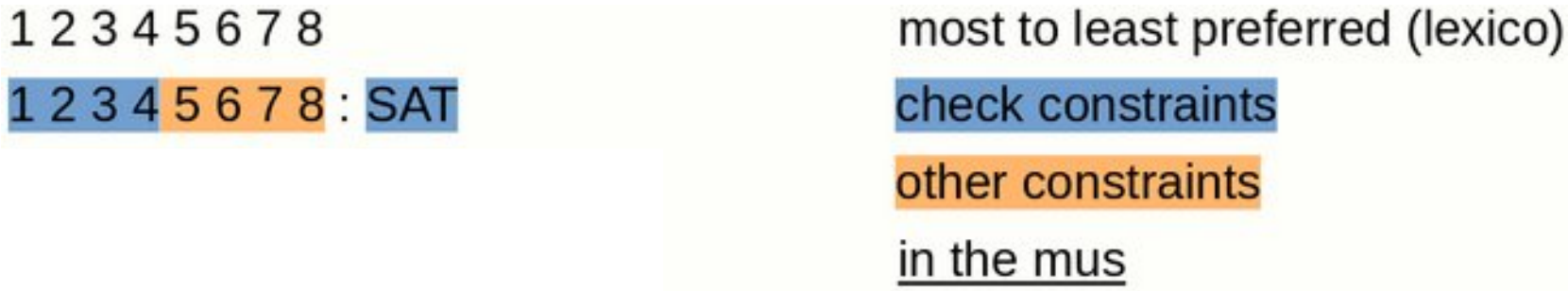
QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

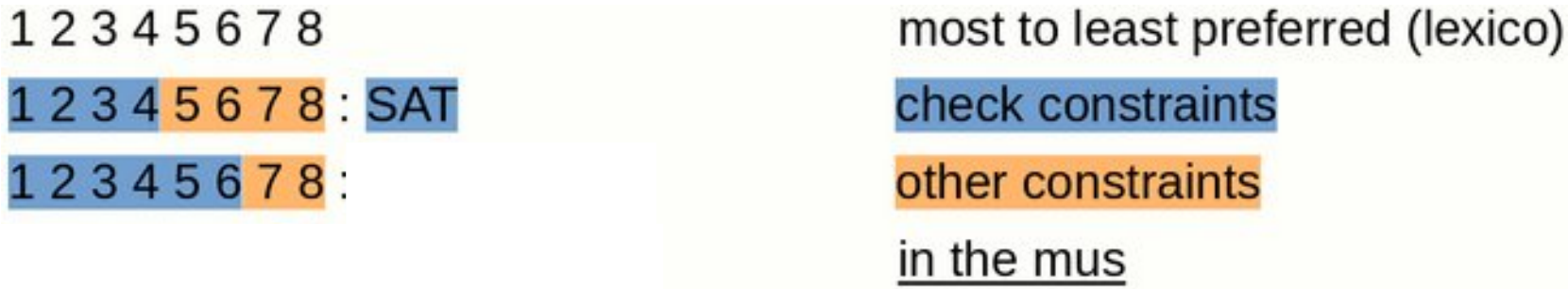
QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

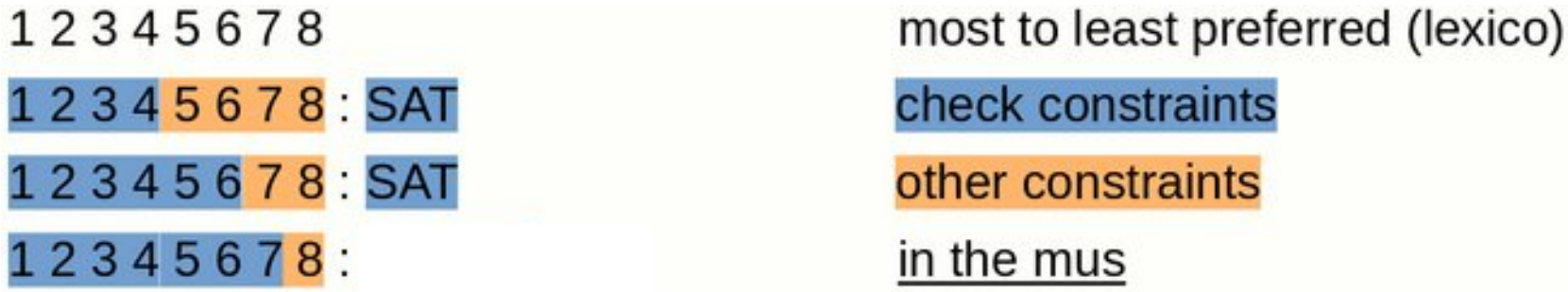
QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

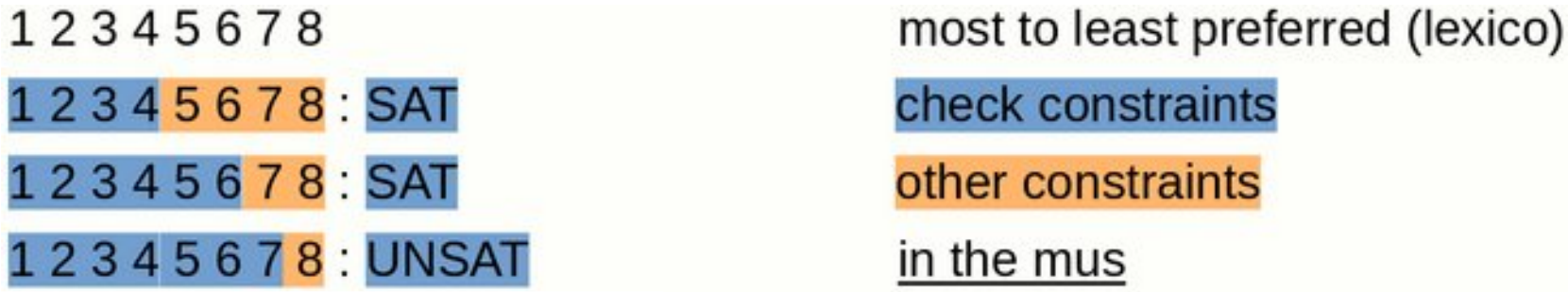
QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

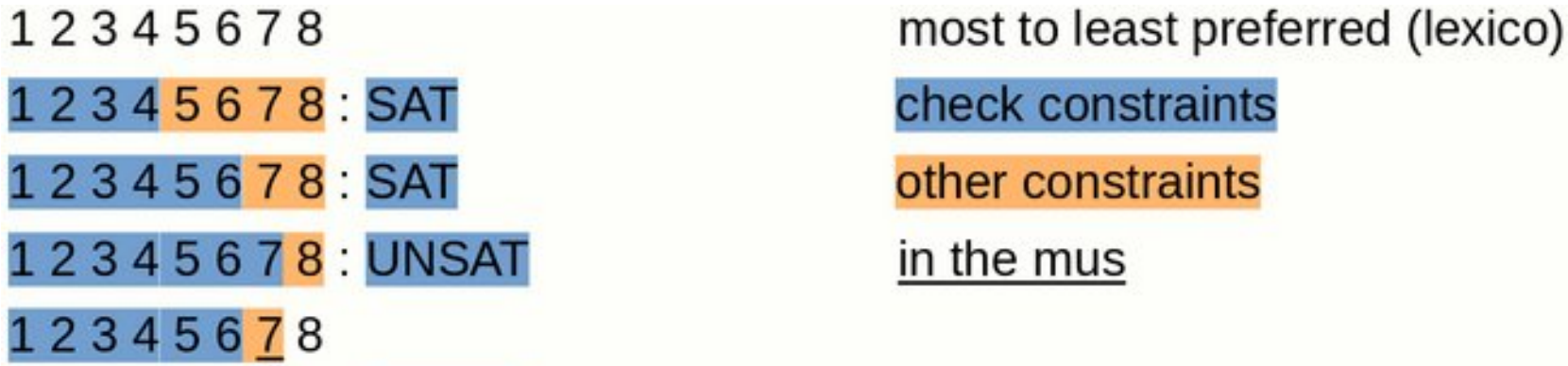
QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:

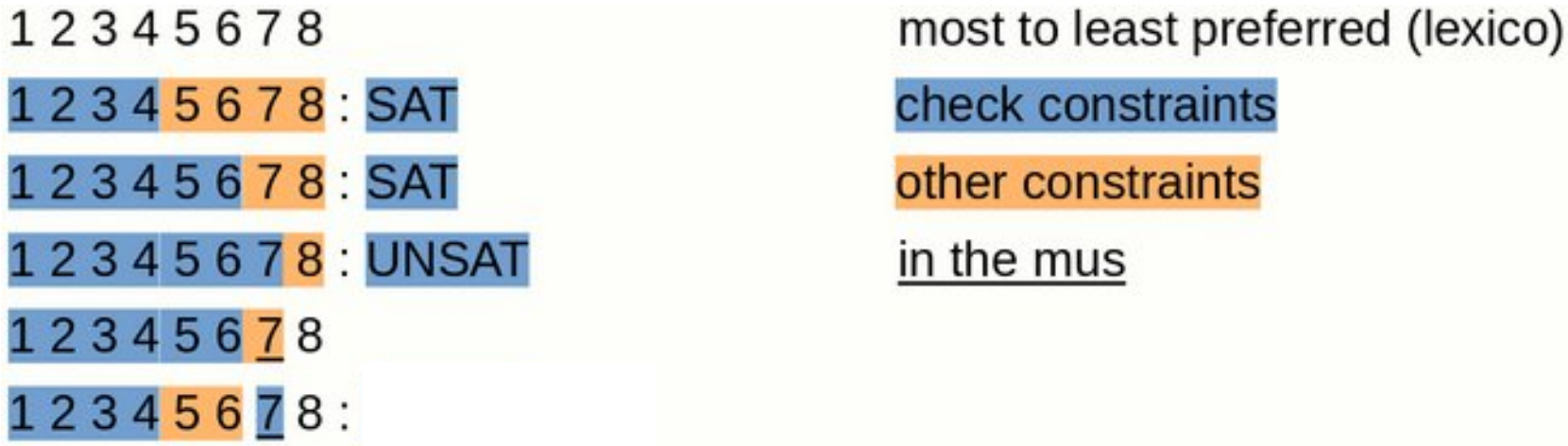


* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

1) add Boolean indicator variables

QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:

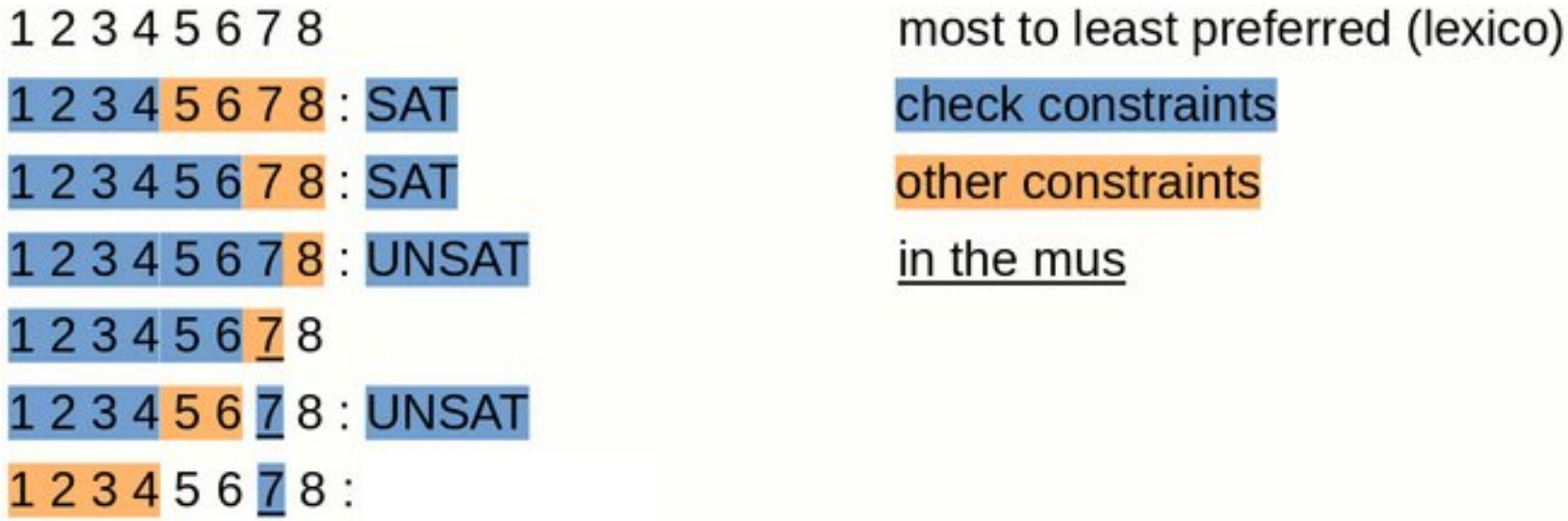


* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

1) add Boolean indicator variables

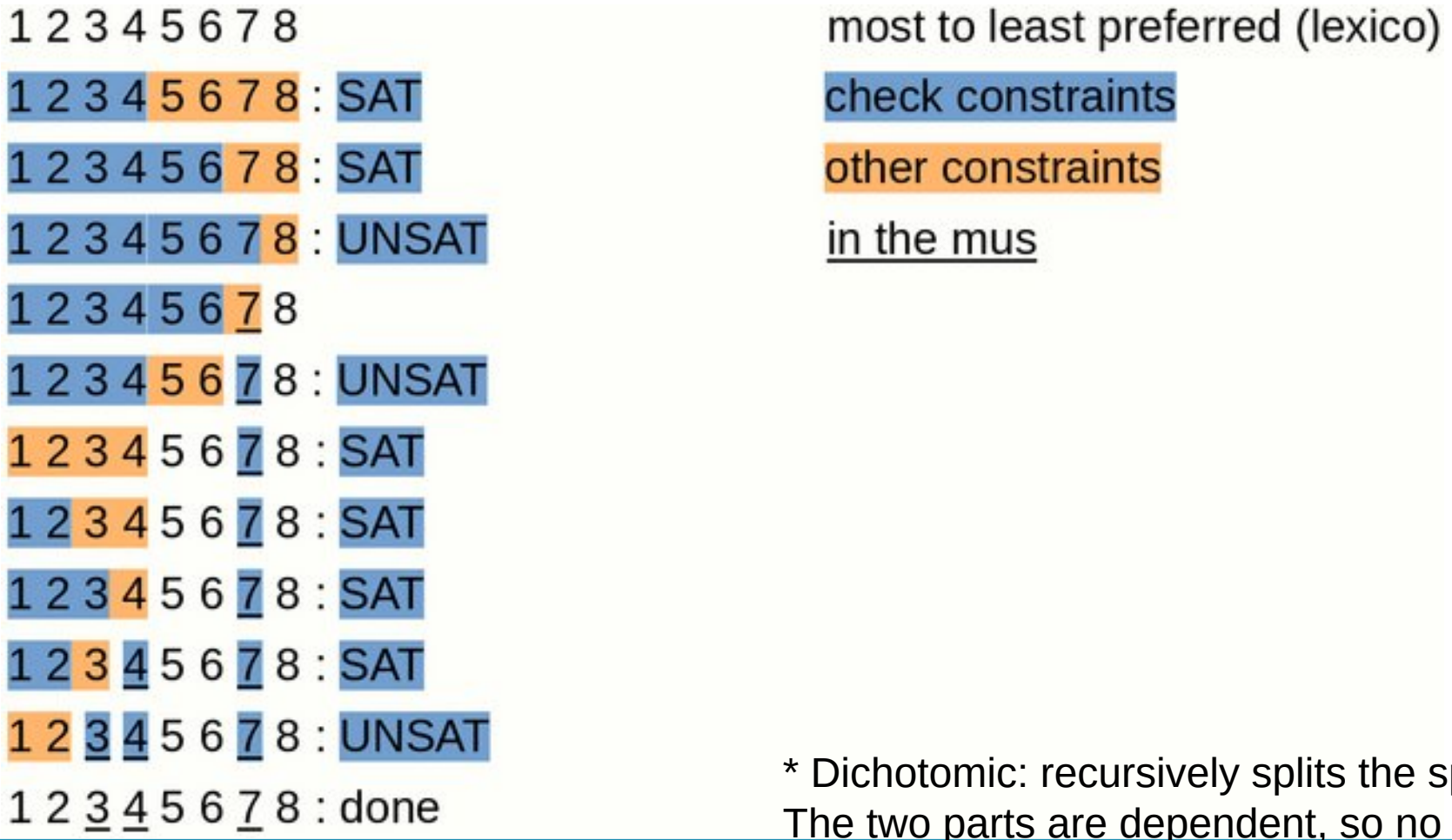
QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

Influencing which MUS is found?

QuickXPlain: Dichotomic search, given a lexicographic *preference* order over the constraints:



* Dichotomic: recursively splits the space in two.
The two parts are dependent, so no *divide-and-conquer*!

In [21]:
```python
# the order of 'soft' matters! lexicographic preference for the first ones
def quickxplain_naive(soft, hard=[], solver="ortools"):
    model, soft, assump = make_assump_model(soft, hard)
    s = cp.SolverLookup.get(solver, model)
    assert s.solve(assumptions=assump) is False, "The model should be UNSAT!"

    # the recursive call
    def do_recursion(tocheck, other, delta):
        print("@", end="")
        if len(delta) != 0 and s.solve(assumptions=tocheck) is False:
            # conflict is in hard constraints, no need to recurse
            return []

        if len(other) == 1:
            # conflict is not in 'tocheck' constraints, but only 1 'other' constraint
            return list(other)  # base case of recursion

        split = len(other) // 2  # determine split point
        more_preferred, less_preferred = other[:split], other[split:]  # split constraints into two sets

        # treat more preferred part as hard and find extra constants from less preferred
        delta2 = do_recursion(tocheck + more_preferred, less_preferred, more_preferred)
        # find which preferred constraints exactly
        delta1 = do_recursion(tocheck + delta2, more_preferred, delta2)
        return delta1 + delta2

    core = do_recursion([], list(assump), [])
    dmap = dict(zip(assump, soft))
    return [dmap[a] for a in core]
```

For the curious

Running example

QuickXPlain algorithm *(Junker, 2004)*.

Widely used, in model-based diagnosis, recommender systems, verification, and more.

```
In [22]: from cpmpy.tools.explain import quickxplain

         t0 = time.time()
         subset = quickxplain_naive(sorted(model.constraints, key=lambda c: -len(str(c))), solver="ortools")  # prefer complex c
         print("\nPrefer complex (long len(str)) constraints: Length of MUS:", len(subset))
         print(f"(in {time.time()-t0} seconds)")

         t0 = time.time()
         subset = quickxplain_naive(sorted(model.constraints, key=lambda c: len(str(c))), solver="ortools")  # prefer short cons
         print("\nPrefer short len(str) constraints: Length of MUS:", len(subset))
         print(f"(in {time.time()-t0} seconds)")
```

```
.....
Prefer complex (long len(str)) constraints: Length of MUS: 39
(in 11.042917013168335 seconds)
.....
Prefer short len(str) constraints: Length of MUS: 3
(in 2.0805091857910156 seconds)
```

Best algorithm when you have a **non-incremental solver**! (best-case logarithmic nr solve calls)

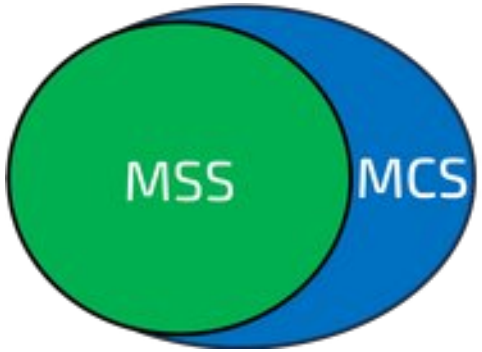
Finding a *smallest* or weighted smallest MUS?

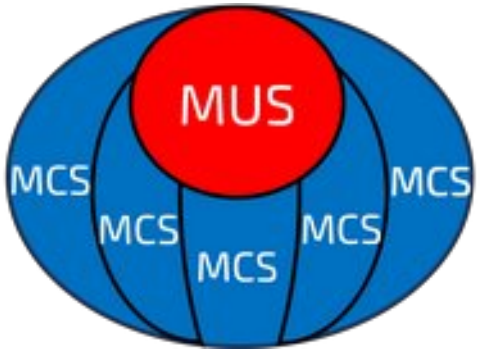
Give every constraint a weight: OUS: **Optimal Unsatisfiable Subsets** *(Gamba, Bogaerts, Guns, 2021).*

An extension of the **implicit hitting-set approach** for finding *smallest* MUSes *(Ignatiev, Janota, Marques-Silva, 2014)*

Key properties we will need:

1. If a subset is SAT, can *grow* it to a Maximal Satisfiable Subset (MSS)
2. The complement of a MSS is a Minimal Correction Subset (MCS)
3. Theorem: A MUS is a hitting set of all MCSes (= shares at least 1 constraint with every MCS)



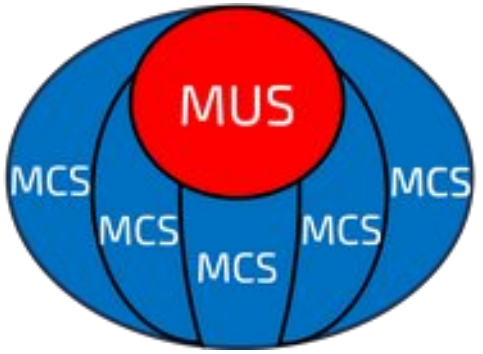


Finding a *smallest* or weighted smallest MUS?

Give every constraint a weight: OUS: **Optimal Unsatisfiable Subsets** *(Gamba, Bogaerts, Guns, 2021)*.

Theorem: A MUS is a hitting set of all MCSes (= covers at least 1 constraint in every MCS)

If we have an **explicit** set of all MCSes: OUS is **optimal hitting set** of this set.



$$\text{Maximize } W^{*}C$$

$$\text{s.t. } C_{j1} \mid C_{j2} \mid \ldots \mid C_{jm} \quad \forall \{j_1, j_2, \ldots, j_m\} \in \text{SetsToHit}$$

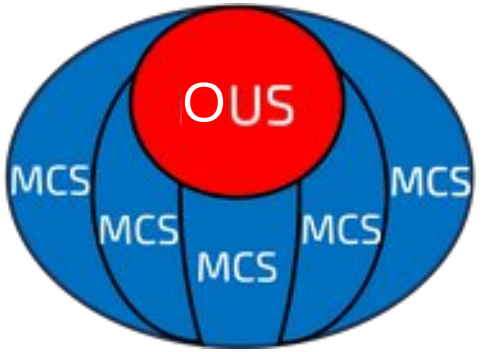
$$C_i \in \{0,1\} \quad \forall i \in \text{Constraints}$$

Finding a *smallest* or weighted smallest MUS?

Give every constraint a weight: OUS: **Optimal Unsatisfiable Subsets** *(Gamba, Bogaerts, Guns, 2021)*.

If we have an **explicit** set of all MCSes: OUS is **optimal hitting set** of this set.

$$\text{Maximize } W^*C$$
$$\text{s.t. } C_{j1} \mid C_{j2} \mid \ldots \mid C_{jm} \quad \forall \{j_1, j_2, \ldots, j_n\} \in \text{SetsToHit}$$
$$C_i \in \{0,1\} \quad \forall i \in \text{Constraints}$$



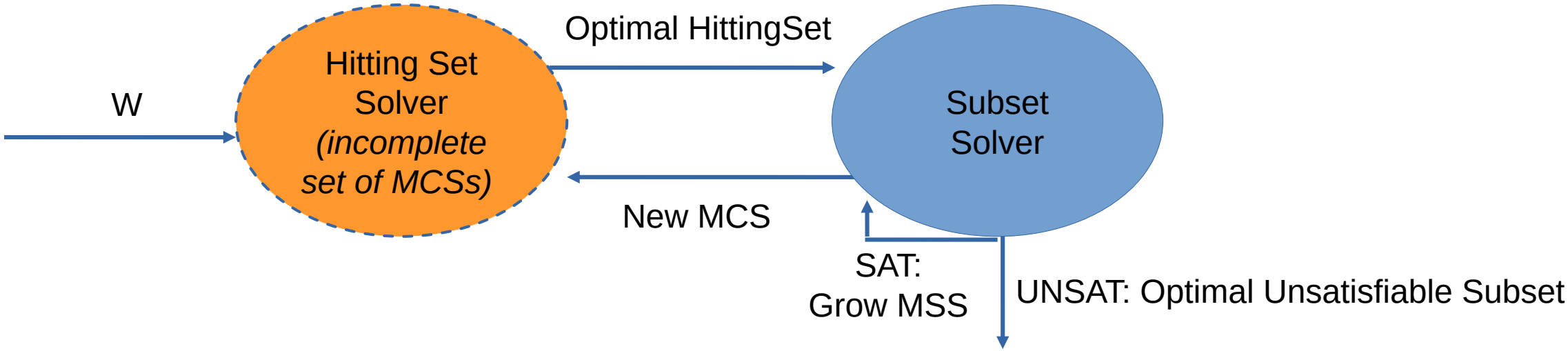
We do **not** have the explicit set… Let's do a **Benders decomposition** and generate it on-the-fly:

1) HS solver: given a subset of SetsToHit (initially empty):
   generate an optimal hitting set/candidate OUS
2) Subset solver *(checks UNSAT and generates cuts for the HS solver)*:
   - If UNSAT: found an optimal subset that is unsatisfiable → OUS!
   - If SAT: grow to an MSS and add the complement (=MCS) to SetsToHit of HS solver

Efficiently optimising which MUS is found?

OUS: Optimal Unsatisfiable Subsets *(Gamba, Bogaerts, Guns, 2021)*. Every constraints has a weight.

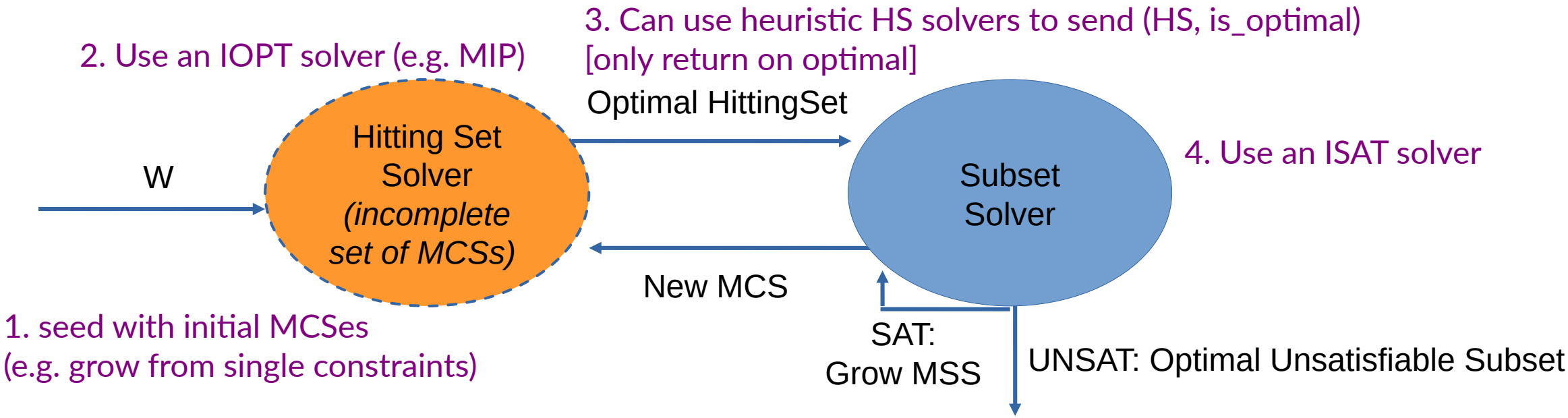
Known in the SAT community as an **implicit hitting-set algorithm.**



Efficiently optimising which MUS is found? Efficiently: HS solver is bottleneck

OUS: Optimal Unsatisfiable Subsets *(Gamba, Bogaerts, Guns, 2021)*. Every constraints has a weight.

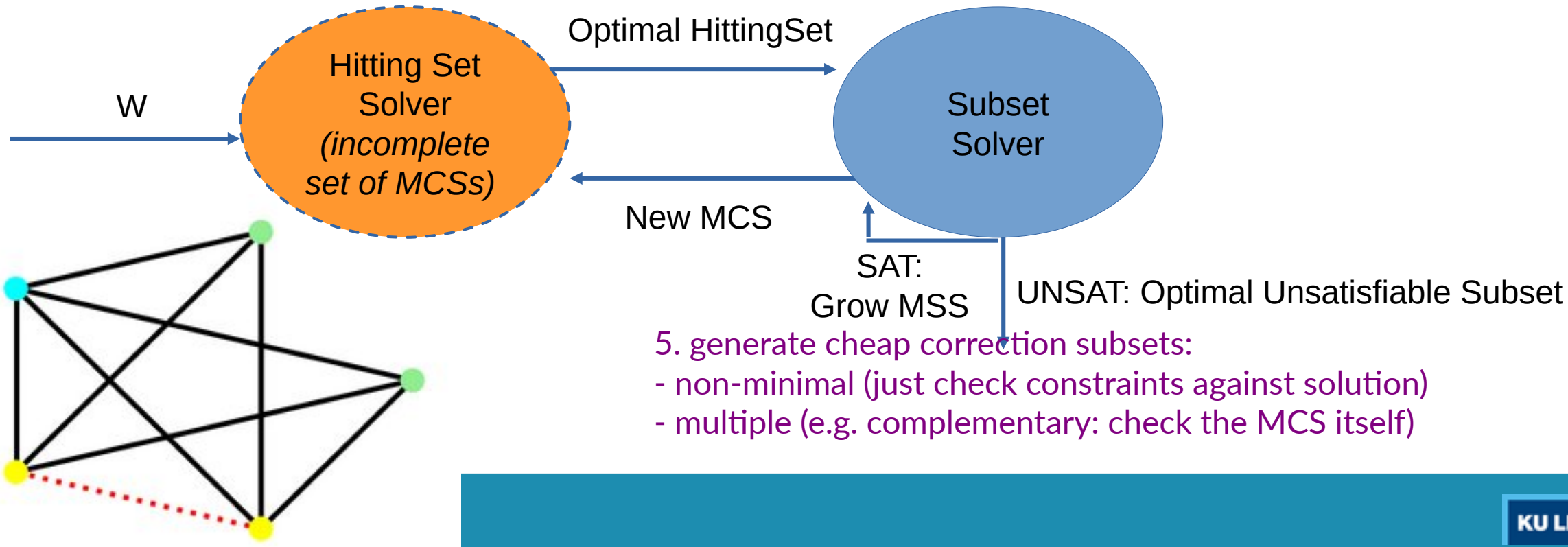
Known in the SAT community as an **implicit hitting-set algorithm.**



Efficiently optimising which MUS is found? Efficiently: HS solver is bottleneck

OUS: Optimal Unsatisfiable Subsets *(Gamba, Bogaerts, Guns, 2021)*. Every constraints has a weight.

Known in the SAT community as an **implicit hitting-set algorithm.**



Running example, nurse rostering

OUS: Optimal Unsatisfiable Subsets *(Gamba, Bogaerts, Guns, 2021)*. Every constraint has a weight.

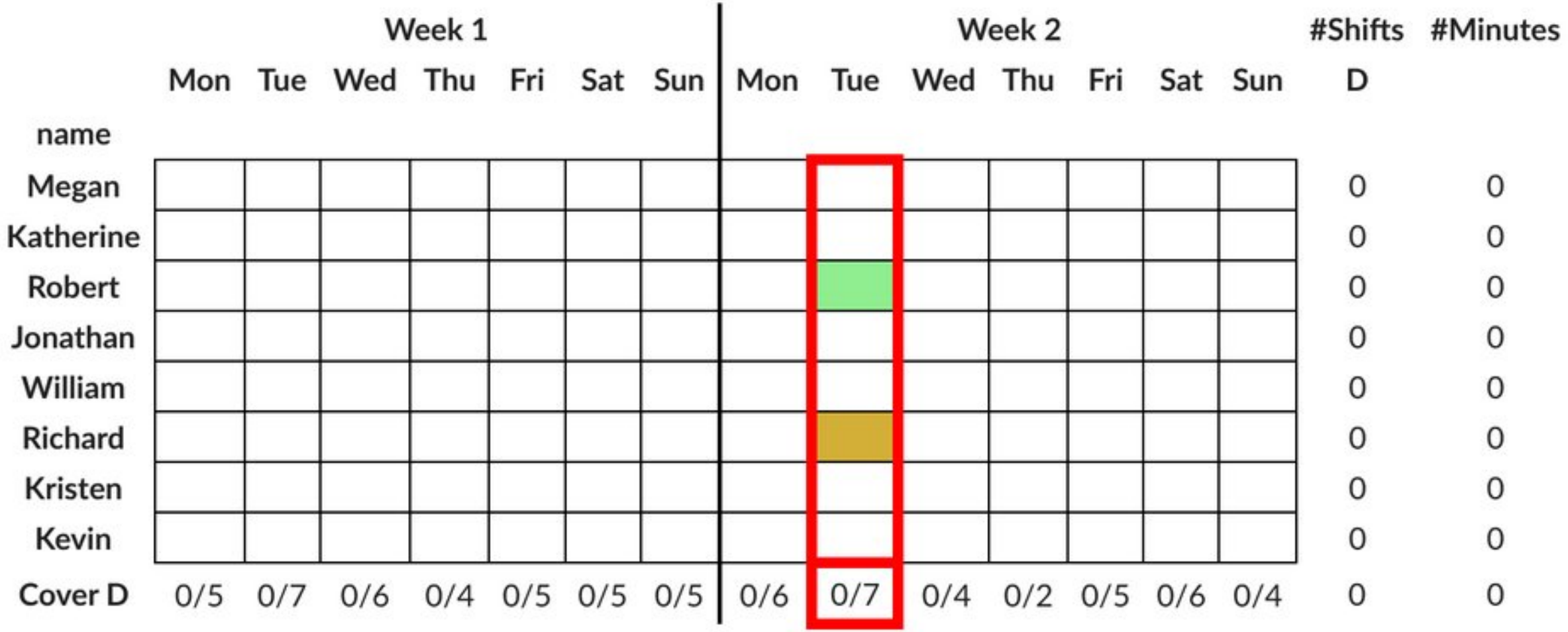
```
In [23]: from cpmpy.tools.explain import optimal_mus

         small_subset = optimal_mus(model.constraints, solver=sat_solver, hs_solver=hs_solver)

         print("Length of sMUS:", len(small_subset))
         for cons in small_subset:
             print("-", cons)
```

Length of sMUS: 3
- Robert has a day off on Tue 2
- Richard requests not to work shift D on Tue 2
- Shift D on Tue 2 must be covered by 7 nurses out of 8

```
In [24]: visualize_constraints(small_subset, nurse_view, factory)
```



Step-wise explanations, each step is an O(C)US

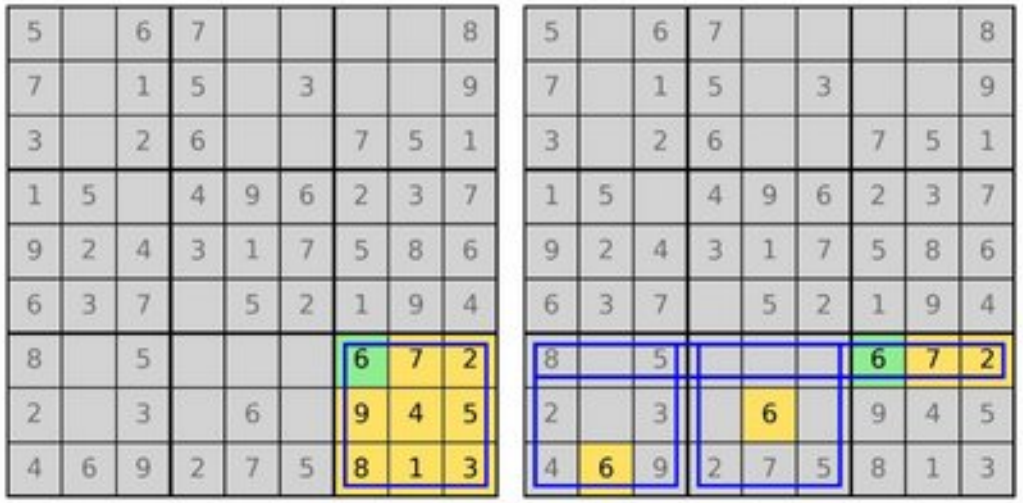
OUS: Optimal Unsatisfiable Subsets *(Gamba, Bogaerts, Guns, 2021)*. Every constraint has a weight.

=> how to determine the weights?

Can we learn them from the user?



Preference Elicitation for Step-Wise Explanations in Logic Puzzles



Features	Left	Right
Facts	8	4
Block Cons.	1	2
Row Cons.	0	1
Column Cons.	0	0

[Dragone, Teso and Passerini, 2018]

Algorithm 1: Constructive Preference Elicitation framework

Input: Problems G, Initial weights w^1, No. of iterations T
Output: Learned weights w^t

1: **for** $t = 1$ to T **do**
2: $\quad \mathcal{Y} \leftarrow$ Instance Selection(G)
3: $\quad (y^1, y^2) \leftarrow$ Query Generation$(\mathcal{Y}, w^t)$
4: $\quad (y^+, y^-) \leftarrow$ Label(y^1, y^2)
5: $\quad w^{t+1} \leftarrow$ Weight Update(y^+, y^-, w^t)
6: **end for**
7: **return** w^T

+ smart ‘fact to explain’ generation
+ UCB-style weighting
+ subobjective normalisation
+ non-domination criteria

Explanation generation as Multi-Objective optimisation!

[Foschini, Defresne, Gamba, Bogaerts, Guns. AAAI 2026, **best poster award**]

Recap: deductive explanations for UNSAT

Algorithm	Characteristics	First solver call	Subsequent calls	Efficiency
Deletion-based MUS	Trivial to implement, arbitrary MUS	All constraints except one	Linear decrease	Efficient for assumption-based incremental solvers.
Prooflog trimming + MUS	Requires a solver supporting prooflogging No assumption variables needed!	All constraints	/	Prooflogging has some overhead, and prooflogs can be long.
QuickXplain	Optional lexicographic preference over constraints.	Half the constraints (n/2)	Divide and conquer	Reasonably efficient for non-incremental solvers.
sMUS/OUS	Guaranteed smallest/optimal US. One weight per constraint.	One constraint	Slow increase	Only calls solver with small subsets of constraints.

Outline of the lectures

Introduction to XCP

Deductive explanations for UNSAT

- **Break**

Deductive explanations for SAT/OPT

Counterfactual explanations for UNSAT/SAT/OPT

Use case, challenges and conclusion

Outline of the lectures

Introduction to XCP

Deductive explanations for UNSAT

Break

- Deductive explanations for SAT/OPT

Counterfactual explanations for UNSAT/SAT/OPT

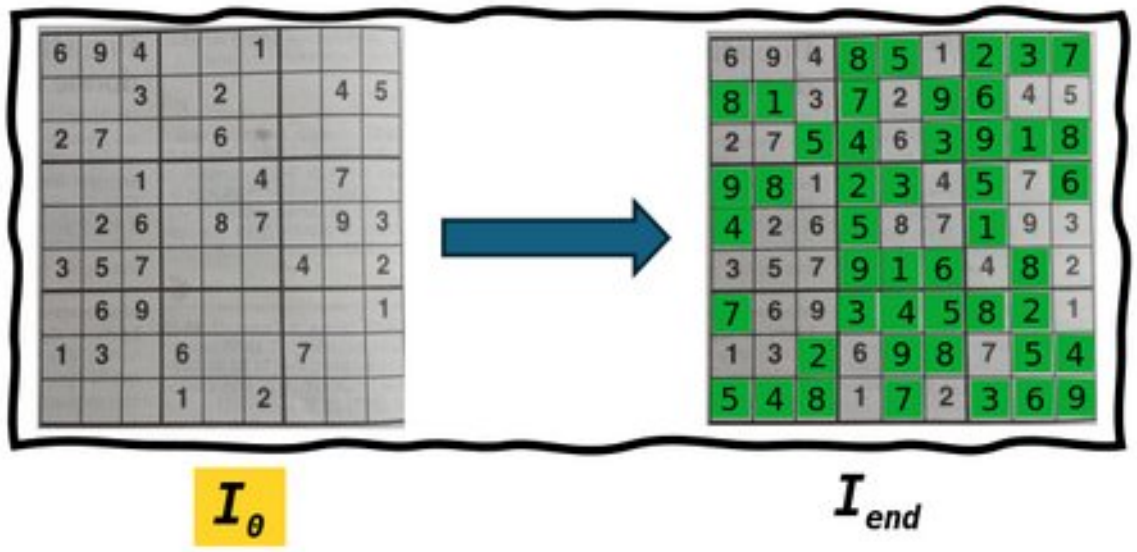
Use case, challenges and conclusion

Deductive Explanations for SAT problems

Deductive Explanations for SAT problems

How to explain satisfiability of a constraint satisfaction problem (CSP) in a human-understandable way?

Explain the maximal consequence of a CSP



C — ***constraints***: e.g. all different values on a row, block, and column

I_θ — initial **(partial) interpretation** (facts):
- ∅
- cells[1,1]=6,cells[1,2]=9,…,cells[8,7]=7

I_{end} — **maximal consequence**: *precision-maximal partial interpretation*

$$C \wedge I_\theta \models I_{end}$$

Deductive Explanations for SAT problems

Explaining logical consequences

Logical consequence: a variable assignment entailed by the constraints and the current partial assignment

One step (user-level variables and constraints only):

partial assignment & subset of constraints => domain change

Maximal consequence: precision- maximal partial assignment

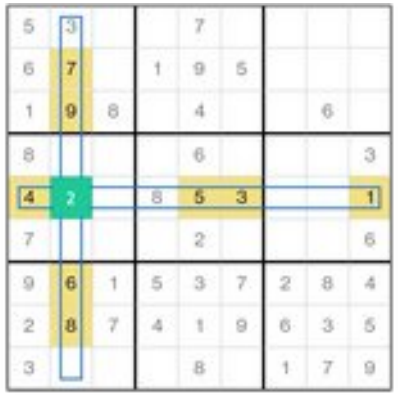
- Maximal consequence = intersection of all possible solutions
- If solution is unique, maximal consequence = unique solution

all constraints => shared domain changes across all solutions

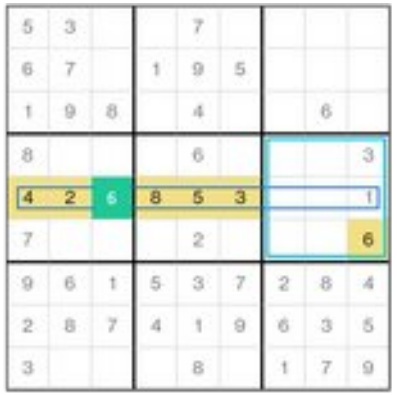
Does not require iterating over all solutions!
Iteratively 'block' shared part only

Deductive Explanations for SAT problems

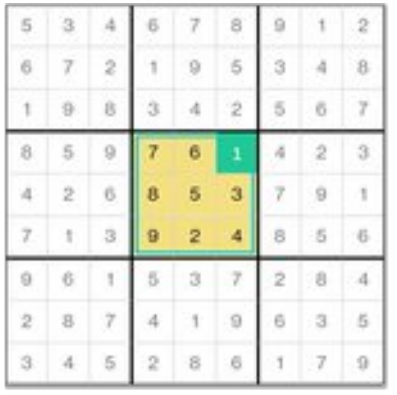
Bogaerts, Bart, Emilio Gamba, and Tias Guns. "A framework for **step-wise explaining** how to solve constraint satisfaction problems." Artificial Intelligence 300 (2021): 103550.



$cell_{2,2} = 7 \wedge cell_{3,2} = 9 \ldots \wedge$
$alldiff(row_5) \wedge alldiff(col_2) \Rightarrow cell_{5,2} = 2$



$cell_{5,1} = 4 \wedge cell_{5,2} = 2 \ldots \wedge$
$alldiff(row_5) \wedge alldiff(block_6) \Rightarrow cell_{5,3} = 6$

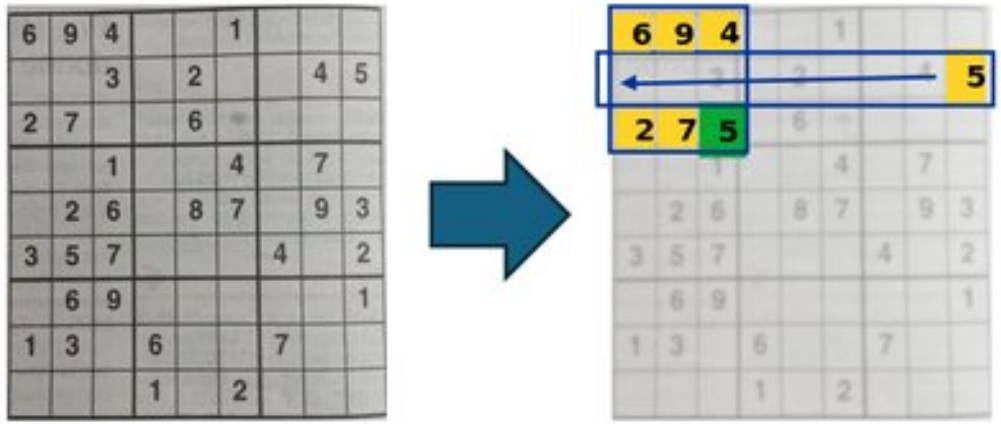


$cell_{4,4} = 7 \wedge cell_{4,5} = 6 \ldots \wedge$
$alldiff(block_5) \Rightarrow cell_{5,3} = 6$

Sudoku has a unique solution
= maximal consequence

Deductive Explanations for SAT problems

Bogaerts, Bart, Emilio Gamba, and Tias Guns. "A framework for step-wise explaining how to solve constraint satisfaction problems." Artificial Intelligence 300 (2021): 103550.



I_0 I_1

An EXPLANATION (E_i, S_i, N_i) of an inference step ***explains***:

$$E_i \wedge S_i \models N_i$$

E_i $\mathbf{E_i \subseteq I_i}$ *The explaining facts are a subset of what was previously derived*

$\mathbf{E_0}$ = {cells[1,1] = 6, cells[1,2] = 9, cells[1,3] = 4, cells[3,1] = 2, cells[3,2] = 7, cells[2,9] = 5}

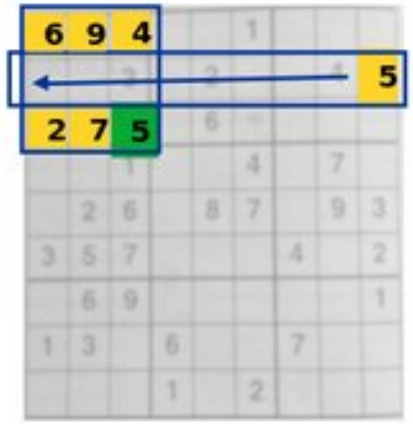
S_i $\mathbf{S_i \subseteq C}$ *A subset of the problem constraints*

$\mathbf{S_0}$ = {alldiff(cells[1:3, 1:3]), alldiff(cells[2, :])}

N_i $\mathbf{I_{i+1} \setminus I_i}$ *All newly derived information entailed by this explanation*

$\mathbf{N_0}$ = {cells[3,3] = 5}

Deductive Explanations for SAT problems



I_1

An EXPLANATION (E_i, S_i, N_i) of an inference step ***explains***:

$$E_i \wedge S_i \models N_i$$

E_i	$\mathbf{E_i} \subseteq \boldsymbol{I_i}$	*The explaining facts are a subset of what was previously derived*
S_i	$\mathbf{S_i} \subseteq \boldsymbol{C}$	*A subset of the problem constraints*
N_i	$\boldsymbol{I_{i+1}} \setminus \boldsymbol{I_i}$	*All newly derived information entailed by this explanation*

We want each explanation step to be as simple as possible.

How? $\mathcal{MUS}(I \wedge C \wedge -n)$

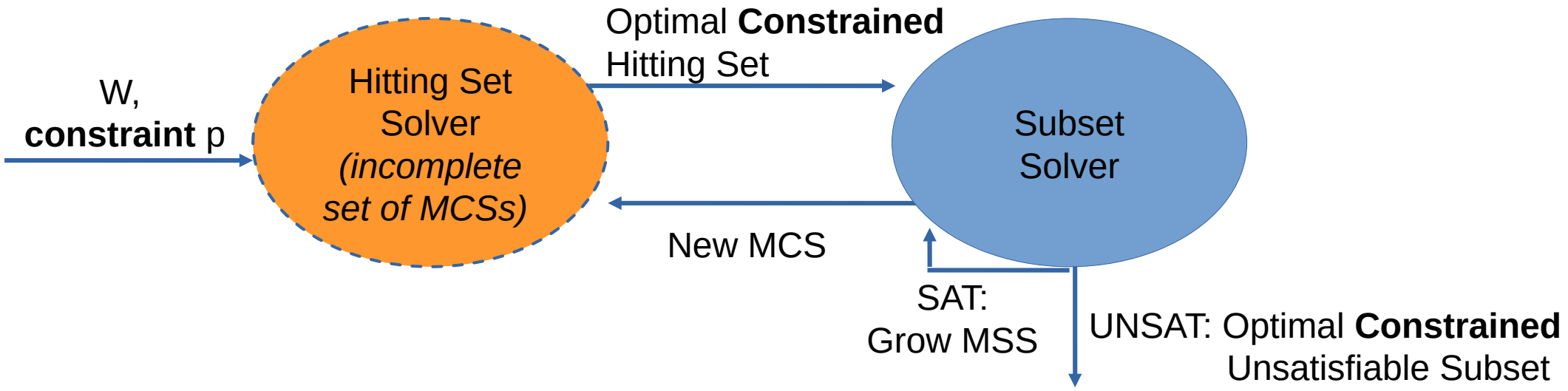
We use OUS because we want the *smallest* not just a minimal one, and then we can put smaller weights on facts and larger weights on constraints.

Efficiently step-wise explanation of the maximal consequence?

Compute the OUS over all assignments in the maximal consequence at once, efficiently:

OCUS Optimal *Constrained* Unsatisfiable Subsets *(Gamba, Bogaerts, Guns, 2021)*.

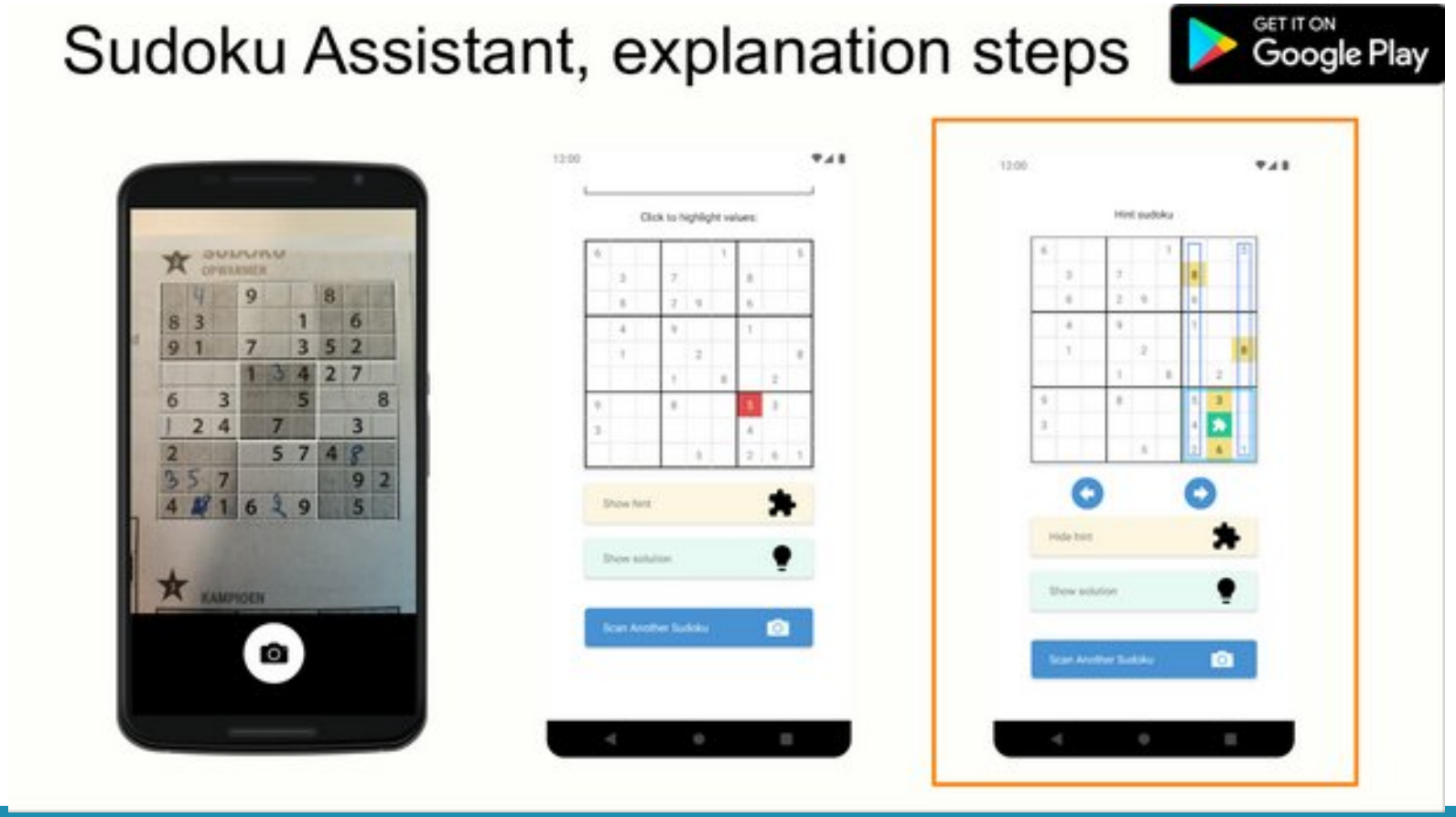
- *meta-constraint p*: use exactly 1 element of the maximal consequence



(not discussed in more detail)

Applications of explanation techniques

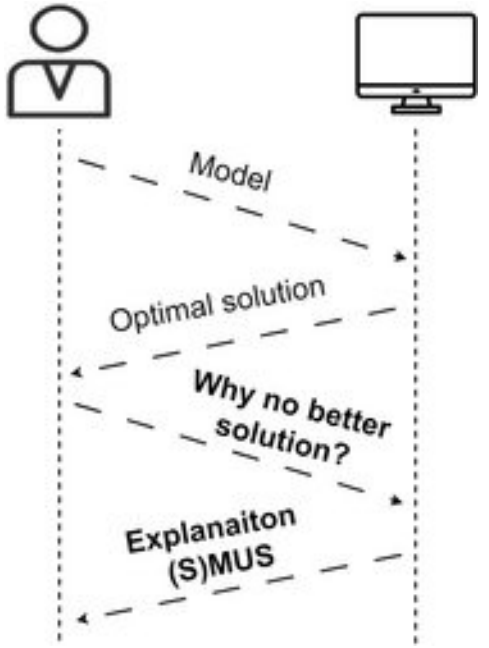
"Sudoku Assistant–an AI-Powered App to Help Solve Pen-And-Paper Sudokus". *Tias Guns, Emilio Gamba, Maxime Mulamba, Ignace Bleukx, Senne Berden, and Milan Pesa*. AAAI-2023 Best Demo Award.



Deductive Explanations for OPT problems

Deductive Explanations for OPT problems

Why is there no even better solution?



What constraints **causes** that there is no better solution?

- Reduce to UNSAT problem:
- Add better-than-optimal objective value as constraint: $C \wedge (f(x) < o) \models \bot$
- Then use deductive explanations for UNSAT!

Deductive Explanations for OPT problems

Can we explain *why* an optimal solution is optimal, e.g. why there does not exist a better solution?

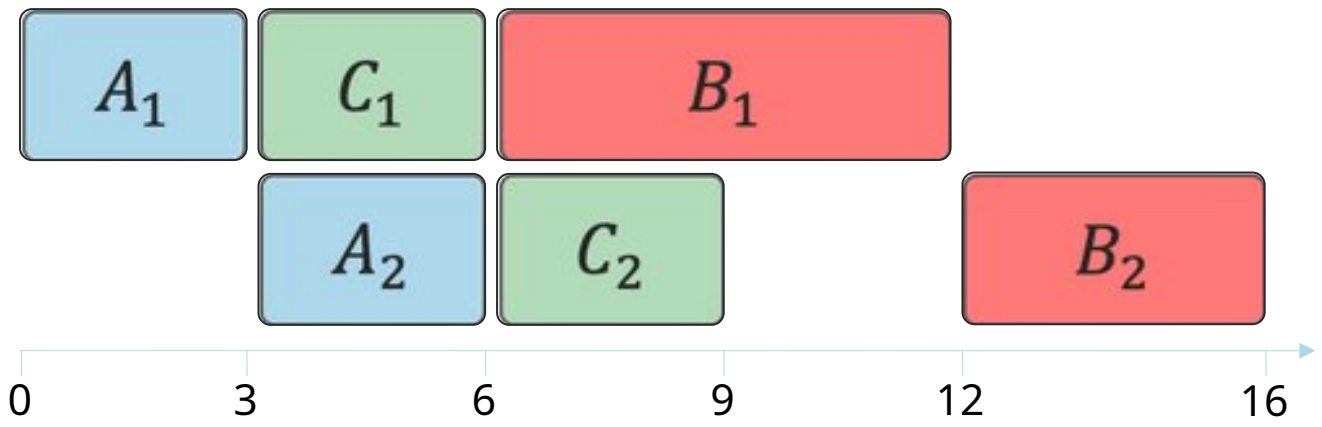
The MUS will typically be very large, upto all constraints…

Step-wise explanations for UNSAT to the rescue!

A *proof of optimality* proves that no better solution exists:

- An increasing number of solvers support *proof logging* (SAT, but also CP: Glasgow Constraint Solver, Pumpkin)
- These proofs are built for *computer* verification (up to gigabytes of log), not to communicate to users; so need trimming and projection on user constraints.

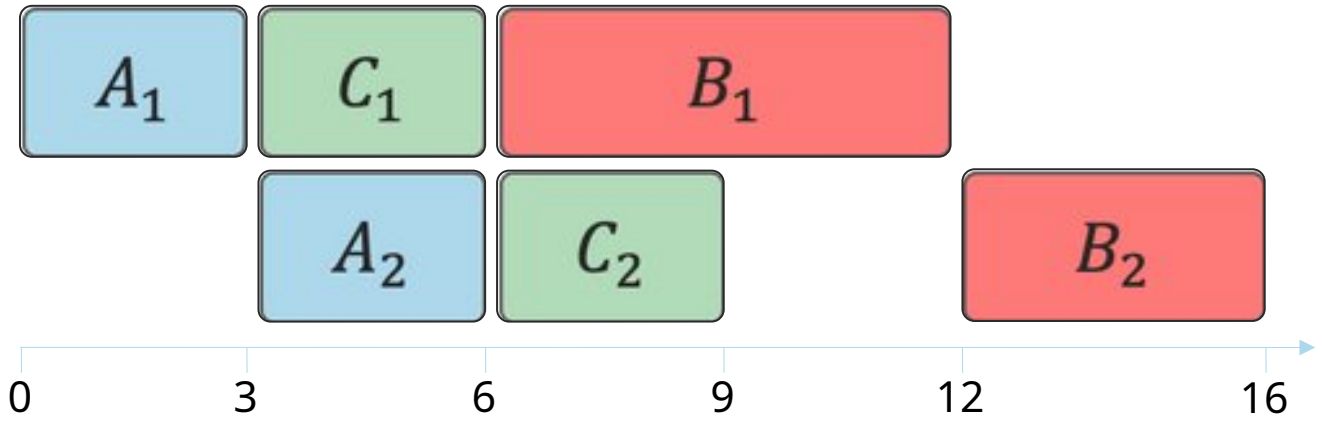
An example - jobshop



$$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$$
$$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$$
$$A_1 + 3 \leq A_2$$
$$B_1 + 6 \leq B_2$$
$$C_1 + 3 \leq C_2$$
$$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$$

“Why is there no solution with a shorter makespan?”

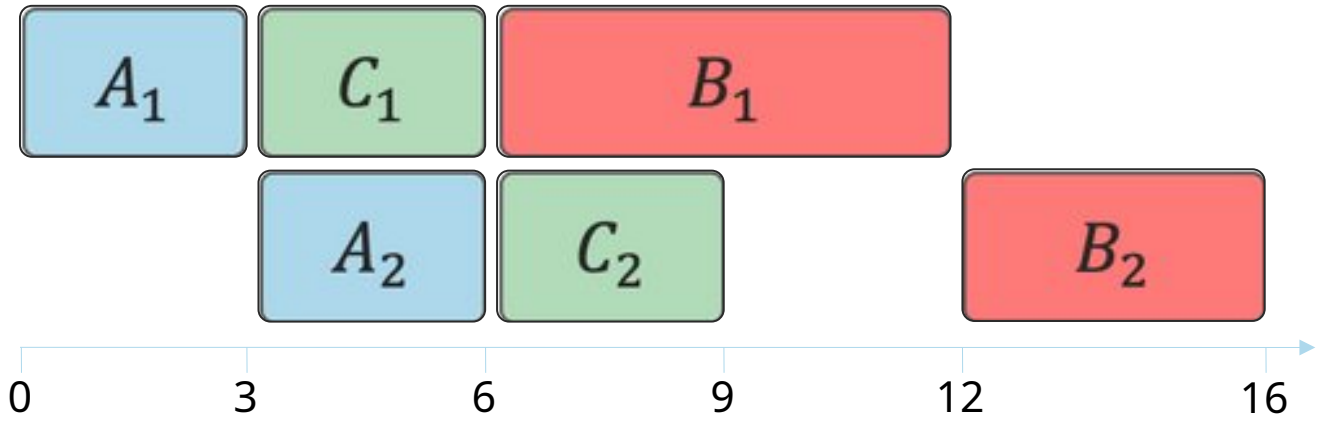
An example - jobshop



$$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$$
$$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$$
$$A_1 + 3 \leq A_2$$
$$B_1 + 6 \leq B_2$$
$$C_1 + 3 \leq C_2$$
$$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$$

$$makespan < 16$$
$$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$$
$$A_2 \leq 12, B_2 \leq 11, C_2 \leq 12$$

An example - jobshop



$$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$$
$$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$$
$$A_1 + 3 \leq A_2$$
$$B_1 + 6 \leq B_2$$
$$C_1 + 3 \leq C_2$$
$$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$$

$makespan < 16$

$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$

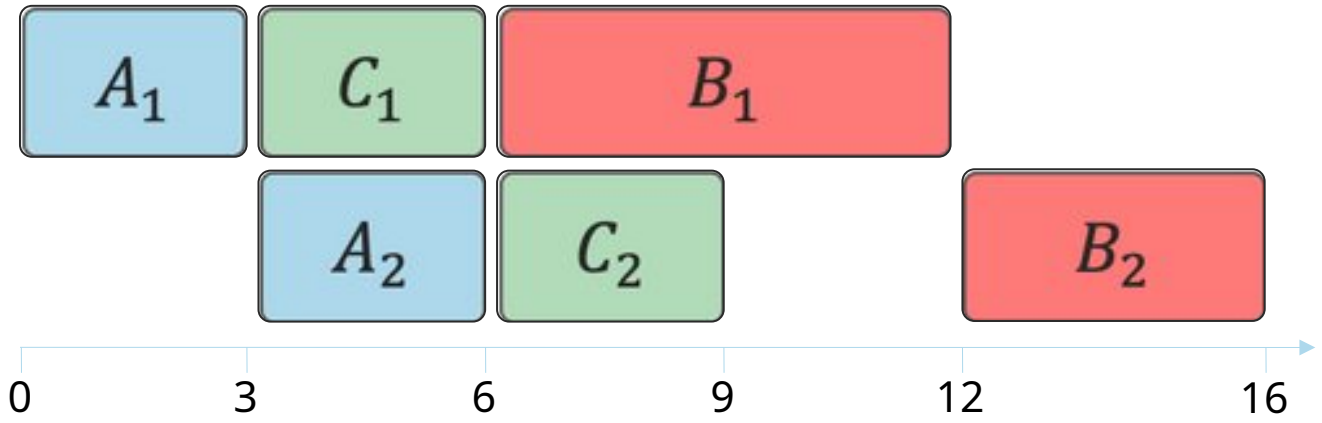
$A_2 \leq 12, B_2 \leq 11, C_2 \leq 12$

$B_1 \geq 0, B_2 \leq 11$

$B_1 + 6 \leq B_2$

$B_1 \leq 5, B_2 \geq 6$

An example - jobshop



$$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$$
$$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$$
$$A_1 + 3 \leq A_2$$
$$B_1 + 6 \leq B_2$$
$$C_1 + 3 \leq C_2$$
$$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$$

$makespan < 16$

$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$

$A_2 \leq 12, B_2 \leq 11, C_2 \leq 12$

$B_1 \geq 0, B_2 \leq 11$

$B_1 + 6 \leq B_2$

$B_1 \leq 5, B_2 \geq 6$

$A_2 \leq 12, 6 \leq B_2 \leq 11, C_2 \leq 12$

$A_1 + 3 \leq A_2$

$C_1 + 3 \leq C_2$

$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$

$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$

$B_1 \geq 3$

“What about problems with search?”

Peter Stuckey at the PhD defence of Ignace Bleukx

Step-wise explanations derive *domain reduction* using set of constraints

Sometimes need lots of constraints to derive domain reduction

Bad interpretability for large CSPs…

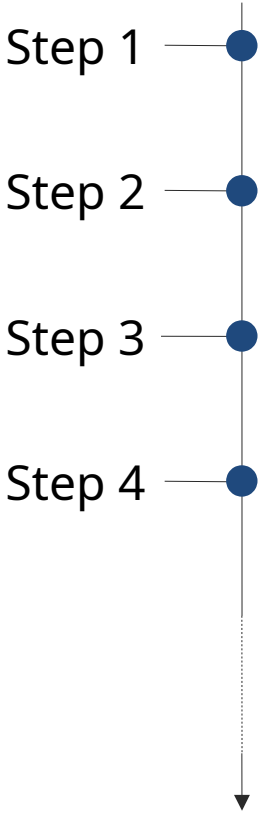
[Bleukx, Stuckey, Guns. CP 2026]

Generate step-wise explanations to explain **why large-scale** CSPs are **UNSAT**

Enabling us to compute nested explanations **until at most 1** (global) **constraint per step**

Experimental evaluation and comparison to **proof-size**

Solution: nested explanations [1]



Given a "complex" step:

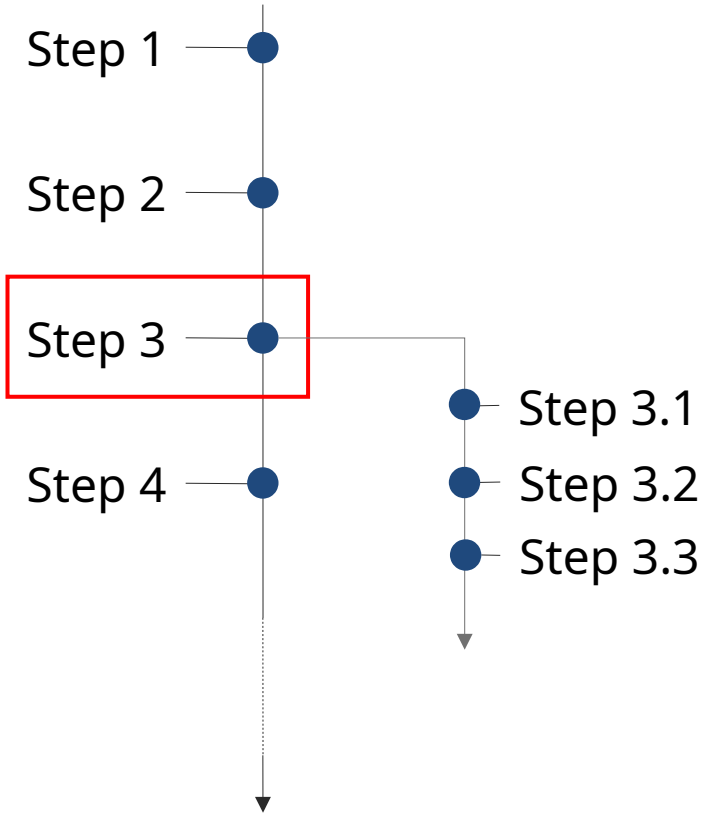
$$Input \wedge Constraints \Rightarrow Output$$

Recursively explain by contradiction

"Why is $Input \wedge Constraints \wedge \neg Output$ unsat?

[1] Bogaerts, Bart, Emilio Gamba, and Tias Guns. "A framework for step-wise explaining how to solve constraint satisfaction problems." Artificial Intelligence 2021

Solution: nested explanations [1]



Given a "complex" step:

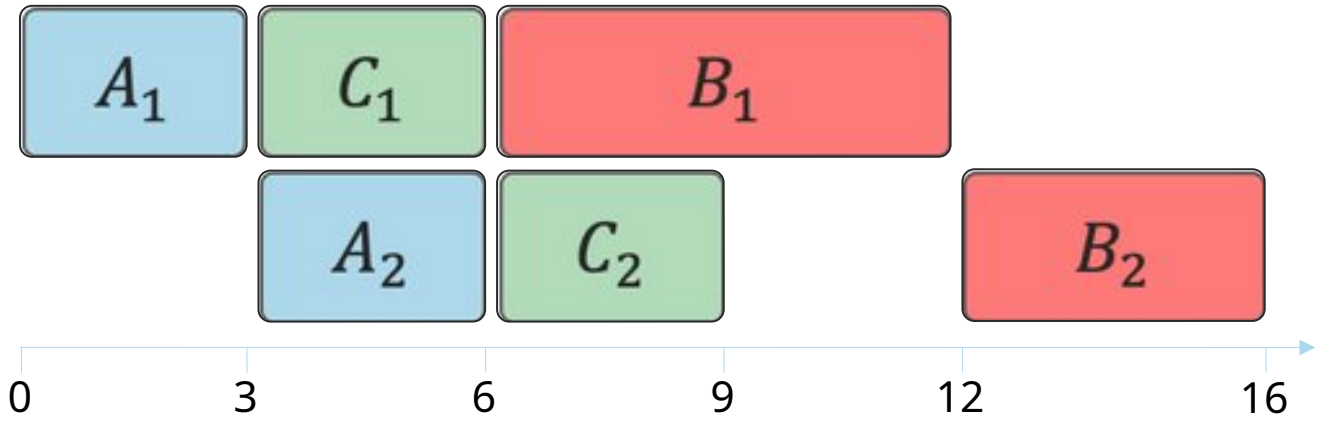
$$Input \wedge Constraints \Rightarrow Output$$

Recursively explain by contradiction

"Why is $Input \wedge Constraints \wedge \neg Output$ unsat?

[1] Bogaerts, Bart, Emilio Gamba, and Tias Guns. "A framework for step-wise explaining how to solve constraint satisfaction problems." Artificial Intelligence 2021

An example - jobshop



$$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$$
$$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$$
$$A_1 + 3 \leq A_2$$
$$B_1 + 6 \leq B_2$$
$$C_1 + 3 \leq C_2$$
$$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$$

Dom(A_i) = Dom(B_i) = Dom(C_i) = {0..30}

$makespan < 16$

$makespan = \max(A_2 + 3, B_2 + 4, C_2 + 3)$

$A_2 \leq 12, B_2 \leq 11, C_2 \leq 12$

$B_1 \geq 0, B_2 \leq 11$

$B_1 + 6 \leq B_2$

$B_1 \leq 5, B_2 \geq 6$

$A_2 \leq 12, 6 \leq B_2 \leq 11, C_2 \leq 12$

$A_1 + 3 \leq A_2$

$C_1 + 3 \leq C_2$

$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$

$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$

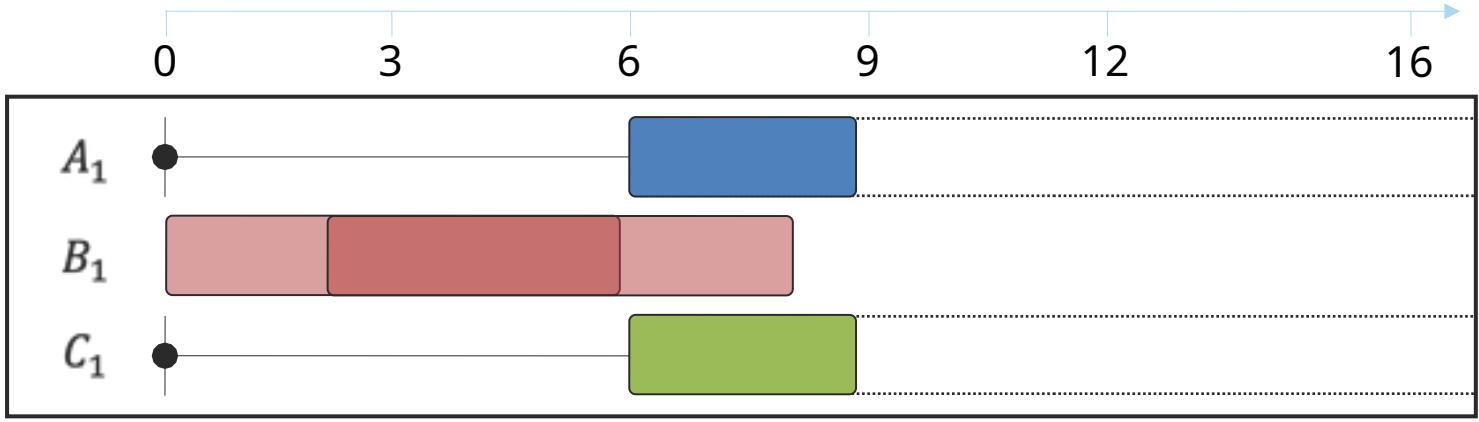
$B_1 \geq 3$

Example - nested

$B_1 < 3$

$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$

$A_1 \geq 6, C_1 \geq 6$



Example - nested

$B_1 < 3$

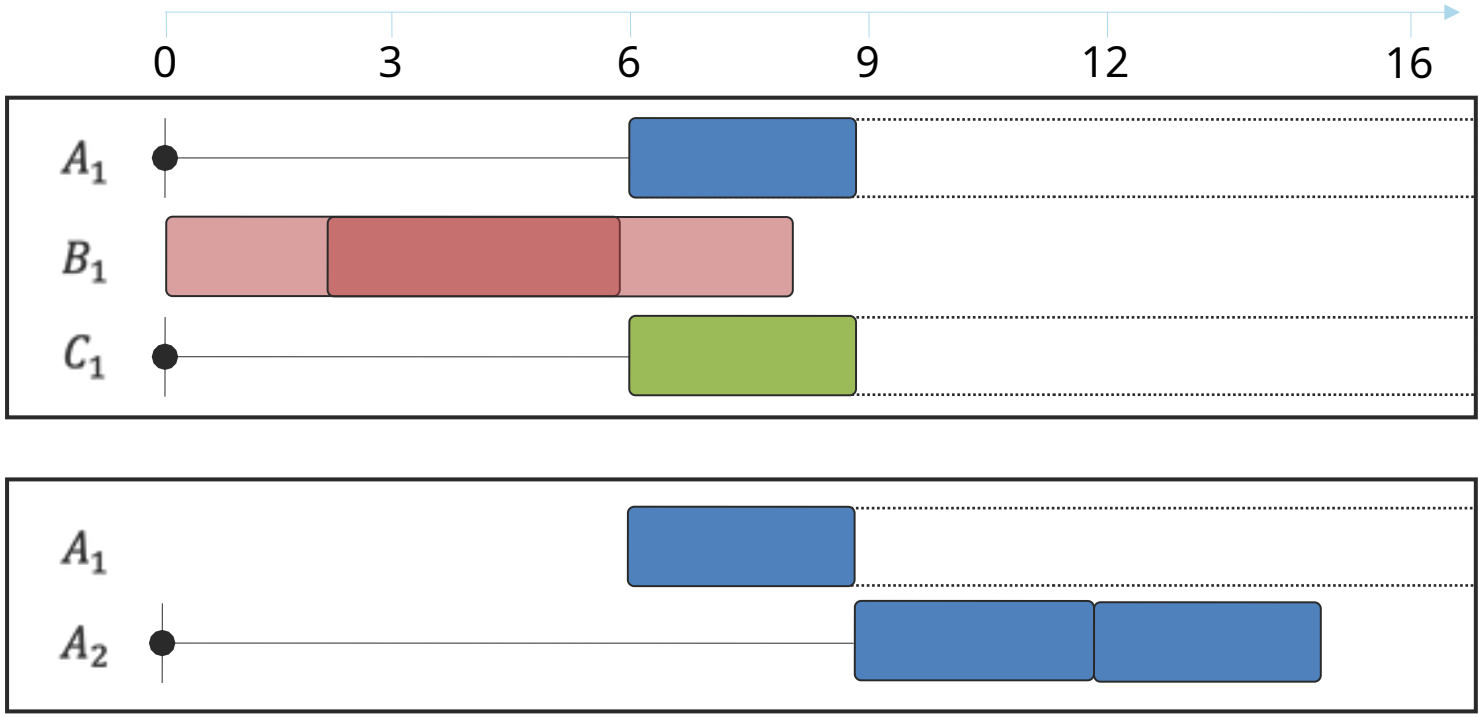
$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$

$A_1 \geq 6, C_1 \geq 6$

$A_1 \geq 6$

$A_1 + 3 \leq A_2$

$A_2 \geq 9$



Example - nested

$B_1 < 3$

$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$

$A_1 \geq 6, C_1 \geq 6$

$A_1 \geq 6$

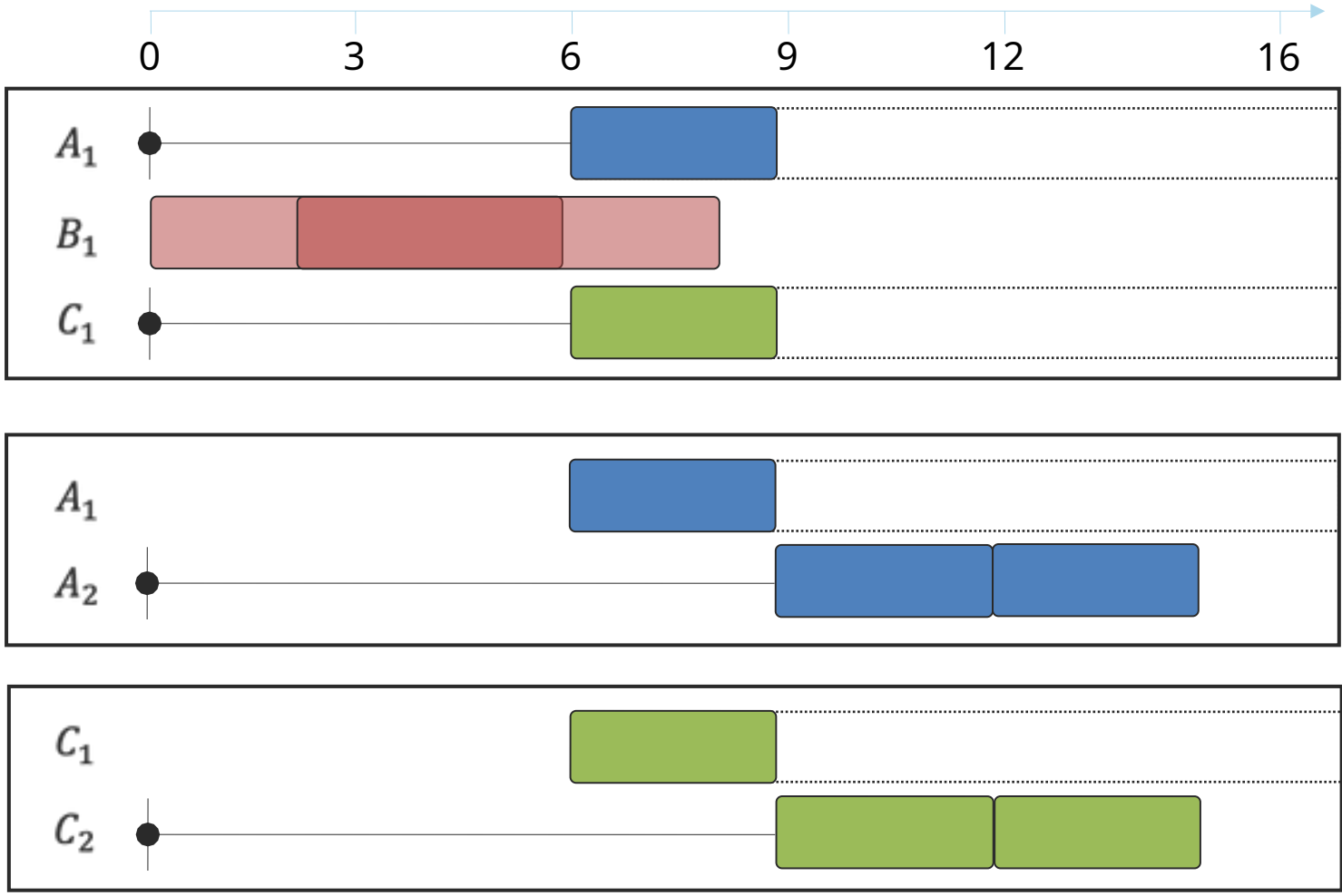
$A_1 + 3 \leq A_2$

$A_2 \geq 9$

$C_2 \geq 6$

$C_1 + 3 \leq C_2$

$C_2 \geq 9$



Example - nested

$B_1 < 3$

$NoOverlap((A_1, 3), (B_1, 6), (C_1, 3))$

$A_1 \geq 6, C_1 \geq 6$

$A_1 \geq 6$

$A_1 + 3 \leq A_2$

$A_2 \geq 9$

$C_2 \geq 6$

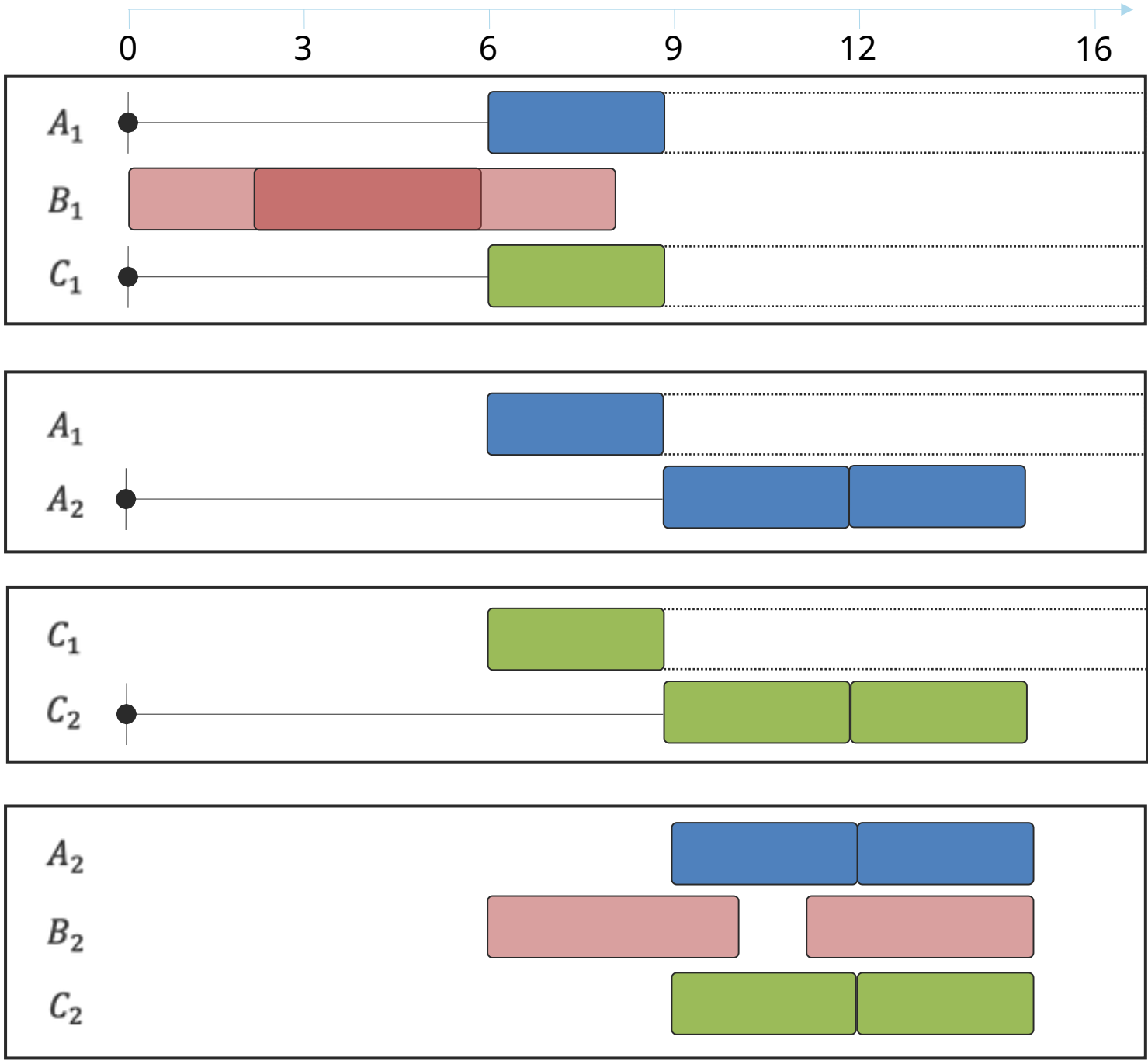
$C_1 + 3 \leq C_2$

$C_2 \geq 9$

$9 \leq A_2 \leq 12, 6 \leq B_2 \leq 11, 9 \leq C_2 \leq 12$

$NoOverlap((A_2, 3), (B_2, 4), (C_2, 3))$

$False$



Outline of the lectures

Introduction to XCP

Deductive explanations for UNSAT

Break

Deductive explanations for SAT/OPT

- Counterfactual explanations for UNSAT/SAT/OPT

Use case, challenges and conclusion

Explainable Constraint Programming (XCP)

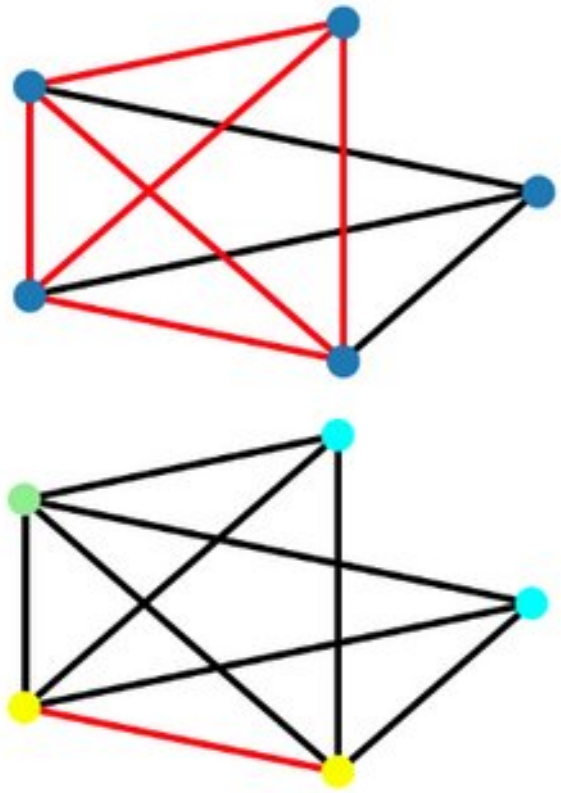
Recap, "**Why X?**" (with X a solution or UNSAT)

Deductive explanation:

- *What constraints cause X?*
- answer: a minimal inference set

Counterfactual explanation:

- *What to change if I want Y instead of X?*
- answer: a constraint relaxation + new solution



Counterfactual explanations to UNSAT problems

Resolving conflicts: "What to change if I want a solution instead of UNSAT?"

First idea: iteratively resolve each conflict

Diagnosis: *interactively* computing and resolving conflicts

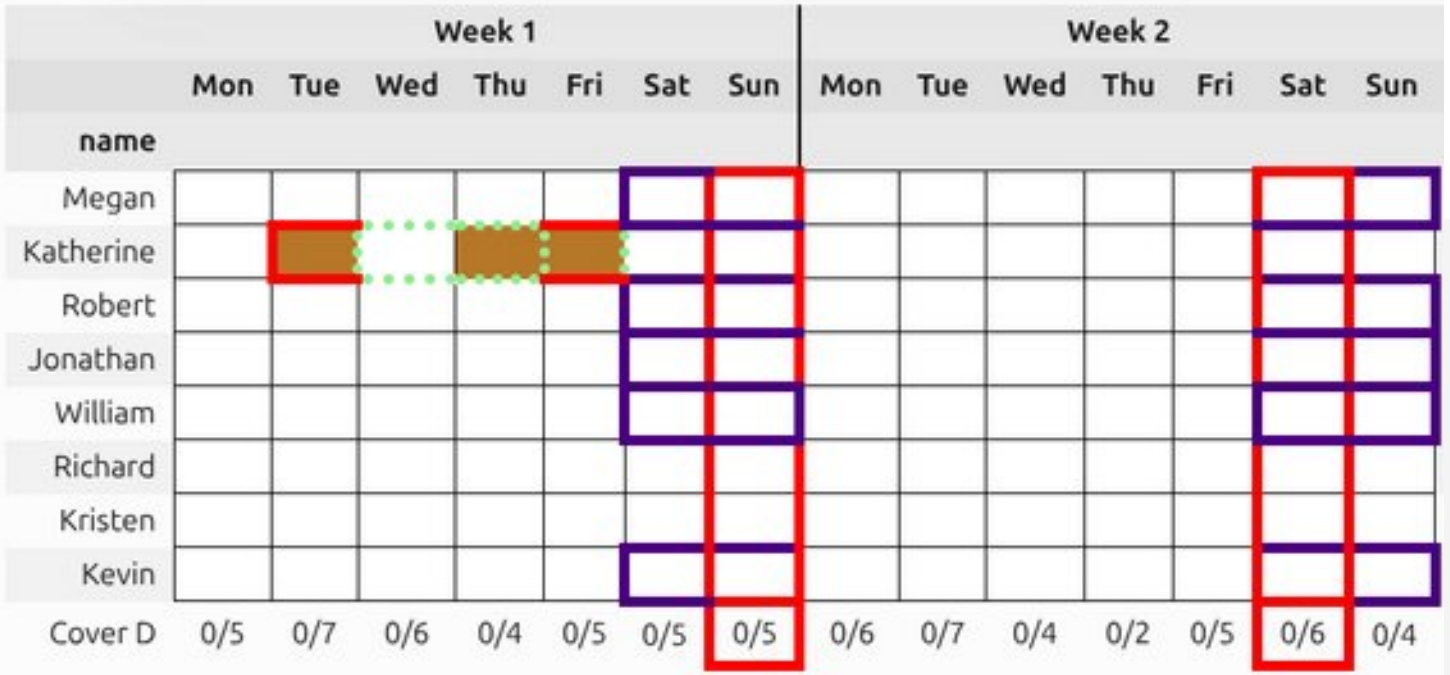
Reiter, Raymond. "A theory of diagnosis from first principles." Artificial intelligence 32.1 (1987): 57-95.

Interactive

```
In [38]:  def diagnos

              cons =
              while c
                  sub
                  pri
                  for

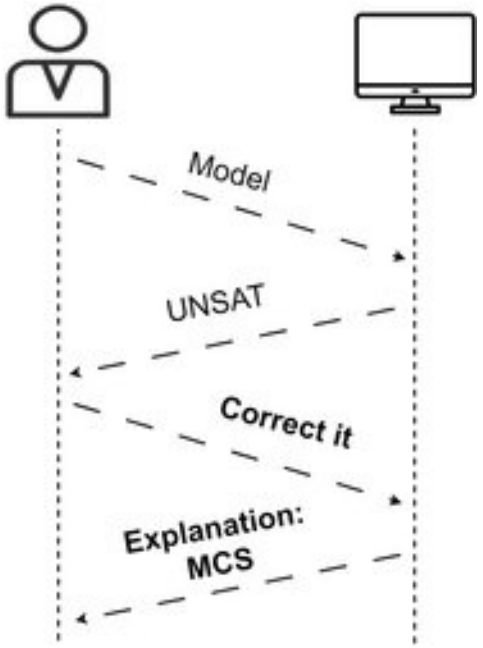
                  idx
                  # r
                  con
                  cle
```



	Week 1							Week 2						
name	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Megan														
Katherine														
Robert														
Jonathan														
William														
Richard														
Kristen														
Kevin														
Cover D	0/5	0/7	0/6	0/4	0/5	0/5	0/5	0/6	0/7	0/4	0/2	0/5	0/6	0/4

```
Constraints in conflict:
0. Jonathan should work at most 1 weekends
1. Katherine can work at most 5 days before having a day off
2. Katherine requests to work shift D on Fri 1
3. Katherine requests to work shift D on Thu 1
4. Katherine requests to work shift D on Tue 1
5. Katherine should have at least 2 consecutive days off
6. Katherine should have at least 2 consecutive days off
7. Kevin should work at most 1 weekends
8. Megan should work at most 1 weekends
9. Robert should work at most 1 weekends
10. Shift D on Sat 2 must be covered by 6 nurses out of 8
11. Shift D on Sun 1 must be covered by 5 nurses out of 8
12. William should work at most 1 weekends
and already removed constraints:
- Katherine has a day off on Sat 1
- Robert requests not to work shift D on Sat 2
```

Counterfactual explanation of an UNSAT problem



How to **change the model**, in order to find a solution?

Previous approach may require many iterations…

AND user only has **local** information, global resolution may be sub-optimal
Instead, directly find subset of *soft constraints* to **remove**

This is textbook Max-CSP solving!

Max-CSP solving

maximize $\sum w_i \cdot b_i$

st. $b_i \Rightarrow c_i$

```
In [40]: indicators = cp.boolvar(shape=len(model.constraints), name="ind")
         ind_model = cp.Model(indicators.implies(model.constraints))
         ind_model.maximize(indicators.sum())

         assert ind_model.solve() is True
         print(f"Found optimal solution with {ind_model.objective_value()} out of {len(ind_model.constraints)} constraints")
```

Found optimal solution with 356 out of 360 constraints

Finds the **maximum** (cardinality) satisfiable subset

```
In [41]: print("Removed these constraints:")
         for c in model.constraints:
             if c.value() is False:
                 print("-", c)

         print("Resulting solution:")
         visualize(nurse_view.value(), factory)
```

Removed these constraints:
- Robert has a day off on Tue 2
- Richard requests not to work shift D on Tue 2
- Shift D on Sat 1 must be covered by 5 nurses out of 8
- Shift D on Sun 1 must be covered by 5 nurses out of 8

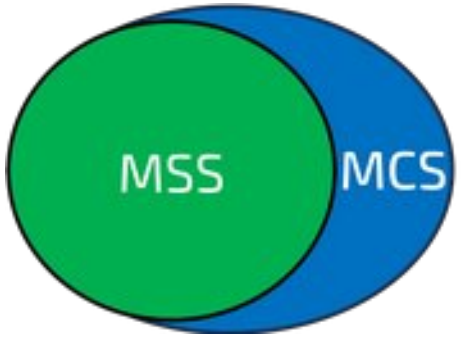
Resulting solution:

	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan	-	D	D	D	D	-	-	D	D	-	-	D	D	D	9	4320
Katherine	D	D	D	D	D	-	-	D	D	-	-	D	D	-	9	4320
Robert	D	D	D	D	D	-	-	D	D	D	-	-	-	-	8	3840
Jonathan	D	D	-	-	D	D	-	-	D	D	D	-	-	-	7	3360
William	-	D	D	-	-	-	-	D	D	-	-	D	D	D	7	3360
Richard	D	D	D	-	-	-	-	D	D	-	-	D	D	-	7	3360
Kristen	-	-	D	D	D	-	-	D	D	D	-	-	D	D	8	3840
Kevin	D	D	-	-	-	-	-	-	-	D	D	D	D	D	7	3360
Cover D	5/5	7/7	6/6	4/4	5/5	1/5	0/5	6/6	7/7	4/4	2/2	5/5	6/6	4/4	0	0

Max-CSP solving

Finds a **maximum** (cardinality) satisfiable subset
BUT may be hard to compute (e.g., stuck proving optimality)

Instead, find non-optimal, but **maximal** (superset) satisfiable subset, fast



We've seen this already! ('grow' in Marco):

- Start from the empty (satisfiable) set
- Grow the satisable subset by adding one constraint at a time
- Each constraint that causes UNSAT is in the MCS

Grow-based MCS

```
In [42]: def mcs_naive(constraints):
             mss = []  # grow a satisfiable subset one-by-one
             mcs = []  # everything else is in the minimum conflict set

             for cons in constraints:
                 if cp.Model(mss + [cons]).solve():
                     mss.append(cons)  # adding it remains SAT
                 else:
                     mcs.append(cons)  # UNSAT, causes conflict

             return mcs
```

By removing these constraints, the model becomes SAT:
- Shift D on Sun 1 must be covered by 5 nurses out of 8
- Shift D on Sat 1 must be covered by 5 nurses out of 8
- Richard requests not to work shift D on Tue 2
- Robert has a day off on Tue 2

	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan	-	D	D	D	D	-	-	D	D	-	-	D	D	-	8	3840
Katherine	D	D	D	D	D	-	-	D	D	-	-	-	D	D	9	4320
Robert	D	D	D	D	D	-	-	D	D	D	-	-	-	-	8	3840
Jonathan	D	D	-	-	D	D	-	-	D	D	D	D	-	-	8	3840
William	-	D	D	-	-	-	-	D	D	-	-	D	D	D	7	3360
Richard	D	D	D	-	-	-	-	D	D	D	-	-	D	D	8	3840
Kristen	-	-	D	D	D	-	-	D	D	-	-	D	D	-	7	3360
Kevin	D	D	-	-	-	-	-	-	-	D	D	D	D	D	7	3360
Cover D	5/5	7/7	6/6	4/4	5/5	1/5	0/5	6/6	7/7	4/4	2/2	5/5	6/6	4/4	0	0

(on this example,
it finds the same one)

Influencing which MCS is found?

- Lexicographic order over constraints: FastDiag is the MCS counterpart of QuickXplain

A. Felfernig, M. Schubert, and C. Zehentner. An efficient diagnosis algorithm for inconsistent constraint sets. AI EDAM, 26(1):53–62, 2012.

```
In [45]: # the order of 'soft' matters! lexicographic preference for the first ones
         def fastdiag_naive(soft, hard=[], solver="ortools"):
             model, soft, assump = make_assump_model(soft, hard)
             s = cp.SolverLookup.get(solver, model)
             assert s.solve(assumptions=assump) is False, "The model should be UNSAT!"

             # the recursive call
             def do_recursion(tocheck, other, delta):
                 print("@", end="")
                 if len(tocheck) != 0 and s.solve(assumptions=delta):
                     # constraints without tocheck is satisfiable, no need to recurse over other
                     return []

                 if len(other) == 1:
                     # only 1 'other' constraint remaining to search over
                     return list(other)  # base case of recursion

                 split = len(other) // 2  # determine split point
                 more_preferred, less_preferred = other[:split], other[split:]  # split constraints into two sets

                 # treat more preferred part as hard and find extra constraints in less preferred
                 delta1 = do_recursion(more_preferred, less_preferred, [c for c in delta if c not in set(more_preferred)])
                 # find which more preferred constraints exactly
                 delta2 = do_recursion(delta1, more_preferred, [c for c in delta if c not in set(delta1)])
                 return delta1 + delta2

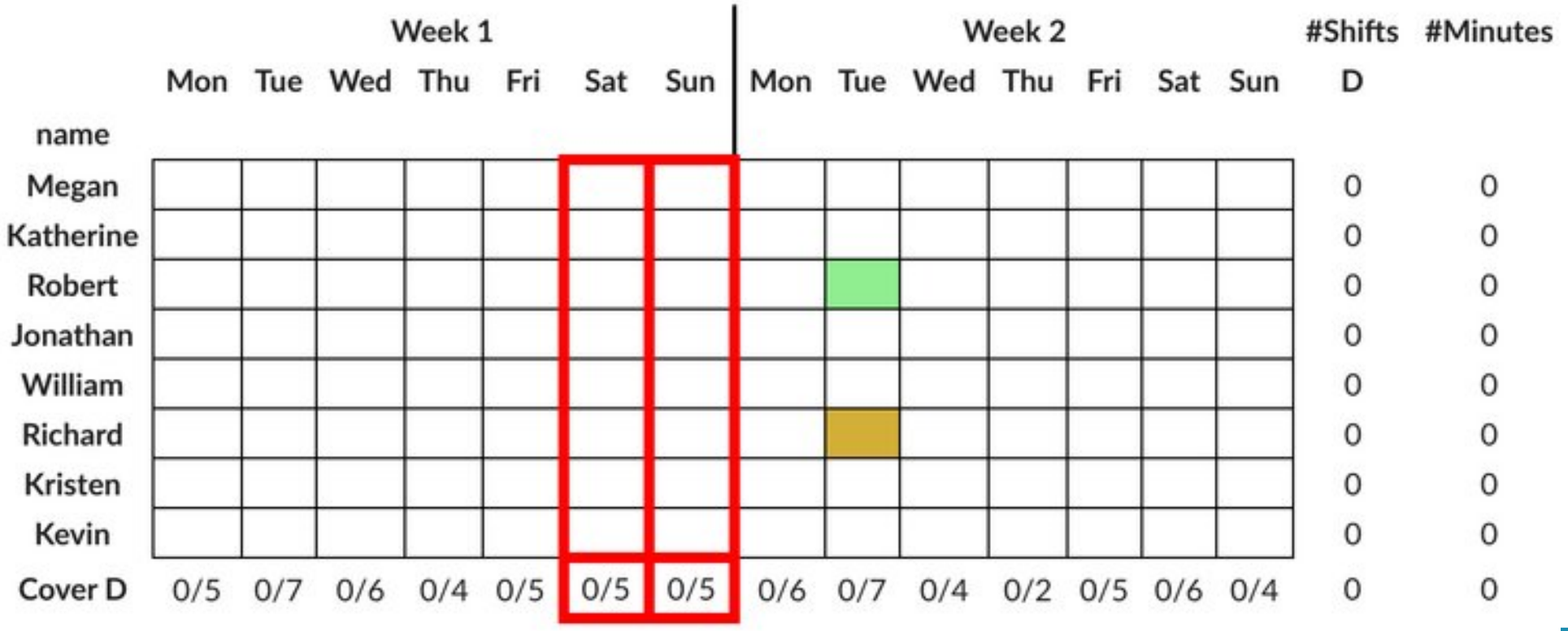
             mcs = do_recursion([], list(assump), list(assump))
             dmap = dict(zip(assump, soft))
             return [dmap[a] for a in mcs]
```

Counterfactual explanation of an UNSAT problem, alternative

A minimal correction **subset** corrects an unsatisfiable problem by **dropping** constraints.

If each constraint is a clause, that is probably the best you can do…

For rich applications, like nurse rostering, there is often a softer **degree of violation**



	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan															0	0
Katherine															0	0
Robert															0	0
Jonathan															0	0
William															0	0
Richard															0	0
Kristen															0	0
Kevin															0	0
Cover D	0/5	0/7	0/6	0/4	0/5	0/5	0/5	0/6	0/7	0/4	0/2	0/5	0/6	0/4	0	0

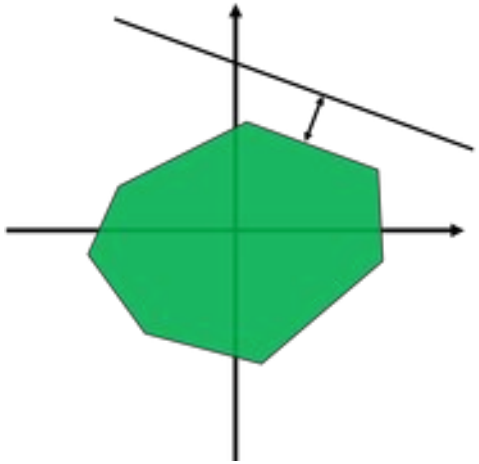
Counterfactual explanation of an UNSAT problem

Finding a ‘correction set’ requires defining **corrective actions**, e.g. constraint violations that allows for *relaxations* of constraints.

E.g. **feasbility restoration** by modifying rather then removing constraints

Senthooran I, Klapperstueck M, Belov G, Czauderna T, Leo K, Wallace M, Wybrow M, Garcia de la Banda M. Human-centred **feasibility restoration** in practice. Constraints. 2023 Jul 20:1-41.

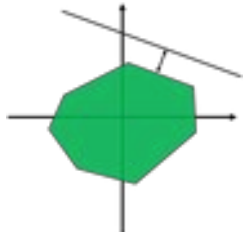
Types of relaxations of constraints:



- Boolean constraints: can only be turned on/off
- Numerical constraints: can be **violated** to some extend
  - Introduce slack for each numerical constraint
  - Slack indicates how much a constraint may be violated
    - = fine grained penalty of solution
  - Minimize (weighted) aggregate of slack values

Running example, nurse rostering

Slack-based relaxations



E.g., allow violation of *cover constraints* (nr of nurses on a specific day)
--> Allow days to be *slightly* under/overstaffed, minimize total violation

	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan	-	D	D	D	D	-	-	D	D	D	-	-	D	D	9	4320
Katherine	D	D	D	D	D	-	-	D	D	-	-	-	D	D	9	4320
Robert	D	D	D	D	D	-	-	-	-	D	D	D	-	-	8	3840
Jonathan	D	D	-	-	-	D	D	D	D	D	-	-	-	-	7	3360
William	-	D	D	D	D	-	-	D	D	-	-	D	D	D	9	4320
Richard	D	D	D	-	-	-	D	D	-	-	D	D	-	-	7	3360
Kristen	-	-	D	D	D	-	-	D	D	-	-	D	D	-	7	3360
Kevin	D	D	-	-	-	-	-	-	-	D	D	D	D	D	7	3360
Cover D	5/5	7/7	6/6	5/4	5/5	1/5	2/5	6/6	5/7	4/4	3/2	5/5	5/6	4/4	0	0
Slack under	[0]	[0]	[0]	[1]	[0]	[0]	[0]	[0]	[0]	[0]	[1]	[0]	[0]	[0]		
Slack over	[0]	[0]	[0]	[0]	[0]	[4]	[3]	[0]	[2]	[0]	[0]	[0]	[1]	[0]		

Running example, nurse rostering

Slack-based relaxations

Minimize nb of non-zero slacks constraints (similar to the MAX-CSP approach)

	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan	-	D	D	D	D	-	-	D	D	D	-	-	D	D	9	4320
Katherine	D	D	D	D	D	-	-	D	D	-	-	D	D	-	9	4320
Robert	D	D	D	D	D	-	-	-	-	D	D	-	-	-	7	3360
Jonathan	D	D	-	-	-	-	-	D	D	D	-	-	D	D	7	3360
William	-	D	D	-	-	-	-	D	D	-	-	D	D	D	7	3360
Richard	D	D	D	-	-	-	D	D	-	-	D	D	-	-	7	3360
Kristen	-	-	D	D	D	-	-	D	D	-	-	D	D	-	7	3360
Kevin	D	D	-	-	-	-	-	-	-	D	D	D	D	D	7	3360
Cover D	5/5	7/7	6/6	4/4	4/5	0/5	1/5	6/6	5/7	4/4	3/2	5/5	6/6	4/4	0	0
Slack under	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[1]	[0]	[0]	[0]		
Slack over	[0]	[0]	[0]	[0]	[1]	[5]	[4]	[0]	[2]	[0]	[0]	[0]	[0]	[0]		

Running example, nurse rostering

Slack-based relaxations

Or minimize any combination

```
In [51]: obj1 = cp.max(slack)        # minimize max violation
         obj2 = cp.sum(slack != 0)   # minimize nb of violations
         obj3 = cp.sum(slack)        # minimize global violation

         slack_model.minimize(10000 * obj1 + 1000 * obj2 + obj3) # multi-objective optimization
```

	Week 1							Week 2							#Shifts	#Minutes
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	D	
name																
Megan	-	D	D	D	D	-	-	D	D	-	-	D	D	D	9	4320
Katherine	D	D	D	D	D	-	-	D	D	-	-	-	D	D	9	4320
Robert	D	D	D	D	D	-	-	-	-	D	D	D	-	-	8	3840
Jonathan	D	D	-	-	-	D	D	D	D	D	-	-	-	-	7	3360
William	-	D	D	-	-	D	D	D	-	-	D	D	-	-	7	3360
Richard	D	D	D	-	-	-	D	D	-	-	D	D	-	-	7	3360
Kristen	-	-	D	D	D	-	-	D	D	D	-	-	D	D	8	3840
Kevin	D	D	-	-	-	-	-	-	-	D	D	D	D	D	7	3360
Cover D	5/5	7/7	6/6	4/4	4/5	2/5	3/5	6/6	4/7	4/4	4/2	5/5	4/6	4/4	0	0
Slack under	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[2]	[0]	[0]	[0]		
Slack over	[0]	[0]	[0]	[0]	[1]	[3]	[2]	[0]	[3]	[0]	[0]	[0]	[2]	[0]		

Recap: counterfactual explanations for UNSAT

MCS = Minimal Correction Subset

Algorithm	Characteristics	First solver call	Subsequent calls	Efficiency
Grow-based MCS	Trivial to implement, arbitrary MCS	One constraint	Linear increase	Efficient for assumption-based incremental solvers.
FastDiag	Optional lexicographic preference over constraints.	Half the constraints (n/2)	Divide and conquer	Reasonably efficient for non-incremental solvers.
Max-CSP	Optimal, e.g. weight per constraint	Reified constraints	/	Reified formulation can be more difficult to solve
Slack-based COP	Optimal, define slack-based relaxation for every constraint	Relaxed constraints	/	Adding slack decision variables can be more difficult to solve

Counterfactual Explanations for SAT problems

The problem is SATisfiable, and the solver returned a solution.

The user asks: "What if Y instead of X?"

Y is a foil: a partial assignment or constraint that is counter-factual, different from the returned solution.

Need to check C + Y, with C the set of constraints and Y the foil

- If C + Y is also SAT: show this solution
- If C + Y is UNSAT: can show a deductive or counterfactual explanation of why the foil leads to UNSAT

Counterfactual Explanations for SAT problems

Example where the user asks: "What if Y instead of X?"

```
In [44]: assert nurse_view[4,5].value()  # William currently scheduled to work on the first Saturday
         v = slack_model.objective_value()

         # what if William would not work on the first Saturday?
         mmodel = slack_model.copy()
         mmodel += (nurse_view[4,5] == 0)

         assert mmodel.solve()
         print("Total penalty: ", mmodel.objective_value(), "versus", v, "before.")
         style = visualize(slack_nurse_view.value(), factory, highlight_cover=True)
         style.data.loc["Slack under"] = list(slack_under.value()) + [" "]
         style.data.loc["Slack over"] = list(slack_over.value()) + [" "]
         display(style)
```

Total penalty: 41.2 versus 40.2 before.

	Week 1							Week 2							Total shifts
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	
name															
Megan	F	D	D	D	D	D	F	F	D	D	F	F	F	F	7
Katherine	D	D	D	D	D	F	F	D	D	F	F	F	D	D	9
Robert	D	D	D	D	D	F	F	F	F	D	D	D	F	F	8
Jonathan	D	D	F	F	F	D	D	D	D	D	F	F	F	F	7
William	F	D	D	D	D	F	F	D	D	F	F	D	D	D	9
Richard	D	D	D	F	F	F	D	D	F	F	D	D	F	F	7
Kristen	F	F	D	D	D	F	F	D	D	F	F	D	D	D	8
Kevin	D	D	F	F	F	F	F	F	F	D	D	D	D	D	7
Cover D	5/5	7/7	6/6	5/4	5/5	2/5	2/5	5/6	5/7	4/4	3/2	5/5	4/6	4/4	14
Slack under	0	0	0	0	0	3	3	1	2	0	0	0	2	0	
Slack over	0	0	0	1	0	0	0	0	0	0	1	0	0	0	

Counterfactual Explanations for OPT problems

- Corrective actions over the constraints? C ~∧ ~(f(x) < o) is UNSAT
  - => get counterfactual explanations from that.

- Corrective actions over the objective function coefficients:

The user asks: "What coefficients need to change so that Y becomes an optimal solution instead of X?"

Y is a foil from the optimisation perspective: it leads to a non-optimal solution.

[Korikov, Anton, and J. Christopher Beck. "Counterfactual explanations via inverse constraint programming." In 27th International Conference on Principles and Practice of Constraint Programming (CP 2021).]

Counterfactual Explanations for OPT problems

Find currently optimal solution X:

```
In [45]: model, nurse_view = factory.get_full_model()

         assert model.solve()
         print("Total penalty: ", model.objective_value())
         visualize(nurse_view.value(), factory)
```

Total penalty: 607

	Week 1							Week 2							Total shifts
	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	
name															
Megan	F	D	D	D	D	F	F	D	D	F	F	D	D	F	8
Katherine	D	D	D	D	D	F	F	F	D	D	F	F	D	D	9
Robert	D	D	D	F	F	D	D	D	F	F	D	D	F	F	8
Jonathan	D	D	F	F	F	D	D	D	D	D	F	F	F	F	7
William	F	D	D	D	D	F	F	D	D	F	F	D	D	D	9
Richard	D	D	D	F	F	F	F	D	D	D	F	F	D	D	8
Kristen	F	F	D	D	D	F	F	D	D	F	F	D	D	D	8
Kevin	D	D	F	F	D	D	D	F	F	D	D	D	F	F	8
Cover D	5/5	7/7	6/6	4/4	5/5	3/5	3/5	6/6	6/7	4/4	2/2	5/5	5/6	4/4	14

Counterfactual Explanations for OPT problems

Robert is unhappy!

```
In [48]: # slack variables can only be positive here (separate over and under relaxation)
         slack_under = cp.intvar(0, len(data.staff), shape=data.horizon, name="slack_under")
         slack_over = cp.intvar(0, len(data.staff), shape=data.horizon, name="slack_over")

         for _, cover in factory.data.cover.iterrows():
             # read the data
             day = cover["# Day"]
             shift = factory.shift_name_to_idx[cover["ShiftID"]]

             nb_nurses = cp.Count(nurse_view[:, day], shift)
             # deviation of `nb_nurses` from `requirement`
             expr = (nb_nurses == cover["Requirement"] - slack_under[day] + slack_over[day])
```

```
False       w:1     Robert's requests to work shift D on Mon 1 is denied
False       w:1     Robert's requests to work shift D on Tue 1 is denied
False       w:1     Robert's requests to work shift D on Wed 1 is denied
True        w:1     Robert's requests to work shift D on Thu 1 is denied
True        w:1     Robert's requests to work shift D on Fri 1 is denied
False       w:1     Robert's requests to not work shift D on Sat 2 is denied
False   w:1     Robert's requests to not work shift D on Sun 2 is denied
```

```
In [47]: desc = "Robert's requests to work shift D on Fri 1 is denied"
         weight,d_on_fri1 = next((w,pref) for w,pref in zip(*model.objective_.args) if str(pref) == desc)
         print(f"{d_on_fri1.value()} \t w:{w} \t{d_on_fri1}")
```

```
True    w:1     Robert's requests to work shift D on Fri 1 is denied
```

Counterfactual Explanations for OPT problems

Robert's request to work on Fri 1 is very important! His daughter has a surgery that day.
How should he minimally change *his* preferences to work that day?

```
In [49]: slack_model, slack_nurse_view, slack_under, slack_over = factory.get_slack_model()  # CMPpy Model

         slack = cp.cpm_array(np.append(slack_under, slack_over))
         slack_model.minimize(cp.sum(slack_under) + cp.sum(slack_over)) # minimize global violation

         # visualize
         assert slack_model.solve()
         style = visualize(slack_nurse_view.value(), factory, highlight_cover=True)
         style.data.loc["Slack under"] = list(slack_under.value()) + [" "," "]
         style.data.loc["Slack over"] = list(slack_over.value()) + [" ", " "]
         display(style)
```

Foil: {not([roster[2,4] == 1]): False}

Robert's other preferences:
- Weight 1 : Robert's requests to work shift D on Mon 1 is denied
- Weight 1 : Robert's requests to work shift D on Tue 1 is denied
- Weight 1 : Robert's requests to work shift D on Wed 1 is denied
- Weight 1 : Robert's requests to work shift D on Thu 1 is denied
- Weight 1 : Robert's requests to not work shift D on Sat 2 is denied
- Weight 1 : Robert's requests to not work shift D on Sun 2 is denied

Counterfactual Explanations for OPT problems

[Korikov, Anton, and J. Christopher Beck. "Counterfactual explanations via inverse constraint programming." In 27th International Conference on Principles and Practice of Constraint Programming (CP 2021).]

Algorithmically, it is a beautiful inverse optimisation problem with a multi-solver main/subproblem algorithm

Done! Found solution with total penalty 607, was 607

Robert should change the following preferences:
- set to weight: 0 -- Robert's requests to not work shift D on Sat 2 is denied

Outline of the lectures

Introduction to XCP

Deductive explanations for UNSAT

Break

Deductive explanations for SAT/OPT

Counterfactual explanations for UNSAT/SAT/OPT

- Use case, challenges and conclusion

The Challenge:
Workforce Allocation at Airbus





Airbus manufacturing sites

Toulouse/Hamburg/…

Assigning worker teams to logistical tasks

e.g., transporting aircraft parts

Transportation “tasks”

Equipment, duration, deadlines …

Current situation

AIRBUS



Team of human planners:

1. Schedules tasks (±500/day)
2. Assigns teams to each task

While considering

Working team qualifications + calendars,

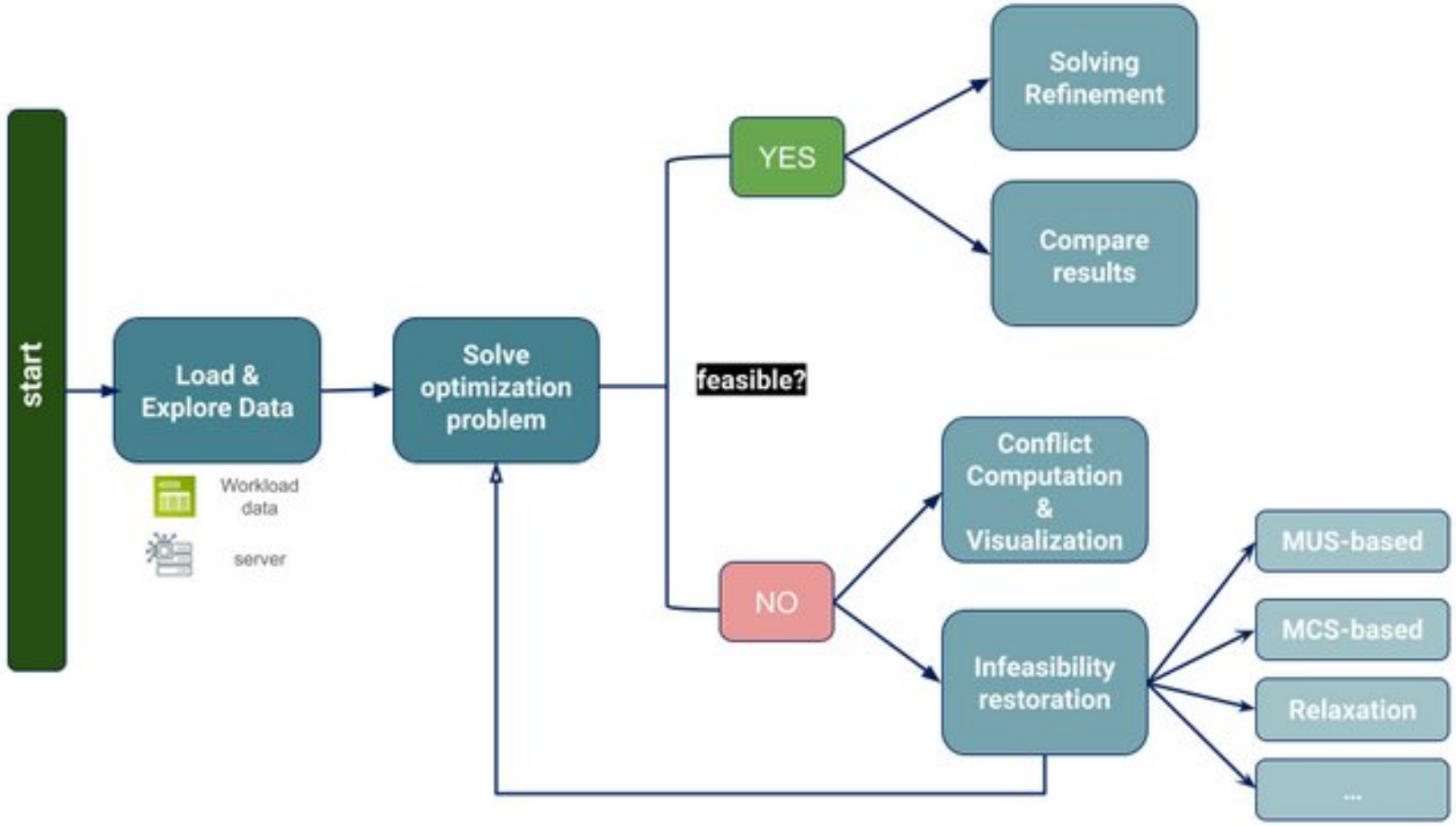
Manufacturing deadlines,

Task precedences,

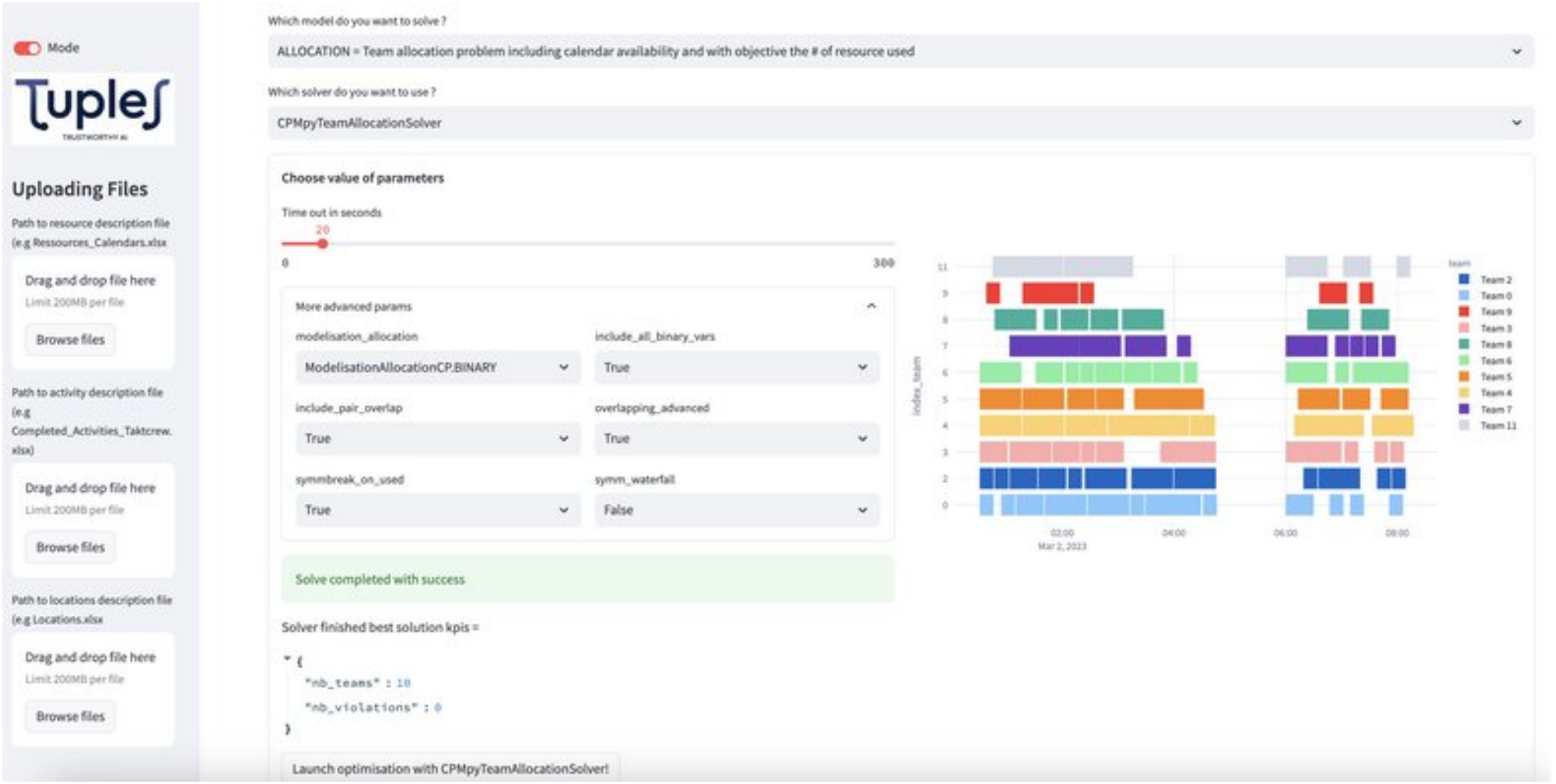
High-level logistics

...

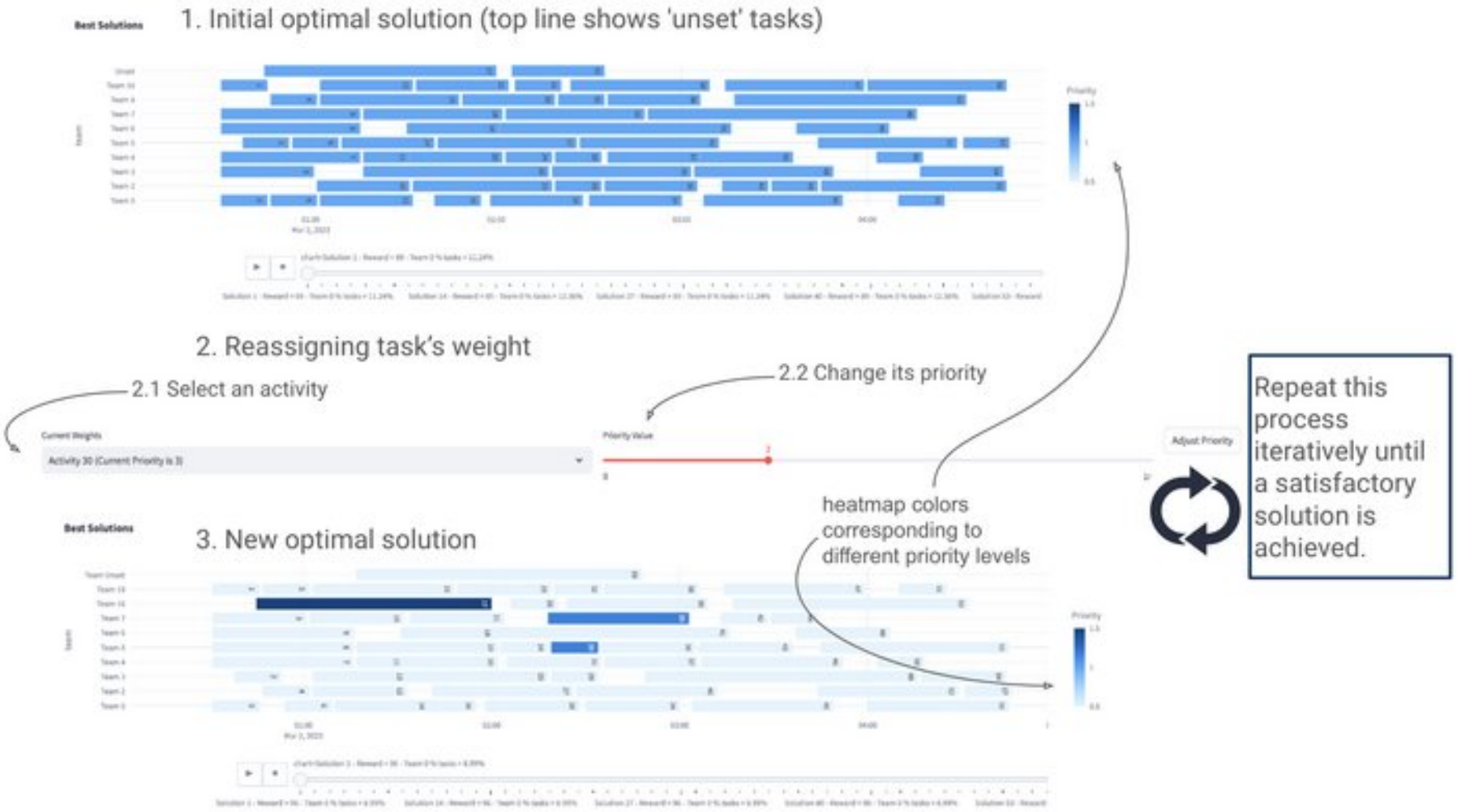
Use Case: Airbus Workforce Allocation example



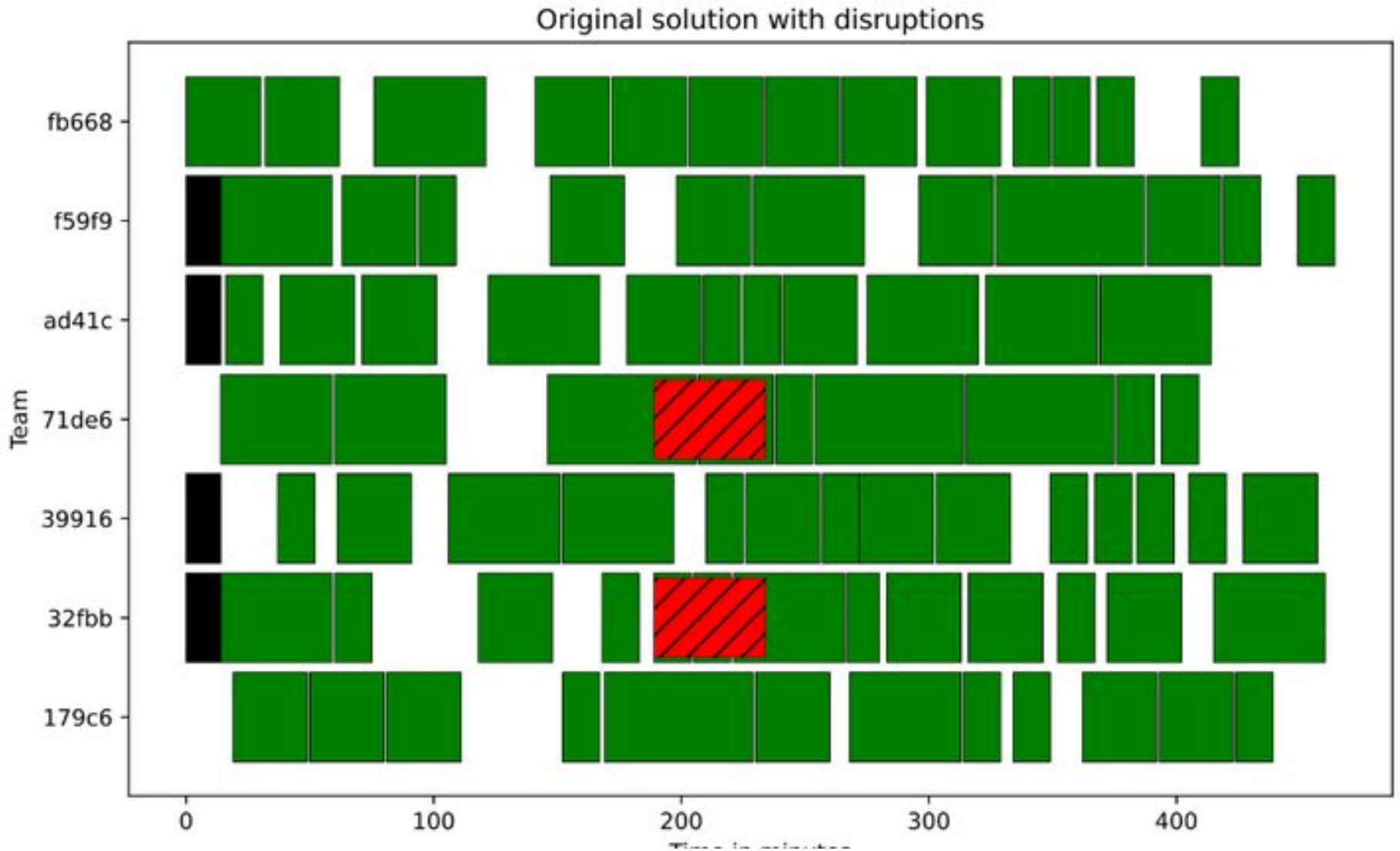
Airbus Workforce Allocation example, solving



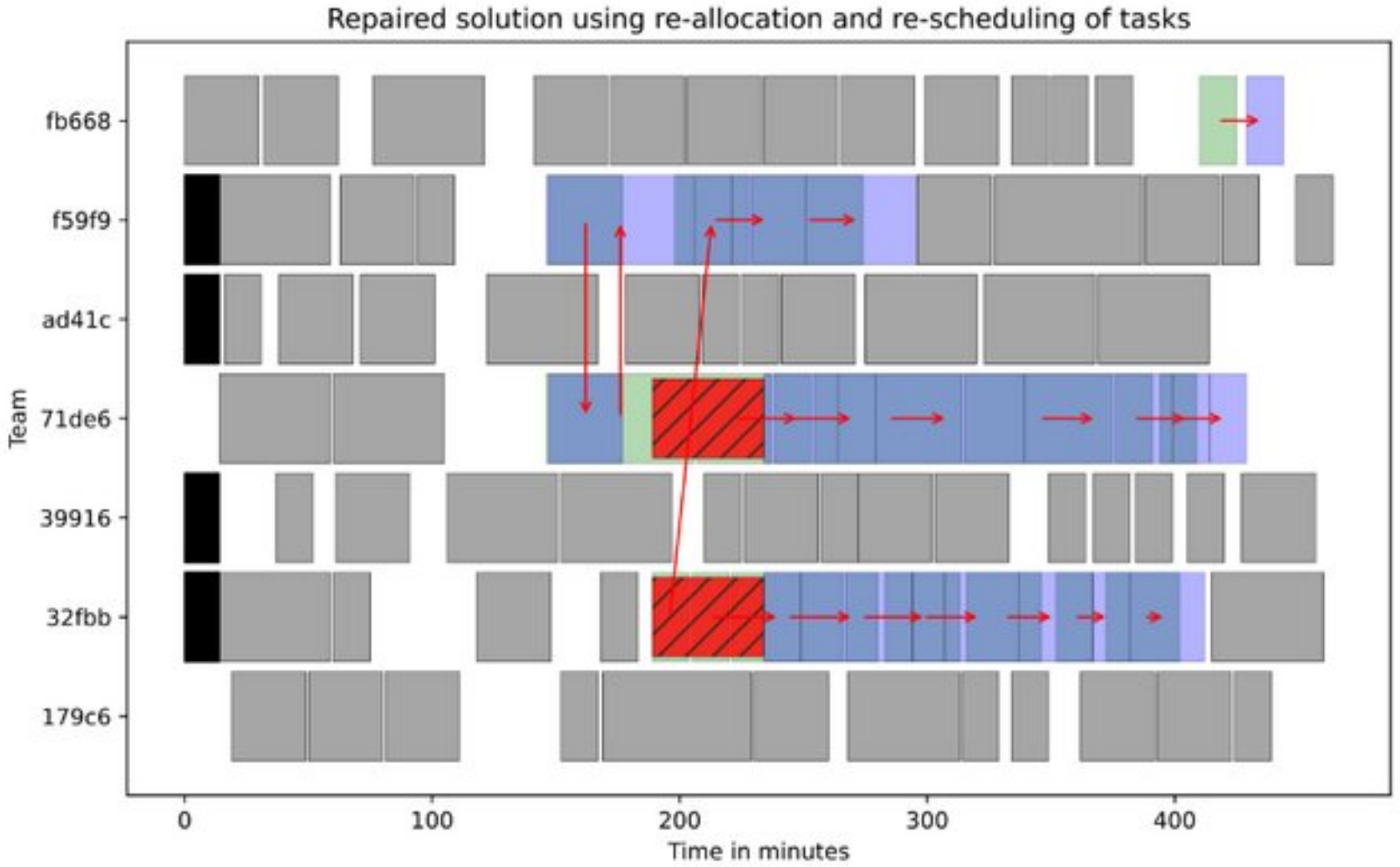
Airbus Workforce Allocation example, reviewing



Disrupted schedule



Fine-grained repair of schedules



Optimize:

1. #tasks allocated
2. #tasks re-allocated
3. #tasks re-scheduled
4. Max-shift of tasks
5. Avg-shift of tasks

Explainable Constraint Programming (XCP)

Recurring challenges:

- **Definition** of explanation: *question and answer format*

- Computational efficiency, **incremental** solvers

- Explanation **selection**: *which explanation to show; learn preferences?*
- User **Interaction**? *(visualisation, conversational, stateful, ...)*

- Explanation **evaluation**: *computational, formal, user survey, user study, ...*

  "Towards a Comprehensive Human-Centred Evaluation Framework for Explainable AI, XAI conference 2023"

Connections to wider XAI

"*Someone* explains *something* to *someone*"

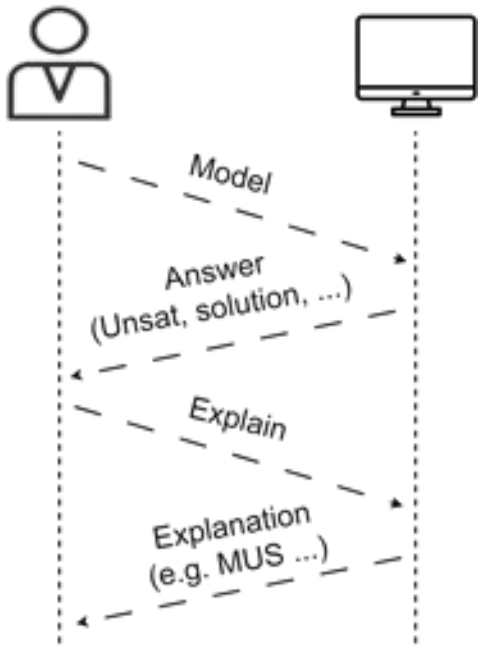
- Explanations in planning, e.g. MUGS *[Eiflet et al]*, Model Reconciliation *[Chakraborti et al]*, ...
- Explanations for KR/justifications *[Swartout et al]*, ASP *[Fandinno et al]*, in OWL *[Kalyanpur et al]*, ...
- Formal explanations of ML models (e.g. impl. hitting-set based, *[Ignatiev et al]*)

Connections beyond XAI:

- In (Mixed Integer) Linear Programming, MUS is called 'Infeasible Irreducible Subset'
  they can use that getting an IIS from an unsatisfiable Linear Program is easy

Conclusion (final slide)

- Deductive and Constrastive Explanation of UNSAT/SAT/Opt
- Deductive explanations relate back to finding a MUS/OUS
- XCP requires programmable (multi-solver) tooling (here: CPMpy)



- CP great fit for user-oriented explanations (high-level languages)
- More and more incremental and prooflogging CP solvers…
- Step-wise explanations are a useful user-oriented explanation type
- Less developed: counterfactual and interactive methods
- Even less developed: the role LLM’s can play in this

- Many open challenges and new problems!

Want to learn more?

Tutorial with notebooks available at

https://github.com/CPMpy/XCP-explain



References mentioned (many more exist!!!)

MUS

- Liffiton, M. H., & Sakallah, K. A. (2008). Algorithms for computing minimal unsatisfiable subsets of constraints. Journal of Automated Reasoning, 40, 1-33.
- Ignatiev, A., Previti, A., Liffiton, M., & Marques-Silva, J. (2015, August). Smallest MUS extraction with minimal hitting set dualization. In International Conference on Principles and Practice of Constraint Programming (pp. 173-182). Cham: Springer International Publishing.
- Joao Marques-Silva. Minimal Unsatisfiability: Models, Algorithms and Applications. ISMVL 2010. pp. 9-14

Feasibility restoration

- Senthooran, I., Klapperstueck, M., Belov, G., Czauderna, T., Leo, K., Wallace, M., ... & De La Banda, M. G. (2021). Human-centred feasibility restoration. In 27th International Conference on Principles and Practice of Constraint Programming (CP 2021). Schloss Dagstuhl-Leibniz-Zentrum für Informatik.

Explaining optimization problems

- Korikov, A., & Beck, J. C. (2021). Counterfactual explanations via inverse constraint programming. In 27th International Conference on Principles and Practice of Constraint Programming (CP 2021). Schloss Dagstuhl-Leibniz-Zentrum für Informatik.

Explanation in planning, ASP, KR

- Eifler, Rebecca, Michael Cashmore, Jörg Hoffmann, Daniele Magazzeni, and Marcel Steinmetz. "A new approach to plan-space explanation: Analyzing plan-property dependencies in oversubscription planning." In Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, no. 06, pp. 9818-9826. 2020.
- Chakraborti, Tathagata, Sarath Sreedharan, Yu Zhang, and Subbarao Kambhampati. "Plan explanations as model reconciliation: moving beyond explanation as soliloquy." In Proceedings of the 26th International Joint Conference on Artificial Intelligence, pp. 156-163. 2017.